

Physlib Monthly Report

1–30 June 2025

Contributors: ben, Eric Wieser, Jeremy Lindsay, Joseph Tooby-Smith, Tomáš Skřivan, Zhi Kai Pong

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit `e8fbff6`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 June 2025 on branch `master` of [leanprover-community/physlib](#), between commits `e41a3c8` and `e8fbff6`. Over that period 6 contributors landed 47 commits, changing 18,556 lines across 178 files and adding 797 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Basic</code>	3
2.2	<code>PhysLean.ClassicalMechanics.WaveEquation.HarmonicWave</code>	3
2.3	<code>PhysLean.Mathematics.Calculus.AdjFDeriv</code>	5
2.4	<code>PhysLean.Mathematics.DataStructures.FourTree.Basic</code>	8
2.5	<code>PhysLean.Mathematics.DataStructures.FourTree.UniqueMap</code>	12
2.6	<code>PhysLean.Mathematics.InnerProductSpace.Adjoint</code>	14
2.7	<code>PhysLean.Mathematics.InnerProductSpace.Basic</code>	16
2.8	<code>PhysLean.Mathematics.VariationalCalculus.Basic</code>	21
2.9	<code>PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv</code>	21
2.10	<code>PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint</code>	24
2.11	<code>PhysLean.Mathematics.VariationalCalculus.HasVarGradient</code>	26
2.12	<code>PhysLean.Mathematics.VariationalCalculus.IsTestFunction</code>	27
2.13	<code>PhysLean.Meta.Linters.Sorry</code>	29
2.14	<code>PhysLean.Optics.Polarization.Basic</code>	29
2.15	<code>PhysLean.Particles.SuperSymmetry.SU5.Charges.AllowsTerm</code>	31
2.16	<code>PhysLean.Particles.SuperSymmetry.SU5.Charges.Basic</code>	32
2.17	<code>PhysLean.Particles.SuperSymmetry.SU5.Charges.Completions</code>	36
2.18	<code>PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.Basic</code>	37
2.19	<code>PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.OfFinset</code>	39

2.20	PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimalSuperSet	41
2.21	PhysLean.Particles.SuperSymmetry.SU5.Charges.OfFieldLabel	42
2.22	PhysLean.Particles.SuperSymmetry.SU5.Charges.OfPotentialTerm	43
2.23	PhysLean.Particles.SuperSymmetry.SU5.Charges.PhenoConstrained	44
2.24	PhysLean.Particles.SuperSymmetry.SU5.Charges.Tree	46
2.25	PhysLean.Particles.SuperSymmetry.SU5.Charges.Yukawa	48
2.26	PhysLean.Particles.SuperSymmetry.SU5.FieldLabels	49
2.27	PhysLean.Particles.SuperSymmetry.SU5.Potential	50
2.28	PhysLean.QuantumMechanics.PlanckConstant	52
2.29	PhysLean.Relativity.LorentzGroup.Boosts.Generalized	52
2.30	PhysLean.Relativity.LorentzGroup.Orthochronous	55
2.31	PhysLean.Relativity.LorentzGroup.Restricted	56
2.32	PhysLean.Relativity.MinkowskiMatrix	56
2.33	PhysLean.Relativity.PauliMatrices.Matrix	57
2.34	PhysLean.Relativity.Tensors.RealTensor.Vector.Basic	57
2.35	PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic	61
2.36	PhysLean.SpaceAndTime.Space.Basic	63
2.37	PhysLean.SpaceAndTime.Space.VectorIdentities	63
2.38	PhysLean.StatisticalMechanics.BoltzmannConstant	64
2.39	PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic	64
2.40	PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState	68
2.41	PhysLean.StatisticalMechanics.Temperature	69
2.42	PhysLean.StringTheory.FTheory.SU5.Fluxes.Basic	70
2.43	PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndices . . .	73
2.44	PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completeness . . .	76
2.45	PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems	80
2.46	PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ToList	82
2.47	PhysLean.StringTheory.FTheory.SU5U1.Charges.OfRationalSection	84
2.48	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Completeness	84
2.49	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Basic	86
2.50	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Card	87
2.51	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.IsComplete	87
2.52	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Lemmas	87
2.53	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.NearestNeighbor	88
2.54	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.NextToNearestNeighbor	
2.55	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoConstrained	89
2.56	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoInsertQ10	90
2.57	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoInsertQ5	90
2.58	PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Same	91
2.59	PhysLean.StringTheory.FTheory.SU5U1.Quanta.AnomalyCancellation	91
2.60	PhysLean.StringTheory.FTheory.SU5U1.Quanta.Basic	93
2.61	PhysLean.StringTheory.FTheory.SU5U1.Quanta.FromParts	94
2.62	PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Basic	97
2.63	PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Cases	100
2.64	PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Completeness . . .	103
2.65	PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Elems	105
2.66	PhysLean.StringTheory.FTheory.SU5U1.Quanta.ToList	106
2.67	PhysLean.StringTheory.FTheory.SU5U1.Quanta.YukawaRegeneration	109
2.68	scripts.MetaPrograms.regex_lint	110
2.69	scripts.MetaPrograms.TODO_to_ym1	110

1 Introduction

During Jun 2025, 6 contributors submitted 47 commits that changed 18,556 lines (+13,492 / -5,064) across 178 files spanning 25 top-level areas of the repository. Of those, 59 files were newly added and 12 were removed outright. This report catalogues 797 newly added Lean declarations: 542 lemmas, 170 defs, 48 instances, 10 abbrevs, 8 inductives, 7 structures, 4 theorems, 2 classs, 2 axioms, 2 semiformal results, and 2 informal lemmas. We're delighted to recognize the following first-time contributors this month: [ben](#) and [Tomáš Skřivan](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
ben	1	1
Eric Wieser	77	1
Jeremy Lindsay	1,460	1
Joseph Tooby-Smith	18,492	21
Tomáš Skřivan	2,588	7
Zhi Kai Pong	812	16

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	11

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/StringTheory	37	+23	-10	+4,709	-3,636
PhysLean/Particles	15	+13	—	+3,007	-4
PhysLean/Mathematics	14	+10	—	+2,645	-3
PhysLean/Relativity	52	+2	-1	+1,432	-931
PhysLean/StatisticalMechanics	5	+4	-1	+470	-21
PhysLean/ClassicalMechanics	5	+1	—	+197	-110
scripts/MetaPrograms	4	+1	—	+260	-19
PhysLean/Electromagnetism	6	—	—	+127	-143
PhysLean/QuantumMechanics	8	+1	—	+96	-69
PhysLean/Meta	1	+1	—	+117	-0
PhysLean/Optics	1	+1	—	+115	-0
PhysLean/SpaceAndTime	6	+1	—	+104	-6
PhysLean.lean	1	—	—	+77	-32
README.md	1	—	—	+32	-23
PhysLean/QFT	10	—	—	+26	-24

scripts	3	—	—	+17	-11
lake-manifest.json	1	—	—	+12	-12
.github/ISSUE_TEMPLATE	1	—	—	+13	-2
PhysLean/Cosmology	1	—	—	+8	-6
PhysLean/CondensedMatter	1	—	—	+8	-4
.vscode	1	—	—	+10	-1
.github/workflows	1	—	—	+4	-4
lakefile.toml	1	—	—	+2	-2
.github	1	+1	—	+3	-0
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	542
def	170
instance	48
abbrev	10
inductive	8
structure	7
theorem	4
class	2
axiom	2
semiformal_result	2
informal_lemma	2

2 Details by file

2.1 PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

def EquationOfMotion

[\[source\]](#)

The definition of the equation of motion for the classical harmonic oscillator defined through the Euler-Lagrange equations.

```
@[sorryful]
def EquationOfMotion (x : Time → ℝ) : Prop
```

2.2 PhysLean.ClassicalMechanics.WaveEquation.HarmonicWave

[PhysLean/ClassicalMechanics/WaveEquation/HarmonicWave.lean](#) on GitHub.

abbrev WaveVector

[\[source\]](#)

The wavevector which indicates a direction and has magnitude $2\pi/\lambda$.

```
abbrev WaveVector (d : ℕ := 3)
```

def toDirection[\[source\]](#)*Direction of a wavevector.*

```
noncomputable def WaveVector.toDirection {d : ℕ} (k : WaveVector d) (h : k ≠ 0) :
  Direction d where
```

def harmonicWave[\[source\]](#)*General form of time-harmonic wave in terms of angular frequency ω and wave vector k .*

```
noncomputable def harmonicWave (a g : ℝ → Space d → ℝ) (ω : WaveVector d → ℝ) (k : WaveVector d
) :
  Time → Space d → ℝ
```

def transverseHarmonicPlaneWave[\[source\]](#)*Transverse monochromatic time-harmonic plane wave where the direction of propagation is taken to be `EuclideanSpace.single 2 1`. f_x and f_y are the respective amplitudes, ω is the angular frequency, δ_x and δ_y are the respective phases for f_x and f_y .*

```
@[nolint unusedArguments]
noncomputable def transverseHarmonicPlaneWave (k : WaveVector) (f_x f_y ω δ_x δ_y : ℝ)
(hk : k = EuclideanSpace.single 2 1 (ω/c)) :
  Time → Space → EuclideanSpace ℝ (Fin 3)
```

theorem equiv_symm_eq_zero_iff[\[source\]](#)*Pending #25552.*

```
@[simp]
theorem _root_.WithLp.equiv_symm_eq_zero_iff {p V} [AddCommGroup V] {v : V} :
  (WithLp.equiv p V).symm v = 0 ↔ v = 0
```

theorem equiv_eq_zero_iff[\[source\]](#)

```
@[simp]
theorem _root_.WithLp.equiv_eq_zero_iff {p V} [AddCommGroup V] {v : WithLp p V} :
  WithLp.equiv p V v = 0 ↔ v = 0
```

theorem single_eq_zero_iff[\[source\]](#)

```
@[simp]
theorem _root_.EuclideanSpace.single_eq_zero_iff {ι ℤ} [RCLike ℤ] [DecidableEq ι] {i : ι} {x :
ℤ} :
  EuclideanSpace.single i x = 0 ↔ x = 0
```

lemma transverseHarmonicPlaneWave_eq_planeWave[\[source\]](#)*The transverse harmonic planewave representation is equivalent to the general planewave expression with $\|k\| = \omega/c$.*

```

lemma transverseHarmonicPlaneWave_eq_planeWave {c : ℝ} {k : WaveVector} {f_x f_y ω δx δy : ℝ}
  (hc_ge_zero : 0 < c) (hω_ge_zero : 0 < ω) (hk : k = EuclideanSpace.single 2 (ω/c)) :
  (transverseHarmonicPlaneWave k f_x f_y ω δx δy hk) = planeWave
  (fun p => (f_x * Real.cos (-(ω/c)*p + δx)) • (EuclideanSpace.single 0 1) +
  (f_y * Real.cos (-(ω/c)*p + δy)) • (EuclideanSpace.single 1 1)) c
  (WaveVector.toDirection k (by rw [hk]; simp [ne_of_gt, hc_ge_zero, hω_ge_zero]))

```

2.3 PhysLean.Mathematics.Calculus.AdjFDeriv

[PhysLean/Mathematics/Calculus/AdjFDeriv.lean](#) on GitHub.

def adjFDeriv

[\[source\]](#)

Adjoint Fréchet derivative

$\text{adjFDeriv } \mathbb{K} f x = (\text{fderiv } \mathbb{K} f x).\text{adjoint}$

*The main purpose of this function is to compute gradient $f x = \text{adjFDeriv } \mathbb{K} f x 1$.
We provide easy to use API to compute adjFDeriv .*

noncomputable

def adjFDeriv (f : E → F) (x : E) (dy : F) : E

structure HasAdjFDerivAt

[\[source\]](#)

Function f has adjoint Fréchet derivative f' at x

$f' = \text{adjFDeriv } \mathbb{K} f x = (\text{fderiv } \mathbb{K} f x).\text{adjoint}$

The main purpose is to compute gradient $f x = f' 1 = \text{adjFDeriv } \mathbb{K} f x 1$.

This structure is analogous to HasFDerivAt and it is often easier to use as theorems for HasAdjFDeriv do not require differentiability assumptions unlike theorems for adjFDeriv .

@[fun_prop]

structure HasAdjFDerivAt (f : E → F) (f' : F → E) (x : E) **where**
 differentiableAt : DifferentiableAt $\mathbb{K} f x$
 hasAdjoint_fderiv : HasAdjoint $\mathbb{K} (\text{fderiv } \mathbb{K} f x) f'$

lemma HasAdjFDerivAt.adjFDeriv

[\[source\]](#)

protected lemma HasAdjFDerivAt.adjFDeriv {f : E → F} {f'} {x} (hf : HasAdjFDerivAt $\mathbb{K} f f' x$) :
 adjFDeriv $\mathbb{K} f x = f'$

lemma hasAdjFDerivAt

[\[source\]](#)

protected lemma DifferentiableAt.hasAdjFDerivAt [CompleteSpace E] [CompleteSpace F]
 {f : E → F} {x} (hf : DifferentiableAt $\mathbb{K} f x$) :
 HasAdjFDerivAt $\mathbb{K} f (\text{adjFDeriv } \mathbb{K} f x) x$ **where**

lemma isBoundedBilinearMap_real

[\[source\]](#)

lemma adjoint.isBoundedBilinearMap_real :
 IsBoundedBilinearMap $\mathbb{R}$ (**fun** (fy : (X → L[$\mathbb{R}$] Y) × Y) => fy.1.adjoint fy.2)

lemma contDiffAt_deriv[\[source\]](#)

```
protected lemma HasAdjFDerivAt.contDiffAt_deriv
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F] [InnerProductSpace' ℝ F]
  {G : Type*} [NormedAddCommGroup G] [NormedSpace ℝ G] [InnerProductSpace' ℝ G]
  [CompleteSpace F] [CompleteSpace G]
  {f : E → F → G} {f' : E → F → _} (hf : ∀ x y, HasAdjFDerivAt ℝ (f x) (f' x y) y)
  (hf' : ContDiff ℝ (n+1) (1f)) :
  ContDiff ℝ n (fun x : E×F×G => f' x.1 x.2.1 x.2.2)
```

lemma gradient_eq_adjFDeriv[\[source\]](#)

```
lemma gradient_eq_adjFDeriv
  {f : U → ℝ} {x : U} (hf : DifferentiableAt ℝ f x) :
  gradient f x = adjFDeriv ℝ f x 1
```

lemma hasAdjFDerivAt_id[\[source\]](#)

```
lemma hasAdjFDerivAt_id (x : E) : HasAdjFDerivAt ℝ (fun x : E => x) (fun dx => dx) x where
```

lemma adjFDeriv_id[\[source\]](#)

```
lemma adjFDeriv_id : adjFDeriv ℝ (fun x : E => x) = fun _ dx => dx
```

lemma hasAdjFDerivAt_const[\[source\]](#)

```
lemma hasAdjFDerivAt_const (x : E) (y : F) :
  HasAdjFDerivAt ℝ (fun _ : E => y) (fun _ => 0) x where
```

lemma adjFDeriv_const[\[source\]](#)

```
lemma adjFDeriv_const (y : F) : adjFDeriv ℝ (fun _ : E => y) = fun _ _ => 0
```

lemma comp[\[source\]](#)

```
lemma HasAdjFDerivAt.comp {f : F → G} {g : E → F} {f' g'} {x : E}
  (hf : HasAdjFDerivAt ℝ f f' (g x)) (hg : HasAdjFDerivAt ℝ g g' x) :
  HasAdjFDerivAt ℝ (fun x => f (g x)) (fun dz => g' (f' dz)) x where
```

lemma adjFDeriv_comp[\[source\]](#)

```
lemma adjFDeriv_comp [CompleteSpace E] [CompleteSpace F] [CompleteSpace G]
  {f : F → G} {g : E → F} {x : E}
  (hf : DifferentiableAt ℝ f (g x)) (hg : DifferentiableAt ℝ g x) :
  adjFDeriv ℝ (fun x => f (g x)) x = fun dy => adjFDeriv ℝ g x (adjFDeriv ℝ f (g x) dy)
```

lemma prodMk[\[source\]](#)

```
lemma HasAdjFDerivAt.prodMk {f : E → F} {g : E → G} {f' g'} {x : E}
  (hf : HasAdjFDerivAt ℝ f f' x) (hg : HasAdjFDerivAt ℝ g g' x) :
  HasAdjFDerivAt ℝ (fun x => (f x, g x)) (fun dyz => f' dyz.fst + g' dyz.snd) x where
```

lemma fst[\[source\]](#)

```
lemma HasAdjFDerivAt.fst {f : E → F×G} {f'} {x : E} (hf : HasAdjFDerivAt ℤ f f' x) :
  HasAdjFDerivAt ℤ (fun x => (f x).fst) (fun dy => f' (dy, 0)) x where
```

lemma adjFDeriv_fst[\[source\]](#)

```
lemma adjFDeriv_fst [CompleteSpace E] [CompleteSpace F] [CompleteSpace G]
  {f : E → F×G} {x : E} (hf : DifferentiableAt ℤ f x) :
  adjFDeriv ℤ (fun x => (f x).fst) x = fun dy => adjFDeriv ℤ f x (dy, 0)
```

lemma snd[\[source\]](#)

```
lemma HasAdjFDerivAt.snd {f : E → F×G} {f'} {x : E} (hf : HasAdjFDerivAt ℤ f f' x) :
  HasAdjFDerivAt ℤ (fun x => (f x).snd) (fun dz => f' (0, dz)) x where
```

lemma adjFDeriv_snd[\[source\]](#)

```
lemma adjFDeriv_snd [CompleteSpace E] [CompleteSpace F] [CompleteSpace G]
  {f : E → F×G} {x : E} (hf : DifferentiableAt ℤ f x) :
  adjFDeriv ℤ (fun x => (f x).snd) x = fun dy => adjFDeriv ℤ f x (0, dy)
```

lemma hasAdjFDerivAt_uncurry[\[source\]](#)

```
lemma hasAdjFDerivAt_uncurry {f : E → F → G} {xy} {fx' fy'}
  (hf : DifferentiableAt ℤ (λf) xy)
  (hfx : HasAdjFDerivAt ℤ (f · xy.2) fx' xy.1) (hfy : HasAdjFDerivAt ℤ (f xy.1 ·) fy' xy.2) :
  HasAdjFDerivAt ℤ (λf) (fun dz => (fx' dz, fy' dz)) xy where
```

lemma adjFDeriv_uncurry[\[source\]](#)

```
lemma adjFDeriv_uncurry [CompleteSpace E] [CompleteSpace F] [CompleteSpace G]
  {f : E → F → G} {xy} (hfx : DifferentiableAt ℤ (λf) xy) :
  adjFDeriv ℤ (λf) xy = fun dz => (adjFDeriv ℤ (f · xy.snd) xy.fst dz,
    adjFDeriv ℤ (f xy.fst ·) xy.snd dz)
```

lemma neg[\[source\]](#)

```
lemma HasAdjFDerivAt.neg {f : E → F} {f'} {x : E} (hf : HasAdjFDerivAt ℤ f f' x) :
  HasAdjFDerivAt ℤ (fun x => - f x) (fun dy => - f' dy) x where
```

lemma adjFDeriv_neg[\[source\]](#)

```
lemma adjFDeriv_neg [CompleteSpace E] [CompleteSpace F]
  {f : E → F} {x : E} (hf : DifferentiableAt ℤ f x) :
  adjFDeriv ℤ (fun x => - f x) x = fun dy => - adjFDeriv ℤ f x dy
```

lemma add[\[source\]](#)

```
lemma HasAdjFDerivAt.add {f g : E → F} {f' g'} {x : E}
  (hf : HasAdjFDerivAt ℤ f f' x) (hg : HasAdjFDerivAt ℤ g g' x) :
  HasAdjFDerivAt ℤ (fun x => f x + g x) (fun dy => f' dy + g' dy) x where
```

lemma adjFDeriv_add[\[source\]](#)

```

lemma adjFDeriv_add [CompleteSpace E] [CompleteSpace F]
  {f g : E → F} {x : E}
  (hf : DifferentiableAt ℚ f x) (hg : DifferentiableAt ℚ g x) :
  adjFDeriv ℚ (fun x => f x + g x) x = fun dy => adjFDeriv ℚ f x dy + adjFDeriv ℚ g x dy

```

lemma sub[\[source\]](#)

```

lemma HasAdjFDerivAt.sub
  {f g : E → F} {f' g'} {x : E}
  (hf : HasAdjFDerivAt ℚ f f' x) (hg : HasAdjFDerivAt ℚ g g' x) :
  HasAdjFDerivAt ℚ (fun x => f x - g x) (fun dy => f' dy - g' dy) x where

```

lemma adjFDeriv_sub[\[source\]](#)

```

lemma adjFDeriv_sub [CompleteSpace E] [CompleteSpace F] {f g : E → F} {x : E}
  (hf : DifferentiableAt ℚ f x) (hg : DifferentiableAt ℚ g x) :
  adjFDeriv ℚ (fun x => f x - g x) x = fun dy => adjFDeriv ℚ f x dy - adjFDeriv ℚ g x dy

```

lemma smul[\[source\]](#)

```

lemma HasAdjFDerivAt.smul {f : E → F} {g : E → ℚ} {f' g'}
  (hf : HasAdjFDerivAt ℚ f f' x) (hg : HasAdjFDerivAt ℚ g g' x) :
  HasAdjFDerivAt ℚ (fun x => g x • f x)
    (fun dy => conj (g x) • f' dy + g' (conj (inner ℚ dy (f x)))) x where

```

lemma adjFDeriv_smul[\[source\]](#)

```

lemma adjFDeriv_smul [CompleteSpace E] [CompleteSpace F]
  {f : E → F} {g : E → ℚ} {x : E}
  (hf : DifferentiableAt ℚ f x) (hg : DifferentiableAt ℚ g x) :
  adjFDeriv ℚ (fun x => g x • f x) x = fun dy => conj (g x) • adjFDeriv ℚ f x dy +
    adjFDeriv ℚ g x (conj (inner ℚ dy (f x)))

```

2.4 PhysLean.Mathematics.DataStructures.FourTree.Basic

[PhysLean/Mathematics/DataStructures/FourTree/Basic.lean](#) on GitHub.

inductive Leaf[\[source\]](#)

A leaf contains has the data of a term of type α_4 .

```

inductive Leaf ( $\alpha_4$  : Type)
| leaf :  $\alpha_4 \rightarrow$  Leaf  $\alpha_4$ 

```

inductive Twig[\[source\]](#)

A twig has the data of a term of type α_3 and a multiset of type Leaf α_4 .

```

inductive Twig ( $\alpha_3$   $\alpha_4$  : Type)
| twig :  $\alpha_3 \rightarrow$  Multiset (Leaf  $\alpha_4$ )  $\rightarrow$  Twig  $\alpha_3$   $\alpha_4$ 

```

inductive Branch[\[source\]](#)

A branch has the data of a term of type α_2 and a multiset of type $\text{Twig } \alpha_3 \alpha_4$.

```
inductive Branch (α2 α3 α4 : Type)
| branch : α2 → Multiset (Twig α3 α4) → Branch α2 α3 α4
```

inductive Trunk[\[source\]](#)

A trunk has the data of a term of type α_1 and a multiset of type $\text{Branch } \alpha_2 \alpha_3 \alpha_4$.

```
inductive Trunk (α1 α2 α3 α4 : Type)
| trunk : α1 → Multiset (Branch α2 α3 α4) → Trunk α1 α2 α3 α4
```

inductive FourTree[\[source\]](#)

A FourTree has the data of a multiset of type $\text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4$.

```
inductive FourTree (α1 α2 α3 α4 : Type)
| root : Multiset (FourTree.Trunk α1 α2 α3 α4) → FourTree α1 α2 α3 α4
```

instance Repr (Leaf α4)[\[source\]](#)

```
unsafe instance (α4 : Type) [Repr α4] : Repr (Leaf α4) where
```

instance Repr (Twig α3 α4)[\[source\]](#)

```
unsafe instance (α3 α4 : Type) [Repr α3] [Repr α4] : Repr (Twig α3 α4) where
```

instance Repr (Branch α2 α3 α4)[\[source\]](#)

```
unsafe instance (α2 α3 α4 : Type) [Repr α2] [Repr α3] [Repr α4] :
  Repr (Branch α2 α3 α4) where
```

instance Repr (Trunk α1 α2 α3 α4)[\[source\]](#)

```
unsafe instance (α1 α2 α3 α4 : Type) [Repr α1] [Repr α2] [Repr α3] [Repr α4] :
  Repr (Trunk α1 α2 α3 α4) where
```

instance Repr (FourTree α1 α2 α3 α4)[\[source\]](#)

```
unsafe instance (α1 α2 α3 α4 : Type) [Repr α1] [Repr α2] [Repr α3] [Repr α4] :
  Repr (FourTree α1 α2 α3 α4) where
```

def fromMultiset[\[source\]](#)

A FourTree from a multiset of $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$.

```
def fromMultiset {α1 α2 α3 α4 : Type} [DecidableEq α1]
  [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]
  (l : Multiset (α1 × α2 × α3 × α4)) : FourTree α1 α2 α3 α4
```

def toMultiset[\[source\]](#)

A FourTree to a multiset of $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$.

```
def toMultiset {α1 α2 α3 α4 : Type} (T : FourTree α1 α2 α3 α4) : Multiset (α1 × α2 × α3 × α4)
```

def Twig.card[\[source\]](#)

The cardinality of a Twig is the number of leafs.

```
def Twig.card {α3 α4 : Type} (T : Twig α3 α4) : Nat
```

def Branch.card[\[source\]](#)

The cardinality of a Branch is the total number of leafs.

```
def Branch.card {α2 α3 α4 : Type} (T : Branch α2 α3 α4) : Nat
```

def Trunk.card[\[source\]](#)

The cardinality of a Trunk is the total number of leafs.

```
def Trunk.card {α1 α2 α3 α4 : Type} (T : Trunk α1 α2 α3 α4) : Nat
```

def FourTree.card[\[source\]](#)

The cardinality of a FourTree is the total number of leafs.

```
def card {α1 α2 α3 α4 : Type} (T : FourTree α1 α2 α3 α4) : Nat
```

lemma card_eq_toMultiset_card[\[source\]](#)

```
lemma card_eq_toMultiset_card (T : FourTree α1 α2 α3 α4s) :  
  T.card = T.toMultiset.card
```

def Leaf.mem[\[source\]](#)

An element of $a : \alpha_4$ is a member of Leaf α_4 if the underlying element of the Leaf is a .

```
def Leaf.mem {α4} (T : Leaf α4) (x : α4) : Prop
```

instance Decidable (T.mem x)[\[source\]](#)

```
instance {α4} [DecidableEq α4] (T : Leaf α4) (x : α4) : Decidable (T.mem x)
```

def Twig.mem[\[source\]](#)

An element of $a : \alpha_3 \times \alpha_4$ is a member of Twig $\alpha_3 \alpha_4$ if the underlying α_3 element of the Twig is $a.1$ and $a.2$ is a member of one of the Leaf.

```
def Twig.mem (T : Twig α3 α4) (x : α3 × α4) : Prop
```

instance Decidable (T.mem x)[\[source\]](#)

```
instance {α3 α4} [DecidableEq α3] [DecidableEq α4] (T : Twig α3 α4) (x : α3 × α4) :
  Decidable (T.mem x)
```

def Branch.mem[\[source\]](#)

An element of $a : \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of Branch $\alpha_2 \alpha_3 \alpha_4$ if the underlying α_2 element of the Branch is $a.1$ and $a.2$ is a member of one of the Twig.

```
def Branch.mem (T : Branch α2 α3 α4) (x : α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)[\[source\]](#)

```
instance [DecidableEq α2] [DecidableEq α3] [DecidableEq α4] (T : Branch α2 α3 α4)
  (x : α2 × α3 × α4) : Decidable (T.mem x)
```

def Trunk.mem[\[source\]](#)

An element of $a : \alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of Trunk $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ if the underlying α_1 element of the Trunk is $a.1$ and $a.2$ is a member of one of the Branch.

```
def Trunk.mem (T : Trunk α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)[\[source\]](#)

```
instance [DecidableEq α1] [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]
  (T : Trunk α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Decidable (T.mem x)
```

def FourTree.mem[\[source\]](#)

An element of $a : \alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ is a member of FourTree $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ if a is a member of one of the Trunk.

```
def mem (T : FourTree α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Prop
```

instance Decidable (T.mem x)[\[source\]](#)

```
instance [DecidableEq α1] [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]
  (T : FourTree α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Decidable (T.mem x)
```

instance Membership (α1 × α2 × α3 × α4) (FourTree α1 α2 α3 α4)[\[source\]](#)

```
instance : Membership (α1 × α2 × α3 × α4) (FourTree α1 α2 α3 α4) where
```

instance Decidable (x ∈ T)[\[source\]](#)

```
instance [DecidableEq α1] [DecidableEq α2] [DecidableEq α3] [DecidableEq α4]
  (T : FourTree α1 α2 α3 α4) (x : α1 × α2 × α3 × α4) : Decidable (x ∈ T)
```

lemma mem_iff_mem_toMultiset[\[source\]](#)

```
lemma mem_iff_mem_toMultiset (T : FourTree  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ ) (x :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) :
  x  $\in$  T  $\leftrightarrow$  x  $\in$  T.toMultiset
```

lemma mem_of_parts[\[source\]](#)

```
lemma mem_of_parts {T : FourTree  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ } {C :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ }
  (trunk : Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ )
  (branch : Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ )
  (twig : Twig  $\alpha_3$   $\alpha_4$ ) (leaf : Leaf  $\alpha_4$ )
  (trunk_mem : trunk  $\in$  T.1) (branch_mem : branch  $\in$  trunk.2)
  (twig_mem : twig  $\in$  branch.2) (leaf_mem : leaf  $\in$  twig.2)
  (heq : C = (trunk.1, branch.1, twig.1, leaf.1)) :
  C  $\in$  T
```

2.5 PhysLean.Mathematics.DataStructures.FourTree.UniqueMap

[PhysLean/Mathematics/DataStructures/FourTree/UniqueMap.lean](#) on GitHub.

def Leaf.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from Leaf $\alpha_4 \rightarrow$ Leaf α_4 mapping the underlying elements.

```
def Leaf.uniqueMap4 : Leaf  $\alpha_4 \rightarrow$  Leaf  $\alpha_4$ 
| .leaf x => .leaf (f x)
```

def Twig.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from Twig $\alpha_3 \alpha_4 \rightarrow$ Twig $\alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original Twig.

```
def Twig.uniqueMap4 (T : Twig  $\alpha_3$   $\alpha_4$ ) : Twig  $\alpha_3$   $\alpha_4$ 
```

def Branch.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from Branch $\alpha_2 \alpha_3 \alpha_4 \rightarrow$ Branch $\alpha_2 \alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original Twig.

```
def Branch.uniqueMap4 (T : Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ ) :
  Branch  $\alpha_2$   $\alpha_3$   $\alpha_4$ 
```

def Trunk.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from Trunk $\alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow$ Trunk $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original Twig.

```
def Trunk.uniqueMap4 (T : Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ ) : Trunk  $\alpha_1$   $\alpha_2$   $\alpha_3$   $\alpha_4$ 
```

def FourTree.uniqueMap4[\[source\]](#)

Given a map $f : \alpha_4 \rightarrow \alpha_4$ the map from $\text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying leafs and deleting any that appear in the original twig of that leaf.

```
def uniqueMap4 (T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

lemma map_mem_uniqueMap4

[\[source\]](#)

```
lemma map_mem_uniqueMap4 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }  
  (x :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (hx : x ∈ T) (f :  $\alpha_4 \rightarrow \alpha_4$ ) :  
  (x.1, x.2.1, x.2.2.1, f x.2.2.2) ∈ T.uniqueMap4 f v  
  (x.1, x.2.1, x.2.2.1, f x.2.2.2) ∈ T
```

lemma exists_of_mem_uniqueMap4

[\[source\]](#)

```
lemma exists_of_mem_uniqueMap4 {T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ }  
  (C :  $\alpha_1 \times \alpha_2 \times \alpha_3 \times \alpha_4$ ) (h : C ∈ T.uniqueMap4 f) :  
  ∃ qHd qHu Q5 Q10, C = (qHd, qHu, Q5, f Q10) ∧ (qHd, qHu, Q5, Q10) ∈ T
```

def Twig.uniqueMap3

[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Twig } \alpha_3 \alpha_4 \rightarrow \text{Twig } \alpha_3 \alpha_4$ mapping the underlying first value of the twig.

```
def Twig.uniqueMap3 (T : Twig  $\alpha_3 \alpha_4$ ) : Twig  $\alpha_3 \alpha_4$ 
```

def Branch.uniqueMap3

[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Branch } \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{Branch } \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def Branch.uniqueMap3 (T : Branch  $\alpha_2 \alpha_3 \alpha_4$ ) : Branch  $\alpha_2 \alpha_3 \alpha_4$ 
```

def Trunk.uniqueMap3

[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{Trunk } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def Trunk.uniqueMap3 (T : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : Trunk  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

def FourTree.uniqueMap3

[\[source\]](#)

Given a map $f : \alpha_3 \rightarrow \alpha_3$ the map from $\text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4 \rightarrow \text{FourTree } \alpha_1 \alpha_2 \alpha_3 \alpha_4$ mapping the underlying first value of the twig, and deleting any new leafs that appeared in the old branch.

```
def uniqueMap3 (T : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ ) : FourTree  $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ 
```

lemma map_mem_uniqueMap3[\[source\]](#)

```

lemma map_mem_uniqueMap3 {T : FourTree α1 α2 α3 α4}
  (x : α1 × α2 × α3 × α4) (hx : x ∈ T) (f : α3 → α3) :
  (x.1, x.2.1, f x.2.2.1, x.2.2.2) ∈ T.uniqueMap3 f v
  (x.1, x.2.1, f x.2.2.1, x.2.2.2) ∈ T

```

lemma exists_of_mem_uniqueMap3[\[source\]](#)

```

lemma exists_of_mem_uniqueMap3 {T : FourTree α1 α2 α3 α4}
  (C : α1 × α2 × α3 × α4) (h : C ∈ T.uniqueMap3 f) :
  ∃ qHd qHu Q5 Q10, C = (qHd, qHu, f Q5, Q10) ∧
  (qHd, qHu, Q5, Q10) ∈ T

```

2.6 PhysLean.Mathematics.InnerProductSpace.Adjoint

[PhysLean/Mathematics/InnerProductSpace/Adjoint.lean](#) on GitHub.

structure HasAdjoint[\[source\]](#)

Adjoint of a linear map f such that $\forall x y, \langle \text{adjoint } \mathbb{k} f y, x \rangle = \langle y, f x \rangle$.

This computes adjoint of a liner map the same way as `ContinuousLinearMap.adjoint` but it is defined over `InnerProductSpace'`, which is a generalization of `InnerProductSpace` that provides instances for products and function types. These instances make it easier to perform computations compared to using the standard `InnerProductSpace` class.

```

structure HasAdjoint (f : E → F) (f' : F → E) where
  adjoint_inner_left (x : E) (y : F) : ⟨f' y, x⟩ = ⟨y, f x⟩

```

def adjoint[\[source\]](#)

```

@[inherit_doc HasAdjoint]
noncomputable
def adjoint (f : E → F)

```

lemma hasAdjoint[\[source\]](#)

```

lemma ContinuousLinearMap.hasAdjoint [CompleteSpace E] [CompleteSpace F] (f : E →L[ $\mathbb{k}$ ] F) :
  HasAdjoint  $\mathbb{k}$  f (fun y => fromL2  $\mathbb{k}$  (((toL2  $\mathbb{k}$ ) ◦L f ◦L (fromL2  $\mathbb{k}$ )).adjoint (toL2  $\mathbb{k}$  y)))
where

```

lemma adjoint_eq_clm_adjoint[\[source\]](#)

```

lemma adjoint_eq_clm_adjoint [CompleteSpace E] [CompleteSpace F] (f : E →L[ $\mathbb{k}$ ] F) :
  _root_.adjoint  $\mathbb{k}$  f = fun y => fromL2  $\mathbb{k}$  ((toL2  $\mathbb{k}$ ) ◦L f ◦L fromL2  $\mathbb{k}$ ).adjoint (toL2  $\mathbb{k}$  y))

```

lemma adjoint_apply_zero[\[source\]](#)

```

lemma HasAdjoint.adjoint_apply_zero {f : E → F} {f' : F → E}
  (hf : HasAdjoint  $\mathbb{k}$  f f') : f' 0 = 0

```

lemma HasAdjoint.adjoint[\[source\]](#)

```

lemma HasAdjoint.adjoint
  {f : E → F} {f' : F → E}
  (hf : HasAdjoint ℤ f f') : adjoint ℤ f = f'

```

lemma congr_adj[\[source\]](#)

```

lemma HasAdjoint.congr_adj (f : E → F) (f' g')
  (adjoint : HasAdjoint ℤ f g')
  (eq : g' = f') : HasAdjoint ℤ f f'

```

lemma hasAdjoint_id[\[source\]](#)

```

lemma hasAdjoint_id : HasAdjoint ℤ (fun x : E => x) (fun x => x)

```

lemma hasAdjoint_zero[\[source\]](#)

```

lemma hasAdjoint_zero : HasAdjoint ℤ (fun _ : E => (0 : F)) (fun _ => 0)

```

lemma comp[\[source\]](#)

```

lemma HasAdjoint.comp {f : F → G} {g : E → F} {f' g'}
  (hf : HasAdjoint ℤ f f') (hg : HasAdjoint ℤ g g') :
  HasAdjoint ℤ (fun x : E => f (g x)) (fun x => g' (f' x))

```

lemma prodMk[\[source\]](#)

```

lemma HasAdjoint.prodMk {f : E → F} {g : E → G} {f' g'}
  (hf : HasAdjoint ℤ f f') (hg : HasAdjoint ℤ g g') :
  HasAdjoint ℤ (fun x : E => (f x, g x)) (fun yz => f' yz.1 + g' yz.2)

```

lemma fst[\[source\]](#)

```

lemma HasAdjoint.fst {f : E → F×G} {f'} (hf : HasAdjoint ℤ f f') :
  HasAdjoint ℤ (fun x : E => (f x).1) (fun y => f' (y, 0))

```

lemma snd[\[source\]](#)

```

lemma HasAdjoint.snd {f : E → F×G} {f'} (hf : HasAdjoint ℤ f f') :
  HasAdjoint ℤ (fun x : E => (f x).2) (fun z => f' (0, z))

```

lemma neg[\[source\]](#)

```

lemma HasAdjoint.neg {f : E → F} {f'} (hf : HasAdjoint ℤ f f') :
  HasAdjoint ℤ (fun x : E => -f x) (fun y => -f' y)

```

lemma add[\[source\]](#)

```

lemma HasAdjoint.add {f g : E → F} {f' g'}
  (hf : HasAdjoint ℤ f f') (hg : HasAdjoint ℤ g g') :
  HasAdjoint ℤ (fun x : E => f x + g x) (fun y => f' y + g' y)

```

lemma sub[\[source\]](#)

```

lemma HasAdjoint.sub {f g : E → F} {f' g'}
  (hf : HasAdjoint ℚ f f') (hg : HasAdjoint ℚ g g') :
  HasAdjoint ℚ (fun x : E => f x - g x) (fun y => f' y - g' y)

```

lemma smul_left[\[source\]](#)

```

lemma HasAdjoint.smul_left {f : E → F} {f'} (c : ℚ)
  (hf : HasAdjoint ℚ f f') :
  HasAdjoint ℚ (fun x : E => c • f x) (fun y => (conj c) • f' y)

```

lemma smul_right[\[source\]](#)

```

lemma HasAdjoint.smul_right {f : E → ℚ} {f'} (v : F)
  (hf : HasAdjoint ℚ f f') :
  HasAdjoint ℚ (fun x : E => f x • v) (fun y => f' (conj ⟨y, v⟩))

```

2.7 PhysLean.Mathematics.InnerProductSpace.Basic

[PhysLean/Mathematics/InnerProductSpace/Basic.lean](#) on GitHub.

class Norm₂[\[source\]](#)

L_2 norm on E .

In particular, on product types $X \times Y$ and ι types $\iota \rightarrow X$ this class provides L_2 norm unlike $\|\cdot\|$.

```

class Norm₂ (E : Type*) where
  norm₂ : E → ℝ

```

class InnerProductSpace'[\[source\]](#)

Effectively as `InnerProductSpace ℚ E` but it does not requires that $\|x\|^2 = \langle x, x \rangle$. It is only required that they are equivalent $\exists (c > 0) (d > 0), c \cdot \|x\|^2 \leq \langle x, x \rangle \leq d \cdot \|x\|^2$. The main purpose of this class is to provide inner product space structure on product types $E \times F$ and ι types $\iota \rightarrow E$ without using `WithLp` gadget.

If you want to access L_2 norm use $\|x\|_2 := \sqrt{\langle x, x \rangle}$.

This class induces `InnerProductSpace ℚ (WithLp 2 E)` which equips $\|\cdot\|$ on X with L_2 norm. This is very useful when translating results from `InnerProductSpace` to `InnerProductSpace'` together with `toL2 : E → L[ℚ]` (`WithLp 2 E`) and `fromL2 : (WithL2 2 E) → L[ℚ]` E .

In short we have these implications: `` InnerProductSpace ℚ E → InnerProductSpace' ℚ E` `InnerProductSpace' ℚ E → InnerProductSpace ℚ (WithLp 2 E)` ```

The reason behind this type class is that with current mathlib design the requirement $\|x\|^2 = \langle x, x \rangle$ prevents us to give inner product space structure on product type $E \times F$ and ι type $\iota \rightarrow E$ as they are equipped with max norm. One has to work with `WithLp 2 (E × F)` and `WithLp 2 (ι → E)`. This places quite a bit inconvenience on users in certain scenarios. In particular, the main motivation behind this class is to make computations of `adjFderiv` and `gradient` easy.

```

class InnerProductSpace' ( $\mathbb{k}$  : Type*) (E : Type*) [RCLike  $\mathbb{k}$ ] [NormedAddCommGroup E] [NormedSpace
 $\mathbb{k}$  E]
  extends Norm2 E where
  core : InnerProductSpace.Core  $\mathbb{k}$  E
  norm2_sq_eq_re_inner :  $\forall x : E, \|x\|_2^2 = \text{re} (\text{core.inner } x \ x)$ 
  inner_top_equiv_norm :  $\exists c \ d : \mathbb{R},$ 
     $0 < c \wedge 0 < d \wedge$ 
     $\forall x : E, (c \cdot \|x\|^2 \leq \text{re} (\text{core.inner } x \ x)) \wedge (\text{re} (\text{core.inner } x \ x) \leq d \cdot \|x\|^2)$ 

```

instance InnerProductSpace.Core $\mathbb{k}$ E

[\[source\]](#)

```

instance [inst : InnerProductSpace'  $\mathbb{k}$  E] : InnerProductSpace.Core  $\mathbb{k}$  E

```

instance Inner $\mathbb{k}$ E

[\[source\]](#)

```

instance [inst : InnerProductSpace'  $\mathbb{k}$  E] : Inner  $\mathbb{k}$  E

```

instance InnerProductSpace' $\mathbb{k}$ E

[\[source\]](#)

```

instance { $\mathbb{k}$  : Type*} {E : Type*} [RCLike  $\mathbb{k}$ ] [NormedAddCommGroup E] [inst : InnerProductSpace
 $\mathbb{k}$  E] :
  InnerProductSpace'  $\mathbb{k}$  E where

```

instance toNormWithL2

[\[source\]](#)

Attach L_2 norm to WithLp 2 E

```

noncomputable
scoped instance toNormWithL2 : Norm (WithLp 2 E) where

```

instance toInnerWithL2

[\[source\]](#)

Attach inner product to WithLp 2 E

```

noncomputable
scoped instance toInnerWithL2 : Inner  $\mathbb{k}$  (WithLp 2 E) where

```

instance toNormedAddCommGroupWitL2

[\[source\]](#)

Attach normed group structure to WithLp 2 E with L_2 norm.

```

noncomputable
scoped instance toNormedAddCommGroupWitL2 : NormedAddCommGroup (WithLp 2 E)

```

instance toNormedSpaceWithL2

[\[source\]](#)

Attach normed space structure to WithLp 2 E with L_2 norm.

```

noncomputable
scoped instance toNormedSpaceWithL2 : NormedSpace  $\mathbb{k}$  (WithLp 2 E) where

```

instance toInnerProductSpaceWithL2[\[source\]](#)

Attach inner product space structure to WithLp 2 E.

noncomputable

instance toInnerProductSpaceWithL2 : InnerProductSpace $\mathbb{k}$ (WithLp 2 E) **where**

def toL2[\[source\]](#)

Continuous linear map from E to WithLp 2 E.

This map is continuous because we require topological equivalence between $\|\cdot\|$ and $\|\cdot\|_2$.

noncomputable

def toL2 : E $\rightarrow$ L[$\mathbb{k}$] WithLp 2 E **where**

def fromL2[\[source\]](#)

Continuous linear map from WithLp 2 E to E.

This map is continuous because we require topological equivalence between $\|\cdot\|$ and $\|\cdot\|_2$.

noncomputable

def fromL2 : WithLp 2 E $\rightarrow$ L[$\mathbb{k}$] E **where**

lemma fromL2_inner_left[\[source\]](#)

lemma fromL2_inner_left (x : WithLp 2 E) (y : E) : $\langle\langle$ fromL2 $\mathbb{k}$ x, y $\rangle\rangle = \langle\langle$ x, toL2 $\mathbb{k}$ y $\rangle\rangle$

lemma toL2_inner_left[\[source\]](#)

lemma toL2_inner_left (x : E) (y : WithLp 2 E) : $\langle\langle$ toL2 $\mathbb{k}$ x, y $\rangle\rangle = \langle\langle$ x, fromL2 $\mathbb{k}$ y $\rangle\rangle$

lemma toL2_fromL2[\[source\]](#)

@[simp]

lemma toL2_fromL2 (x : WithLp 2 E) : toL2 $\mathbb{k}$ (fromL2 $\mathbb{k}$ x) = x

lemma fromL2_toL2[\[source\]](#)

@[simp]

lemma fromL2_toL2 (x : E) : fromL2 $\mathbb{k}$ (toL2 $\mathbb{k}$ x) = x

def equivL2[\[source\]](#)

Continuous linear equivalence between WithLp 2 E and E under InnerProductSpace' $\mathbb{k}$ E.

noncomputable

def equivL2 : (WithLp 2 E) $\approx$ L[$\mathbb{k}$] E **where**

instance CompleteSpace (WithLp 2 E)[\[source\]](#)

```
instance [CompleteSpace E] : CompleteSpace (WithLp 2 E)
```

lemma ext_inner_left'[\[source\]](#)

```
lemma ext_inner_left' {x y : E} (h : ∀ v, ⟨v, x⟩ = ⟨v, y⟩) : x = y
```

lemma ext_inner_right'[\[source\]](#)

```
lemma ext_inner_right' {x y : E} (h : ∀ v, ⟨x, v⟩ = ⟨y, v⟩) : x = y
```

lemma inner_conj_symm'[\[source\]](#)

```
@[simp]
lemma inner_conj_symm' (x y : E) : ⟨y, x⟩† = ⟨x, y⟩
```

lemma inner_smul_left'[\[source\]](#)

```
lemma inner_smul_left' (x y : E) (r : ℝ) : ⟨r • x, y⟩ = r† * ⟨x, y⟩
```

lemma inner_smul_right'[\[source\]](#)

```
lemma inner_smul_right' (x y : E) (r : ℝ) : ⟨x, r • y⟩ = r * ⟨x, y⟩
```

lemma inner_zero_left'[\[source\]](#)

```
@[simp]
lemma inner_zero_left' (x : E) : ⟨0, x⟩ = 0
```

lemma inner_zero_right'[\[source\]](#)

```
@[simp]
lemma inner_zero_right' (x : E) : ⟨x, 0⟩ = 0
```

lemma inner_add_left'[\[source\]](#)

```
lemma inner_add_left' (x y z : E) : ⟨x + y, z⟩ = ⟨x, z⟩ + ⟨y, z⟩
```

lemma inner_add_right'[\[source\]](#)

```
lemma inner_add_right' (x y z : E) : ⟨x, y + z⟩ = ⟨x, y⟩ + ⟨x, z⟩
```

lemma inner_sub_left'[\[source\]](#)

```
lemma inner_sub_left' (x y z : E) : ⟨x - y, z⟩ = ⟨x, z⟩ - ⟨y, z⟩
```

lemma inner_sub_right'[\[source\]](#)

```
lemma inner_sub_right' (x y z : E) : ⟨x, y - z⟩ = ⟨x, y⟩ - ⟨x, z⟩
```

lemma inner_neg_left'[\[source\]](#)

```
@[simp]
lemma inner_neg_left' (x y : E) : ⟨⟨-x, y⟩⟩ = -⟨⟨x, y⟩⟩
```

lemma inner_neg_right'[\[source\]](#)

```
@[simp]
lemma inner_neg_right' (x y : E) : ⟨⟨x, -y⟩⟩ = -⟨⟨x, y⟩⟩
```

lemma inner_self_eq_zero'[\[source\]](#)

```
@[simp]
lemma inner_self_eq_zero' {x : E} : ⟨⟨x, x⟩⟩ = 0 ↔ x = 0
```

lemma Continuous.inner'[\[source\]](#)

```
@[fun_prop]
lemma Continuous.inner' {α} [TopologicalSpace α] (f g : α → E)
  (hf : Continuous f) (hg : Continuous g) : Continuous (fun a => ⟨⟨f a, g a⟩⟩)
```

lemma real_inner_self_nonneg'[\[source\]](#)

```
lemma real_inner_self_nonneg' {x : F} : 0 ≤ re (⟨⟨x, x⟩⟩)
```

lemma ContDiffAt.inner'[\[source\]](#)

```
@[fun_prop]
lemma ContDiffAt.inner' {f g : E → F} {x : E}
  (hf : ContDiffAt ℝ n f x) (hg : ContDiffAt ℝ n g x) :
  ContDiffAt ℝ n (fun x => ⟨⟨f x, g x⟩⟩) x
```

lemma ContDiff.inner'[\[source\]](#)

```
@[fun_prop]
lemma ContDiff.inner' {f g : E → F}
  (hf : ContDiff ℝ n f) (hg : ContDiff ℝ n g) :
  ContDiff ℝ n (fun x => ⟨⟨f x, g x⟩⟩)
```

instance Inner ℚ (E×F)[\[source\]](#)

Inner product on product types E×F defined as $\langle\langle x, y \rangle\rangle = \langle\langle x.fst, y.fst \rangle\rangle + \langle\langle x.snd, y.snd \rangle\rangle$.

This is just local instance as it is superseded by the following instance for InnerProductSpace'.

```
local instance : Inner ℚ (E×F)
```

lemma prod_inner_apply'[\[source\]](#)

```
@[simp]
lemma prod_inner_apply' (x y : (E × F)) : ⟨⟨x, y⟩⟩ = ⟨⟨x.fst, y.fst⟩⟩ + ⟨⟨x.snd, y.snd⟩⟩
```

instance InnerProductSpace' $\mathbb{K}$ (E×F)

[\[source\]](#)

noncomputable

instance : InnerProductSpace' $\mathbb{K}$ (E×F) **where**

instance InnerProductSpace' $\mathbb{K}$ ($\iota \rightarrow E$)

[\[source\]](#)

noncomputable

instance { ι : Type*} [Fintype ι] : InnerProductSpace' $\mathbb{K}$ ($\iota \rightarrow E$) **where**

2.8 PhysLean.Mathematics.VariationalCalculus.Basic

[PhysLean/Mathematics/VariationalCalculus/Basic.lean](#) on GitHub.

lemma fundamental_theorem_of_variational_calculus

[\[source\]](#)

lemma fundamental_theorem_of_variational_calculus { $f : X \rightarrow V$ }
 (μ : Measure X) [IsFiniteMeasureOnCompacts μ] [μ .IsOpenPosMeasure]
 [OpensMeasurableSpace X]
 (hf : IsTestFunction f) (hg : $\forall g$, IsTestFunction $g \rightarrow \int x, \langle f x, g x \rangle_{\mathbb{R}} \partial \mu = 0$) :
 $f = 0$

lemma fundamental_theorem_of_variational_calculus'

[\[source\]](#)

The assumption IsTestFunction f in fundamental_theorem_of_variational_calculus can be weakened to Continuous f .

The proof is by contradiction, assume that there is x_0 such that $f x_0$ then you can easily construct g test function with support on the neighborhood of x_0 such that $\langle f x, g x \rangle \geq 0$.

@[sorryful]

lemma fundamental_theorem_of_variational_calculus' { $f : X \rightarrow V$ }
 (μ : Measure X) [IsFiniteMeasureOnCompacts μ] [μ .IsOpenPosMeasure]
 [OpensMeasurableSpace X]
 (hf : Continuous f) (hg : $\forall g$, IsTestFunction $g \rightarrow \int x, \langle f x, g x \rangle_{\mathbb{R}} \partial \mu = 0$) :
 $f = 0$

2.9 PhysLean.Mathematics.VariationalCalculus.HasVarAdjDeriv

[PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean](#) on GitHub.

structure HasVarAdjDerivAt

[\[source\]](#)

This is analogue of saying $F' = (fderiv \mathbb{R} F u).adjoint$.

This definition is useful as we can prove composition theorem for it and HasVarGradient $F grad u$ can be computed by $grad := F' (fun _ => 1)$.

structure HasVarAdjDerivAt ($F : (X \rightarrow U) \rightarrow (Y \rightarrow V)$) ($F' : (Y \rightarrow V) \rightarrow (X \rightarrow U)$) ($u : X \rightarrow U$) : Prop
where
 smooth_at : ContDiff $\mathbb{R} \infty u$
 diff : $\forall (\phi : \mathbb{R} \rightarrow X \rightarrow U)$, ContDiff $\mathbb{R} \infty 1\phi \rightarrow$
 ContDiff $\mathbb{R} \infty$ (**fun** $sx : \mathbb{R} \times Y \Rightarrow F (\phi sx.1) sx.2$)
 linearize :
 $\forall (\phi : \mathbb{R} \rightarrow X \rightarrow U)$, ContDiff $\mathbb{R} \infty 1\phi \rightarrow$

```

    ∀ x,
      deriv (fun s' : ℝ => F (φ s') x) 0
    =
      deriv (fun s' : ℝ => F (fun x => φ 0 x + s' • deriv (φ · x) 0) x) 0
adjoint : HasVarAdjoint (fun δu x => deriv (fun s : ℝ => F (fun x' => u x' + s • δu x') x) 0)
    F'

```

lemma apply_smooth_of_smooth[\[source\]](#)

```

lemma apply_smooth_of_smooth {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u v : X → U}
  (h : HasVarAdjDerivAt F F' u) (hv : ContDiff ℝ ∞ v) : ContDiff ℝ ∞ (F v)

```

lemma apply_smooth_self[\[source\]](#)

```

lemma apply_smooth_self {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u : X → U}
  (h : HasVarAdjDerivAt F F' u) : ContDiff ℝ ∞ (F u)

```

lemma smooth_R[\[source\]](#)

```

lemma smooth_R {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u : X → U}
  (h : HasVarAdjDerivAt F F' u) {φ : ℝ → X → U} (hφ : ContDiff ℝ ∞ 1φ) (x : X) :
  ContDiff ℝ ∞ (fun s : ℝ => F (fun x' => φ s x') x)

```

lemma smooth_linear[\[source\]](#)

```

lemma smooth_linear {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u : X → U}
  (h : HasVarAdjDerivAt F F' u) {φ : ℝ → X → U} (hφ : ContDiff ℝ ∞ 1φ) :
  ContDiff ℝ ∞ (fun s' : ℝ => F (fun x => φ 0 x + s' • deriv (φ · x) 0) x)

```

lemma differentiable_linear[\[source\]](#)

```

lemma differentiable_linear {F : (X → U) → (X → V)} {F' : (X → V) → (X → U)} {u : X → U}
  (h : HasVarAdjDerivAt F F' u) {φ : ℝ → X → U} (hφ : ContDiff ℝ ∞ 1φ) (x : X) :
  Differentiable ℝ (fun s' : ℝ => F (fun x => φ 0 x + s' • deriv (φ · x) 0) x)

```

lemma id[\[source\]](#)

```

lemma id (u) (hu : ContDiff ℝ ∞ u) : HasVarAdjDerivAt (fun φ : X → U => φ) (fun ψ => ψ) u where

```

lemma const[\[source\]](#)

```

lemma const (u : X → U) (v : X → V) (hu : ContDiff ℝ ∞ u) (hv : ContDiff ℝ ∞ v) :
  HasVarAdjDerivAt (fun _ : X → U => v) (fun _ => 0) u where

```

lemma comp[\[source\]](#)

```

lemma comp {F : (Y → V) → (Z → W)} {G : (X → U) → (Y → V)} {u : X → U}
  {F' G'} (hF : HasVarAdjDerivAt F F' (G u)) (hG : HasVarAdjDerivAt G G' u) :
  HasVarAdjDerivAt (fun u => F (G u)) (fun ψ => G' (F' ψ)) u where

```

lemma unique_on_test_functions[\[source\]](#)

```

lemma unique_on_test_functions
  [IsFiniteMeasureOnCompacts (@volume X _)] [(@volume X _).IsOpenPosMeasure]
  [OpensMeasurableSpace X]
  (F : (X → U) → (Y → V)) (u : X → U)
  (F' G') (hF : HasVarAdjDerivAt F F' u) (hG : HasVarAdjDerivAt F G' u)
  (φ : Y → V) (hφ : IsTestFunction φ) :
  F' φ = G' φ

```

lemma unique[\[source\]](#)

```

lemma unique {X : Type*} [NormedAddCommGroup X] [InnerProductSpace ℝ X]
  [MeasureSpace X] [OpensMeasurableSpace X]
  [IsFiniteMeasureOnCompacts (@volume X _)] [(@volume X _).IsOpenPosMeasure]
  {Y : Type*} [NormedAddCommGroup Y] [InnerProductSpace ℝ Y]
  [FiniteDimensional ℝ Y] [MeasureSpace Y]
  {F : (X → U) → (Y → V)} {u : X → U}
  {F' G'} (hF : HasVarAdjDerivAt F F' u) (hG : HasVarAdjDerivAt F G' u)
  (φ : Y → V) (hφ : ContDiff ℝ ∞ φ) :
  F' φ = G' φ

```

lemma deriv'[\[source\]](#)

```

lemma deriv' (u : ℝ → ℝ) (hu : ContDiff ℝ ∞ u) :
  HasVarAdjDerivAt (fun φ : ℝ → ℝ => deriv φ) (fun φ x => - deriv φ x) u where

```

lemma deriv[\[source\]](#)

```

protected lemma deriv (F : (ℝ → U) → (ℝ → ℝ)) (F') (u) (hF : HasVarAdjDerivAt F F' u) :
  HasVarAdjDerivAt (fun φ : ℝ → U => deriv (F φ))
  (fun ψ x => F' (fun x' => - deriv ψ x') x) u

```

lemma fmap[\[source\]](#)

```

lemma fmap [CompleteSpace U] [CompleteSpace V]
  (f : X → U → V) {f' : X → U → _}
  (u : X → U) (hu : ContDiff ℝ ∞ u)
  (hf' : ContDiff ℝ ∞ 1f) (hf : ∀ x u, HasAdjFDerivAt ℝ (f x) (f' x u) u) :
  HasVarAdjDerivAt (fun (φ : X → U) x => f x (φ x)) (fun ψ x => f' x (u x) (ψ x)) u where

```

lemma neg[\[source\]](#)

```

lemma neg (F : (X → U) → (X → V)) (F') (u) (hF : HasVarAdjDerivAt F F' u) :
  HasVarAdjDerivAt (fun φ x => -F φ x) (fun ψ x => - F' ψ x) u where

```

lemma add[\[source\]](#)

```

lemma add
  (F G : (X → U) → (X → V)) (F' G') (u)
  (hF : HasVarAdjDerivAt F F' u) (hG : HasVarAdjDerivAt G G' u) :
  HasVarAdjDerivAt (fun φ x => F φ x + G φ x) (fun ψ x => F' ψ x + G' ψ x) u where

```

lemma mul[\[source\]](#)**lemma mul**

```

(F G : (X → ℝ) → (X → ℝ)) (F' G') (u)
(hF : HasVarAdjDerivAt F F' u) (hG : HasVarAdjDerivAt G G' u) :
HasVarAdjDerivAt (fun φ x => F φ x * G φ x)
  (fun ψ x => F' (fun x' => ψ x' * G u x') x + G' (fun x' => F u x' * ψ x') x) u where

```

2.10 PhysLean.Mathematics.VariationalCalculus.HasVarAdjoint

[PhysLean/Mathematics/VariationalCalculus/HasVarAdjoint.lean](#) on GitHub.

def IsLocalizedFunctionTransform[\[source\]](#)

Function transformation F is localizable if the values of the transformed function $F \varphi$ on some compact set K can depend only on the values of φ on some another compact set L .

```

def IsLocalizedFunctionTransform (F : (X → U) → (Y → V)) : Prop

```

structure HasVarAdjoint[\[source\]](#)

Map F from $(X \rightarrow U)$ to $(Y \rightarrow V)$ has a variational adjoint F' if it preserves test functions and satisfies the adjoint relation $\langle\langle F \varphi, \psi \rangle\rangle = \langle\langle \varphi, F' \psi \rangle\rangle$ for all test functions φ and ψ for $\langle\langle \varphi, \psi \rangle\rangle = \int x, \langle\langle \varphi x, \psi x \rangle\rangle_{\mathbb{R}} \partial \mu$.

The canonical example is the function $F = \text{deriv}$ that has adjoint $F' = - \text{deriv}$.

This notion of adjoint allows us to do formally variational calculus as often encountered in physics textbooks. In mathematical literature, the adjoint is often defined for unbounded operators, but such formal treatment is unnecessarily complicated for physics applications.

structure HasVarAdjoint

```

(F : (X → U) → (Y → V)) (F' : (Y → V) → (X → U)) where
test_fun_preserving : ∀ φ, IsTestFunction φ → IsTestFunction (F φ)
test_fun_preserving' : ∀ φ, IsTestFunction φ → IsTestFunction (F' φ)
adjoint : ∀ φ ψ, IsTestFunction φ → IsTestFunction ψ →
  ∫ y, ⟨⟨ F φ y, ψ y ⟩⟩_ℝ = ∫ x, ⟨⟨ φ x, F' ψ x ⟩⟩_ℝ
ext : IsLocalizedFunctionTransform F'

```

lemma id[\[source\]](#)

```

lemma id : HasVarAdjoint (fun φ : X → U => φ) (fun φ => φ) where

```

lemma zero[\[source\]](#)

```

lemma zero : HasVarAdjoint (fun (_ : X → U) (_ : Y) => (0 : V)) (fun _ _ => 0) where

```

lemma comp[\[source\]](#)

```

lemma comp {F : (Y → V) → (Z → W)} {G : (X → U) → (Y → V)} {F' G'}
(hF : HasVarAdjoint F F') (hG : HasVarAdjoint G G') :
HasVarAdjoint (fun φ => F (G φ)) (fun φ => G' (F' φ)) where

```

lemma deriv[\[source\]](#)

```
protected lemma deriv :
  HasVarAdjoint (fun  $\phi$  :  $\mathbb{R} \rightarrow \mathbb{R} \Rightarrow$  deriv  $\phi$ ) (fun  $\phi$  x  $\Rightarrow$  - deriv  $\phi$  x) where
```

lemma congr_fun[\[source\]](#)

```
lemma congr_fun {F G : (X  $\rightarrow$  U)  $\rightarrow$  (Y  $\rightarrow$  V)} {F' : (Y  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  (h : HasVarAdjoint G F') (h' :  $\forall \phi$ , IsTestFunction  $\phi \rightarrow$  F  $\phi$  = G  $\phi$ ) :
  HasVarAdjoint F F' where
```

lemma unique_on_test_functions[\[source\]](#)

Variational adjoint is unique only when applied to test functions.

```
lemma unique_on_test_functions {F : (X  $\rightarrow$  U)  $\rightarrow$  (Y  $\rightarrow$  V)} {F' G' : (Y  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  [IsFiniteMeasureOnCompacts (@volume X _)] [(@volume X _).IsOpenPosMeasure]
  [OpensMeasurableSpace X] (hF' : HasVarAdjoint F F') (hG' : HasVarAdjoint F G') :
   $\forall \phi$ , IsTestFunction  $\phi \rightarrow$  F'  $\phi$  = G'  $\phi$ 
```

lemma unique[\[source\]](#)

Variational adjoint is unique only when applied to smooth functions.

```
lemma unique
  {X : Type*} [NormedAddCommGroup X] [InnerProductSpace  $\mathbb{R}$  X]
  [MeasureSpace X] [OpensMeasurableSpace X]
  {Y : Type*} [NormedAddCommGroup Y] [InnerProductSpace  $\mathbb{R}$  Y]
  [FiniteDimensional  $\mathbb{R}$  Y] [MeasureSpace Y]
  {F : (X  $\rightarrow$  U)  $\rightarrow$  (Y  $\rightarrow$  V)} {F' G' : (Y  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  [IsFiniteMeasureOnCompacts (@volume X _)] [(@volume X _).IsOpenPosMeasure]
  (hF : HasVarAdjoint F F') (hG : HasVarAdjoint F G') :
   $\forall f$ , ContDiff  $\mathbb{R} \infty f \rightarrow$  F' f = G' f
```

lemma neg[\[source\]](#)

```
lemma neg {F : (X  $\rightarrow$  U)  $\rightarrow$  (X  $\rightarrow$  V)} {F' : (X  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  (hF : HasVarAdjoint F F') :
  HasVarAdjoint (fun  $\phi$  x  $\Rightarrow$  - F  $\phi$  x) (fun  $\phi$  x  $\Rightarrow$  - F'  $\phi$  x) where
```

lemma add[\[source\]](#)

```
lemma add {F G : (X  $\rightarrow$  U)  $\rightarrow$  (X  $\rightarrow$  V)} {F' G' : (X  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  (hF : HasVarAdjoint F F') (hG : HasVarAdjoint G G') :
  HasVarAdjoint (fun  $\phi$  x  $\Rightarrow$  F  $\phi$  x + G  $\phi$  x) (fun  $\phi$  x  $\Rightarrow$  F'  $\phi$  x + G'  $\phi$  x) where
```

lemma sub[\[source\]](#)

```
lemma sub {F G : (X  $\rightarrow$  U)  $\rightarrow$  (X  $\rightarrow$  V)} {F' G' : (X  $\rightarrow$  V)  $\rightarrow$  (X  $\rightarrow$  U)}
  (hF : HasVarAdjoint F F') (hG : HasVarAdjoint G G') :
  HasVarAdjoint (fun  $\phi$  x  $\Rightarrow$  F  $\phi$  x - G  $\phi$  x) (fun  $\phi$  x  $\Rightarrow$  F'  $\phi$  x - G'  $\phi$  x)
```

lemma mul_left[\[source\]](#)

```
lemma mul_left {F : (X → ℝ) → (X → ℝ)} {ψ : X → ℝ} {F' : (X → ℝ) → (X → ℝ)}
  (hF : HasVarAdjoint F F') (hψ : ContDiff ℝ ∞ ψ) :
  HasVarAdjoint (fun φ x => ψ x * F φ x) (fun φ x => F' (fun x => ψ x * φ x) x) where
```

lemma mul_right[\[source\]](#)

```
lemma mul_right {F : (X → ℝ) → (X → ℝ)} {ψ : X → ℝ} {F' : (X → ℝ) → (X → ℝ)}
  (hF : HasVarAdjoint F F') (hψ : ContDiff ℝ ∞ ψ) :
  HasVarAdjoint (fun φ x => F φ x * ψ x) (fun φ x => F' (fun x => φ x * ψ x) x) where
```

lemma smul_left[\[source\]](#)

```
lemma smul_left {F : (X → U) → (X → V)} {ψ : X → ℝ} {F' : (X → V) → (X → U)}
  (hF : HasVarAdjoint F F') (hψ : ContDiff ℝ ∞ ψ) :
  HasVarAdjoint (fun φ x => ψ x • F φ x) (fun φ x => F' (fun x' => ψ x' • φ x') x) where
```

lemma smul_right[\[source\]](#)

```
lemma smul_right {F : (X → U) → (X → V)} {ψ : X → ℝ} {F' : (X → V) → (X → U)}
  (hF : HasVarAdjoint F F') (hψ : ContDiff ℝ ∞ ψ) :
  HasVarAdjoint (fun φ x => ψ x • F φ x) (fun φ x => F' (fun x' => ψ x' • φ x') x) where
```

2.11 PhysLean.Mathematics.VariationalCalculus.HasVarGradient

[PhysLean/Mathematics/VariationalCalculus/HasVarGradient.lean](#) on GitHub.

inductive HasVarGradientAt[\[source\]](#)

Function *grad* is variational gradient of functional *S* at point *u*.

This formalizes the notion of variational gradient $\delta S / \delta u$ of a functional *S* at a point *u*.

However, it is not defined for a functional $S : (X \rightarrow U) \rightarrow \mathbb{R}$ but rather for the function $S' : (X \rightarrow U) \rightarrow (X \rightarrow \mathbb{R})$ which is related to the usual functional as $S u = \int x, S' (u x) x \, \partial \mu$. For example for action integral, $S u = \int t, L (u t) (deriv u t)$ we have $S' u t = L (u t) (deriv u t)$. Working with *S'* rather than with *S* allows us to ignore certain technicalities with integrability.

Examples:

Euler-Lagrange equations: $\delta / \delta x \int L(x, \dot{x}) dt = \partial L / \partial x - d/dt (\partial L / \partial \dot{x})$ can be expressed as `HasVarGradientAt (fun u t => L (u t) (deriv u t)) (fun t => deriv (L · (deriv u t)) ((u t)) - deriv (fun t' => deriv (L (u t')) ·) (deriv u t')) t) u`

Laplace equation is variational gradient of Dirichlet energy: $\delta / \delta u \int 1/2 * ||\nabla u||^2 = -\Delta u$ can be expressed as `HasVarGradientAt (fun u t => 1/2 * deriv u t^2) (fun t => - deriv (deriv u) t) u`

```
inductive HasVarGradientAt (F : (X → U) → (X → ℝ)) (grad : X → U) (u : X → U) : Prop
  | intro (F') (hF' : HasVarAdjDerivAt F F' u) (hgrad : grad = F' (fun _ => 1))
```

def varGradient[\[source\]](#)

The variational gradient of a function $F : (X \rightarrow U) \rightarrow (X \rightarrow \mathbb{R})$ evaluated at a function $u : X \rightarrow U$.

This not defined defined for a functional $S : (X \rightarrow U) \rightarrow \mathbb{R}$ but rather for the function $F : (X \rightarrow U) \rightarrow (X \rightarrow \mathbb{R})$ which is the integrand of the functional $S u = \int x, F (u x) x \partial \mu$. For example for action integral, $S u = \int t, L (u t) (deriv u t)$ we have $S' u t = L (u t) (deriv u t)$.

On functions $F : (X \rightarrow U) \rightarrow (X \rightarrow \mathbb{R})$ which do not have a variational gradient, this function is defined to give $\emptyset$.

```
noncomputable def varGradient (F : (X → U) → (X → ℝ)) (u : X → U) : X → U
```

lemma unique[\[source\]](#)**lemma** unique

```
{S' : (X → U) → (X → ℝ)} {grad grad' : X → U} {u : X → U}
(h : HasVarGradientAt S' grad u) (h' : HasVarGradientAt S' grad' u) :
grad = grad'
```

lemma HasVarGradientAt.varGradient[\[source\]](#)**protected lemma** varGradient

```
(F : (X → U) → (X → ℝ)) (grad : X → U) (u : X → U)
(hF : HasVarGradientAt F grad u) :
varGradient F u = grad
```

2.12 PhysLean.Mathematics.VariationalCalculus.IsTestFunction

[PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean](#) on GitHub.

structure IsTestFunction[\[source\]](#)

A test function is a smooth function with compact support.

```
@[fun_prop]
structure IsTestFunction (f : X → U) where
  smooth : ContDiff ℝ ∞ f
  supp : HasCompactSupport f
```

lemma integrable[\[source\]](#)**@[fun_prop]**

```
lemma IsTestFunction.integrable [MeasurableSpace X] [OpensMeasurableSpace X]
  {f : X → U} (hf : IsTestFunction f)
  (μ : Measure X) [IsFiniteMeasureOnCompacts μ] :
  MeasureTheory.Integrable f μ
```

lemma differentiable[\[source\]](#)**@[fun_prop]**

```
lemma IsTestFunction.differentiable {f : X → U} (hf : IsTestFunction f) :
  Differentiable ℝ f
```

lemma contDiff[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.contDiff {f : X → U} (hf : IsTestFunction f) :
  ContDiff ℝ ∞ f
```

lemma deriv[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.deriv {f : ℝ → U} (hf : IsTestFunction f) :
  IsTestFunction (fun x => deriv f x) where
```

lemma zero[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.zero :
  IsTestFunction (fun _ : X => (0 : U)) where
```

lemma add[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.add {f g : X → U} (hf : IsTestFunction f) (hg : IsTestFunction g) :
  IsTestFunction (fun x => f x + g x) where
```

lemma neg[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.neg {f : X → U} (hf : IsTestFunction f) :
  IsTestFunction (fun x => - f x) where
```

lemma sub[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.sub {f g : X → U} (hf : IsTestFunction f) (hg : IsTestFunction g) :
  IsTestFunction (f - g) where
```

lemma mul[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.mul {f g : X → ℝ} (hf : IsTestFunction f) (hg : IsTestFunction g) :
  IsTestFunction (fun x => f x * g x) where
```

lemma inner[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.inner {f g : X → V} (hf : IsTestFunction f) (hg : IsTestFunction g) :
  IsTestFunction (fun x => ⟨⟨f x, g x⟩⟩_ℝ) where
```

lemma mul_left[\[source\]](#)

```
@[fun_prop]
lemma IsTestFunction.mul_left {f g : X → ℝ} (hf : ContDiff ℝ ∞ f) (hg : IsTestFunction g) :
```

```
IsTestFunction (fun x => f x * g x) where
```

lemma mul_right

[\[source\]](#)

```
@[fun_prop]
```

```
lemma IsTestFunction.mul_right {f g : X → ℝ} (hf : IsTestFunction f) (hg : ContDiff ℝ ∞ g) :
  IsTestFunction (fun x => f x * g x) where
```

lemma smul_left

[\[source\]](#)

```
@[fun_prop]
```

```
lemma IsTestFunction.smul_left {f : X → ℝ} {g : X → U}
  (hf : ContDiff ℝ ∞ f) (hg : IsTestFunction g) : IsTestFunction (fun x => f x • g x) where
```

lemma smul_right

[\[source\]](#)

```
@[fun_prop]
```

```
lemma IsTestFunction.smul_right {f : X → ℝ} {g : X → U}
  (hf : IsTestFunction f) (hg : ContDiff ℝ ∞ g) : IsTestFunction (fun x => f x • g x) where
```

2.13 PhysLean.Meta.Linters.Sorry

[PhysLean/Meta/Linters/Sorry.lean](#) on GitHub.

structure SorryfulInfo

[\[source\]](#)

The information for stored for a declaration marked with sorryful.

```
structure SorryfulInfo where
```

```
  name : Name
  docstring : String
  fileName : Name
  line : Nat
```

def addSorryfulEntry

[\[source\]](#)

Adds an entry to sorryfulExtension.

```
def addSorryfulEntry {m : Type → Type} [MonadEnv m]
  (declName : Name) (docString : String) (fileName : Name) (line : Nat) : m Unit
```

2.14 PhysLean.Optics.Polarization.Basic

[PhysLean/Optics/Polarization/Basic.lean](#) on GitHub.

def monochromX

[\[source\]](#)

x-component of monochromatic time-harmonic wave.

```
noncomputable def monochromX (k : WaveVector) (E₀ x ω δx : ℝ)
```

def monochromY[\[source\]](#)

y-component of monochromatic time-harmonic wave.

```
noncomputable def monochromY (k : WaveVector) (E0y ω δy : ℝ)
```

def harmonicElectromagneticPlaneWave[\[source\]](#)

General form of monochromatic time-harmonic electromagnetic plane wave where the direction of propagation is taken to be `EuclideanSpace.single 2 1`. E_{0x} and E_{0y} are the respective amplitudes, ω is the angular frequency, δx and δy are the respective phases for E_x and E_y .

```
@[nolint unusedArguments]
```

```
noncomputable def harmonicElectromagneticPlaneWave (k : WaveVector) (E0x E0y ω δx δy : ℝ)
  (hk : k = EuclideanSpace.single 2 (ω/c)) :
  ElectricField
```

lemma harmonicElectromagneticPlaneWave_eq_electricplaneWave[\[source\]](#)

The transverse harmonic planewave representation is equivalent to the general electric field planewave expression with $\|k\| = \omega/c$.

```
lemma harmonicElectromagneticPlaneWave_eq_electricplaneWave {c : ℝ} {k : WaveVector}
  {E0x E0y ω δx δy : ℝ} (hc_ge_zero : 0 < c) (hw_ge_zero : 0 < ω)
  (hk : k = EuclideanSpace.single 2 (ω/c)) :
  (harmonicElectromagneticPlaneWave k E0x E0y ω δx δy hk) = electricPlaneWave
  (fun p => (E0x * cos (-(ω/c)*p + δx)) • (EuclideanSpace.single 0 1) +
  (E0y * cos (-(ω/c)*p + δy)) • (EuclideanSpace.single 1 1)) c
  (WaveVector.toDirection k (by rw [hk]; simp [ne_of_gt, hc_ge_zero, hw_ge_zero]))
```

lemma eq_monochromX[\[source\]](#)

*monochromX is equivalent to $E_{0x} * \cos (\tau + \delta x)$ with $\tau = \omega * t - \text{inner } \mathbb{R} k r$.*

```
lemma eq_monochromX (h : τ = ω * t - inner ℝ k r) :
  monochromX k E0x ω δx t r = E0x * cos (τ + δx)
```

lemma eq_monochromY[\[source\]](#)

*monochromY is equivalent to $E_{0y} * \cos (\tau + \delta y)$ with $\tau = \omega * t - \text{inner } \mathbb{R} k r$.*

```
lemma eq_monochromY (h : τ = ω * t - inner ℝ k r) :
  monochromY k E0y ω δy t r = E0y * cos (τ + δy)
```

theorem polarizationEllipse[\[source\]](#)

The locus of the electric field vector of an monochromatic time-harmonic electromagnetic plane wave is an ellipse.

```
theorem polarizationEllipse (hx : E0x ≠ 0) (hy : E0y ≠ 0) (h : τ = ω * t - inner ℝ k r) :
  (Ex / E0x)^2 + (Ey / E0y)^2 - 2 * (Ex / E0x) * (Ey / E0y) * cos (δy - δx) =
  sin (δy - δx) ^ 2
```

2.15 PhysLean.Particles.SuperSymmetry.SU5.Charges.AllowsTerm

[PhysLean/Particles/SuperSymmetry/SU5/Charges/AllowsTerm.lean](#) on GitHub.

def AllowsTerm [\[source\]](#)

The charges of representations $x : \text{Charges}$ allow a potential term $T : \text{PotentialTerm}$ if the zero charge is in the set of charges associated with that potential term.

```
def AllowsTerm (x : Charges) (T : PotentialTerm) : Prop
```

lemma allowsTerm_iff_zero_mem_ofPotentialTerm' [\[source\]](#)

```
lemma allowsTerm_iff_zero_mem_ofPotentialTerm' {x : Charges} {T : PotentialTerm} :
  x.AllowsTerm T ↔ 0 ∈ x.ofPotentialTerm' T
```

instance Decidable (x.AllowsTerm T) [\[source\]](#)

```
instance (x : Charges) (T : PotentialTerm) : Decidable (x.AllowsTerm T)
```

lemma allowsTerm_mono [\[source\]](#)

```
lemma allowsTerm_mono {T : PotentialTerm} {y x : Charges}
  (h : y ⊆ x) (hy : y.AllowsTerm T) : x.AllowsTerm T
```

def allowsTermForm [\[source\]](#)

A element of Charges from three intergers $a \ b \ c : \mathbb{Z}$ for a given potential term T . Defined such that `allowsTermForm a b c T` always allows the potential term T , and if any over charge x allows T then it is due to a subset of the form `allowsTermForm a b c T`.

```
def allowsTermForm (a b c : ℤ) : (T : PotentialTerm) → Charges
| .μ => (some a, some a, ∅, ∅)
| .β => (none, some a, {a}, ∅)
| .Λ => (none, none, {a, b}, {- a - b})
| .W1 => (none, none, {- a - b - c}, {a, b, c})
| .W2 => (some (- a - b - c), none, ∅, {a, b, c})
| .W3 => (none, some (- a), {b, - b - 2 * a}, ∅)
| .W4 => (some (- c - 2 * b), some (- b), {c}, ∅)
| .K1 => (none, none, {-a}, {b, - a - b})
| .K2 => (some a, some b, ∅, {- a - b})
| .topYukawa => (none, some (-a), ∅, {b, - a - b})
| .bottomYukawa => (some a, none, {b}, {- a - b})
```

lemma allowsTermForm_allowsTerm [\[source\]](#)

```
lemma allowsTermForm_allowsTerm {a b c : ℤ} {T : PotentialTerm} :
  (allowsTermForm a b c T).AllowsTerm T
```

lemma allowsTerm_of_eq_allowsTermForm [\[source\]](#)

```
lemma allowsTerm_of_eq_allowsTermForm {T : PotentialTerm}
  (x : Charges) (h : ∃ a b c, x = allowsTermForm a b c T) :
```

```
x.AllowsTerm T
```

lemma allowsTermForm_eq_of_subset

[\[source\]](#)

```
lemma allowsTermForm_eq_of_subset {a b c a' b' c' : ℤ} {T : PotentialTerm}
  (h : allowsTermForm a b c T ⊆ allowsTermForm a' b' c' T) (hT : T ≠ W1 ∧ T ≠ W2) :
  allowsTermForm a b c T = allowsTermForm a' b' c' T
```

lemma allowsTermForm_card_le_degree

[\[source\]](#)

```
lemma allowsTermForm_card_le_degree {a b c : ℤ} {T : PotentialTerm} :
  (allowsTermForm a b c T).card ≤ T.degree
```

lemma allowsTermForm_subset_allowsTerm_of_allowsTerm

[\[source\]](#)

```
lemma allowsTermForm_subset_allowsTerm_of_allowsTerm {T : PotentialTerm} {x : Charges}
  (h : x.AllowsTerm T) :
  ∃ a b c, allowsTermForm a b c T ⊆ x ∧ (allowsTermForm a b c T).AllowsTerm T
```

lemma subset_card_le_degree_allowsTerm_of_allowsTerm

[\[source\]](#)

```
lemma subset_card_le_degree_allowsTerm_of_allowsTerm {T : PotentialTerm} {x : Charges}
  (h : x.AllowsTerm T) : ∃ y ∈ x.powerset, y.card ≤ T.degree ∧ y.AllowsTerm T
```

2.16 PhysLean.Particles.SuperSymmetry.SU5.Charges.Basic

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Basic.lean](#) on GitHub.

def Charges

[\[source\]](#)

The type such that an element corresponds to the collection of charges associated with the matter content of the theory. The order of charges is implicitly taken to be q_H^d , q_H^u , Q_5 , Q_{10} .

*The Q_5 and Q_{10} charges are represented by *Finset* rather than *Multiset*, so multiplicity is not included.*

```
def Charges : Type
```

instance DecidableEq Charges

[\[source\]](#)

```
instance : DecidableEq Charges
```

instance Repr Charges

[\[source\]](#)

```
unsafe instance : Repr Charges where
```

def toProd

[\[source\]](#)

*The explicit casting of a term of *Charges* to a term of $\text{Option } \mathbb{Z} \times \text{Option } \mathbb{Z} \times \text{Finset } \mathbb{Z} \times \text{Finset } \mathbb{Z}$.*

```
def toProd (x : Charges) : Option  $\mathbb{Z}$  × Option  $\mathbb{Z}$  × Finset  $\mathbb{Z}$  × Finset  $\mathbb{Z}$ 
```

lemma eq_of_parts

[\[source\]](#)

```
lemma eq_of_parts {x y : Charges} (h1 : x.1 = y.1) (h2 : x.2.1 = y.2.1)
  (h3 : x.2.2.1 = y.2.2.1) (h4 : x.2.2.2 = y.2.2.2) : x = y
```

instance hasSubset

[\[source\]](#)

```
instance hasSubset : HasSubset Charges where
```

instance hasSSubset

[\[source\]](#)

```
instance hasSSubset : HasSSubset Charges where
```

instance subsetDecidable

[\[source\]](#)

```
instance subsetDecidable (x y : Charges) : Decidable (x  $\subseteq$  y)
```

lemma subset_def

[\[source\]](#)

```
lemma subset_def {x y : Charges} : x  $\subseteq$  y  $\leftrightarrow$  x.1.toFinset  $\subseteq$  y.1.toFinset  $\wedge$ 
  x.2.1.toFinset  $\subseteq$  y.2.1.toFinset  $\wedge$  x.2.2.1  $\subseteq$  y.2.2.1  $\wedge$  x.2.2.2  $\subseteq$  y.2.2.2
```

lemma subset_refl

[\[source\]](#)

```
@[simp, refl]
lemma subset_refl (x : Charges) : x  $\subseteq$  x
```

lemma subset_trans

[\[source\]](#)

```
lemma subset_trans {x y z : Charges} (hxy : x  $\subseteq$  y) (hyz : y  $\subseteq$  z) : x  $\subseteq$  z
```

lemma subset_antisymm

[\[source\]](#)

```
lemma subset_antisymm {x y : Charges} (hxy : x  $\subseteq$  y) (hyx : y  $\subseteq$  x) : x = y
```

instance emptyInst

[\[source\]](#)

```
instance emptyInst : EmptyCollection Charges where
```

lemma empty_subset

[\[source\]](#)

```
@[simp]
lemma empty_subset (x : Charges) :  $\emptyset \subseteq$  x
```

lemma subset_of_empty_iff_empty[\[source\]](#)

```
@[simp]
lemma subset_of_empty_iff_empty {x : Charges} :
  x ⊆ ∅ ↔ x = ∅
```

lemma empty_qHd[\[source\]](#)

```
@[simp]
lemma empty_qHd : (∅ : Charges).1 = none
```

lemma empty_qHu[\[source\]](#)

```
@[simp]
lemma empty_qHu : (∅ : Charges).2.1 = none
```

lemma empty_Q5[\[source\]](#)

```
@[simp]
lemma empty_Q5 : (∅ : Charges).2.2.1 = ∅
```

lemma empty_Q10[\[source\]](#)

```
@[simp]
lemma empty_Q10 : (∅ : Charges).2.2.2 = ∅
```

def card[\[source\]](#)

The cardinality of a Charges is defined to be the sum of the cardinalities of each of the underlying finite sets of charges, with Option $\mathbb{Z}$ turned to finsets.

```
def card (x : Charges) : Nat
```

lemma card_empty[\[source\]](#)

```
@[simp]
lemma card_empty : card (∅ : Charges) = 0
```

lemma card_mono[\[source\]](#)

```
lemma card_mono {x y : Charges} (h : x ⊆ y) : card x ≤ card y
```

lemma eq_of_subset_card[\[source\]](#)

```
lemma eq_of_subset_card {x y : Charges} (h : x ⊆ y) (hcard : card x = card y) : x = y
```

def powerset[\[source\]](#)

The powerset of a charge . Given a charge $x : \text{Charges}$ it's powerset is the finite set of all Charges which are subsets of x .

```
def powerset (x : Charges) : Finset Charges
```

lemma mem_powerset_iff_subset[\[source\]](#)

```
@[simp]
lemma mem_powerset_iff_subset {x y : Charges} :
  x ∈ powerset y ↔ x ⊆ y
```

lemma self_mem_powerset[\[source\]](#)

```
lemma self_mem_powerset (x : Charges) :
  x ∈ powerset x
```

lemma empty_mem_powerset[\[source\]](#)

```
lemma empty_mem_powerset (x : Charges) :
  ∅ ∈ powerset x
```

lemma powerset_of_empty[\[source\]](#)

```
@[simp]
lemma powerset_of_empty :
  powerset ∅ = {∅}
```

lemma powerset_mono[\[source\]](#)

```
lemma powerset_mono {x y : Charges} :
  powerset x ⊆ powerset y ↔ x ⊆ y
```

lemma min_exists_inductive[\[source\]](#)

```
lemma min_exists_inductive (S : Finset Charges) (hS : S ≠ ∅) :
  (n : ℕ) → (hn : S.card = n) →
  ∃ y ∈ S, powerset y ∩ S = {y}
| 0, h => by simp_all
| 1, h => by
  rw [Finset.card_eq_one] at h
  obtain (y, rfl)
```

lemma min_exists[\[source\]](#)

```
lemma min_exists (S : Finset Charges) (hS : S ≠ ∅) :
  ∃ y ∈ S, powerset y ∩ S = {y}
```

def offFinset[\[source\]](#)

Given $S5$ $S10$: Finset $\mathbb{Z}$ the finite set of charges associated with for which the 5-bar representation charges sit in $S5$ and the 10d representation charges sit in $S10$.

```
def offFinset (S5 S10 : Finset ℤ) : Finset Charges
```

lemma qHd_mem_offFinset[\[source\]](#)

```

lemma qHd_mem_ofFinset (S5 S10 : Finset  $\mathbb{Z}$ ) (z :  $\mathbb{Z}$ ) (x2 : Option  $\mathbb{Z} \times$  Finset  $\mathbb{Z} \times$  Finset  $\mathbb{Z}$ )
  (hsub : (some z, x2)  $\in$  ofFinset S5 S10) :
  z  $\in$  S5

```

lemma qHu_mem_ofFinset

[\[source\]](#)

```

lemma qHu_mem_ofFinset (S5 S10 : Finset  $\mathbb{Z}$ ) (z :  $\mathbb{Z}$ ) (x1 : Option  $\mathbb{Z}$ ) (x2 : Finset  $\mathbb{Z} \times$  Finset  $\mathbb{Z}$ )
  (hsub : (x1, some z, x2)  $\in$  ofFinset S5 S10) :
  z  $\in$  S5

```

lemma mem_ofFinset_Q5_subset

[\[source\]](#)

```

lemma mem_ofFinset_Q5_subset (S5 S10 : Finset  $\mathbb{Z}$ )
  {x : Charges} (hx : x  $\in$  ofFinset S5 S10) :
  x.2.2.1  $\subseteq$  S5

```

lemma mem_ofFinset_Q10_subset

[\[source\]](#)

```

lemma mem_ofFinset_Q10_subset (S5 S10 : Finset  $\mathbb{Z}$ )
  {x : Charges} (hx : x  $\in$  ofFinset S5 S10) :
  x.2.2.2  $\subseteq$  S10

```

lemma mem_ofFinset_antitone

[\[source\]](#)

```

lemma mem_ofFinset_antitone (S5 S10 : Finset  $\mathbb{Z}$ )
  {x y : Charges} (h : x  $\subseteq$  y) (hy : y  $\in$  ofFinset S5 S10) :
  x  $\in$  ofFinset S5 S10

```

lemma mem_ofFinset_iff

[\[source\]](#)

```

lemma mem_ofFinset_iff {S5 S10 : Finset  $\mathbb{Z}$ } {x : Charges} :
  x  $\in$  ofFinset S5 S10  $\leftrightarrow$  x.1.toFinset  $\subseteq$  S5  $\wedge$  x.2.1.toFinset  $\subseteq$  S5  $\wedge$ 
  x.2.2.1  $\subseteq$  S5  $\wedge$  x.2.2.2  $\subseteq$  S10

```

lemma ofFinset_subset_of_subset

[\[source\]](#)

```

lemma ofFinset_subset_of_subset {S5 S5' S10 S10' : Finset  $\mathbb{Z}$ }
  (h5 : S5  $\subseteq$  S5') (h10 : S10  $\subseteq$  S10') :
  ofFinset S5 S10  $\subseteq$  ofFinset S5' S10'

```

2.17 PhysLean.Particles.SuperSymmetry.SU5.Charges.Completions

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Completions.lean](#) on GitHub.

def IsComplete

[\[source\]](#)

A collection of charges is complete if it has all types of fields.

```

def IsComplete (x : Charges) : Prop

```

instance Decidable (IsComplete x)[\[source\]](#)

```
instance (x : Charges) : Decidable (IsComplete x)
```

lemma not_isComplete_empty[\[source\]](#)

```
@[simp]
lemma not_isComplete_empty : ¬ IsComplete ∅
```

lemma isComplete_mono[\[source\]](#)

```
lemma isComplete_mono {x y : Charges} (h : x ⊆ y) (hx : IsComplete x) :
  IsComplete y
```

def completions[\[source\]](#)

Given a collection of charges x in $\text{offFinset } S5 \ S10$, the minimal charges y in $\text{offFinset } S5 \ S10$ which are a super sets of x and are complete.

```
def completions (S5 S10 : Finset ℤ) (x : Charges) : Multiset Charges
```

lemma completions_eq_singleton_of_complete[\[source\]](#)

```
lemma completions_eq_singleton_of_complete {S5 S10 : Finset ℤ} (x : Charges)
  (hcomplete : IsComplete x) :
  completions S5 S10 x = {x}
```

lemma self_mem_completions_iff_isComplete[\[source\]](#)

```
@[simp]
lemma self_mem_completions_iff_isComplete {S5 S10 : Finset ℤ} (x : Charges) :
  x ∈ completions S5 S10 x ↔ IsComplete x
```

lemma mem_completions_isComplete[\[source\]](#)

```
lemma mem_completions_isComplete {S5 S10 : Finset ℤ} {x y : Charges}
  (hx : y ∈ completions S5 S10 x) : IsComplete y
```

lemma self_subset_mem_completions[\[source\]](#)

```
lemma self_subset_mem_completions (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ completions S5 S10 x) : x ⊆ y
```

lemma exist_completions_subset_of_complete[\[source\]](#)

```
lemma exist_completions_subset_of_complete (S5 S10 : Finset ℤ) (x y : Charges)
  (hsubset : x ⊆ y) (hy : y ∈ offFinset S5 S10) (hcomplete : IsComplete y) :
  ∃ z ∈ completions S5 S10 x, z ⊆ y
```

2.18 PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.Basic

[PhysLean/Particles/SuperSymmetry/SU5/Charges/MinimallyAllowsTerm/Basic.lean](#) on GitHub.

def MinimallyAllowsTerm[\[source\]](#)

A collection of charges $x : \text{Charges}$ is said to minimally allow the potential term T if it allows T and no strict subset of it allows T .

```
def MinimallyAllowsTerm (x : Charges) (T : PotentialTerm) : Prop
```

instance Decidable (x.MinimallyAllowsTerm T)[\[source\]](#)

```
instance (x : Charges) (T : PotentialTerm) : Decidable (x.MinimallyAllowsTerm T)
```

lemma allowsTerm_of_minimallyAllowsTerm[\[source\]](#)

```
lemma allowsTerm_of_minimallyAllowsTerm (h : x.MinimallyAllowsTerm T) : x.AllowsTerm T
```

lemma allowsTerm_of_has_minimallyAllowsTerm_subset[\[source\]](#)

```
lemma allowsTerm_of_has_minimallyAllowsTerm_subset
  (hx : ∃ y ∈ powerset x, y.MinimallyAllowsTerm T) : x.AllowsTerm T
```

lemma minimallyAllowsTerm_iff_powerset_filter_eq[\[source\]](#)

```
lemma minimallyAllowsTerm_iff_powerset_filter_eq :
  x.MinimallyAllowsTerm T ↔ x.powerset.filter (fun y => y.AllowsTerm T) = {x}
```

lemma minimallyAllowsTerm_iff_powerset_countP_eq_one[\[source\]](#)

```
lemma minimallyAllowsTerm_iff_powerset_countP_eq_one :
  x.MinimallyAllowsTerm T ↔ x.powerset.val.countP (fun y => y.AllowsTerm T) = 1
```

lemma subset_minimallyAllowsTerm_of_allowsTerm[\[source\]](#)

```
lemma subset_minimallyAllowsTerm_of_allowsTerm
  (hx : x.AllowsTerm T) : ∃ y ∈ powerset x, y.MinimallyAllowsTerm T
```

lemma allowsTerm_iff_subset_minimallyAllowsTerm[\[source\]](#)

```
lemma allowsTerm_iff_subset_minimallyAllowsTerm :
  x.AllowsTerm T ↔ ∃ y ∈ powerset x, y.MinimallyAllowsTerm T
```

lemma card_le_degree_of_minimallyAllowsTerm[\[source\]](#)

```
lemma card_le_degree_of_minimallyAllowsTerm (h : x.MinimallyAllowsTerm T) :
  x.card ≤ T.degree
```

lemma eq_allowsTermForm_of_minimallyAllowsTerm[\[source\]](#)

```
lemma eq_allowsTermForm_of_minimallyAllowsTerm (h1 : x.MinimallyAllowsTerm T) :
  ∃ a b c, x = allowsTermForm a b c T
```

lemma allowsTermForm_minimallyAllowsTerm[\[source\]](#)

```

lemma allowsTermForm_minimallyAllowsTerm {a b c :  $\mathbb{Z}$ } {T : PotentialTerm}
  (hT : T  $\neq$  W1  $\wedge$  T  $\neq$  W2) :
  MinimallyAllowsTerm (allowsTermForm a b c T) T

```

2.19 PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimallyAllowsTerm.OfFinset

[PhysLean/Particles/SuperSymmetry/SU5/Charges/MinimallyAllowsTerm/OfFinset.lean](#) on GitHub.

def toMultisetsOne[\[source\]](#)

The multisets of cardinality 1 containing elements from a finite set s.

```

def toMultisetsOne (s : Finset  $\mathbb{Z}$ ) : Multiset (Multiset  $\mathbb{Z}$ )

```

lemma mem_toMultisetsOne_iff[\[source\]](#)

```

@[simp]
lemma mem_toMultisetsOne_iff {s : Finset  $\mathbb{Z}$ } (X : Multiset  $\mathbb{Z}$ ) :
  X  $\in$  toMultisetsOne s  $\leftrightarrow$  X.toFinset  $\subseteq$  s  $\wedge$  X.card = 1

```

def toMultisetsTwo[\[source\]](#)

The multisets of cardinality 2 containing elements from a finite set s.

```

def toMultisetsTwo (s : Finset  $\mathbb{Z}$ ) : Multiset (Multiset  $\mathbb{Z}$ )

```

lemma mem_toMultisetsTwo_iff[\[source\]](#)

```

@[simp]
lemma mem_toMultisetsTwo_iff {s : Finset  $\mathbb{Z}$ } (X : Multiset  $\mathbb{Z}$ ) :
  X  $\in$  toMultisetsTwo s  $\leftrightarrow$  X.toFinset  $\subseteq$  s  $\wedge$  X.card = 2

```

def toMultisetsThree[\[source\]](#)

The multisets of cardinality 3 containing elements from a finite set s.

```

def toMultisetsThree (s : Finset  $\mathbb{Z}$ ) : Multiset (Multiset  $\mathbb{Z}$ )

```

lemma mem_toMultisetsThree_iff[\[source\]](#)

```

@[simp]
lemma mem_toMultisetsThree_iff {s : Finset  $\mathbb{Z}$ } (X : Multiset  $\mathbb{Z}$ ) :
  X  $\in$  toMultisetsThree s  $\leftrightarrow$  X.toFinset  $\subseteq$  s  $\wedge$  X.card = 3

```

def minimallyAllowsTermsOfFinset[\[source\]](#)

The multiset of all charges within ofFinset S5 S10 which minimally allow the potential term T.

```

def minimallyAllowsTermsOfFinset (S5 S10 : Finset  $\mathbb{Z}$ ) : (T : PotentialTerm)  $\rightarrow$  Multiset Charges
|  $\mu \Rightarrow$ 
  let SqHd

```

lemma eq_allowsTermForm_of_mem_minimallyAllowsTermOfFinset[\[source\]](#)

```
lemma eq_allowsTermForm_of_mem_minimallyAllowsTermOfFinset {S5 S10 : Finset  $\mathbb{Z}$ } {T : PotentialTerm}
  {x : Charges} (hx : x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T) :
   $\exists$  a b c, x = allowsTermForm a b c T
```

lemma allowsTerm_of_mem_minimallyAllowsTermOfFinset[\[source\]](#)

```
lemma allowsTerm_of_mem_minimallyAllowsTermOfFinset {S5 S10 : Finset  $\mathbb{Z}$ } {T : PotentialTerm}
  {x : Charges} (hx : x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T) :
  x.AllowsTerm T
```

lemma mem_minimallyAllowsTermOfFinset_of_minimallyAllowsTerm[\[source\]](#)

```
lemma mem_minimallyAllowsTermOfFinset_of_minimallyAllowsTerm {S5 S10 : Finset  $\mathbb{Z}$ }
  {T : PotentialTerm} (x : Charges) (h : x.MinimallyAllowsTerm T) (hx : x  $\in$  ofFinset S5 S10)
  :
  x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T
```

lemma minimallyAllowsTerm_of_mem_minimallyAllowsTermOfFinset[\[source\]](#)

```
lemma minimallyAllowsTerm_of_mem_minimallyAllowsTermOfFinset {S5 S10 : Finset  $\mathbb{Z}$ }
  {T : PotentialTerm} {x : Charges}
  (hx : x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T) :
  x.MinimallyAllowsTerm T
```

lemma mem_ofFinset_of_mem_minimallyAllowsTermOfFinset[\[source\]](#)

```
lemma mem_ofFinset_of_mem_minimallyAllowsTermOfFinset {S5 S10 : Finset  $\mathbb{Z}$ } {T : PotentialTerm}
  {x : Charges} (hx : x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T) :
  x  $\in$  ofFinset S5 S10
```

lemma minimallyAllowsTermOfFinset_subset_ofFinset[\[source\]](#)

```
lemma minimallyAllowsTermOfFinset_subset_ofFinset {S5 S10 : Finset  $\mathbb{Z}$ } {T : PotentialTerm} :
  minimallyAllowsTermsOfFinset S5 S10 T  $\subseteq$  (ofFinset S5 S10).val
```

lemma minimallyAllowsTerm_iff_mem_minimallyAllowsTermOfFinset[\[source\]](#)

```
lemma minimallyAllowsTerm_iff_mem_minimallyAllowsTermOfFinset
  {S5 S10 : Finset  $\mathbb{Z}$ } {T : PotentialTerm}
  {x : Charges} (hx : x  $\in$  ofFinset S5 S10) :
  x.MinimallyAllowsTerm T  $\leftrightarrow$  x  $\in$  minimallyAllowsTermsOfFinset S5 S10 T
```

lemma minimallyAllowsTermOfFinset_subset_of_subset[\[source\]](#)

```
lemma minimallyAllowsTermOfFinset_subset_of_subset {S5 S5' S10 S10'} {T : PotentialTerm}
  (h5 : S5'  $\subseteq$  S5) (h10 : S10'  $\subseteq$  S10) :
  minimallyAllowsTermsOfFinset S5' S10' T  $\subseteq$  minimallyAllowsTermsOfFinset S5 S10 T
```

2.20 PhysLean.Particles.SuperSymmetry.SU5.Charges.MinimalSuperSet

[PhysLean/Particles/SuperSymmetry/SU5/Charges/MinimalSuperSet.lean](#) on GitHub.

def minimalSuperSet [\[source\]](#)

Given a collection of charges x in `offFinset S5 S10`, the minimal charges y in `offFinset S5 S10` which are a super sets of x .

```
def minimalSuperSet (S5 S10 : Finset ℤ) (x : Charges) : Finset Charges
```

lemma self_subset_mem_minimalSuperSet [\[source\]](#)

```
lemma self_subset_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ minimalSuperSet S5 S10 x) : x ⊆ y
```

lemma self_not_mem_minimalSuperSet [\[source\]](#)

```
@[simp]
lemma self_not_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x : Charges) :
  x ∉ minimalSuperSet S5 S10 x
```

lemma self_neq_mem_minimalSuperSet [\[source\]](#)

```
lemma self_neq_mem_minimalSuperSet (S5 S10 : Finset ℤ) (x y : Charges)
  (hy : y ∈ minimalSuperSet S5 S10 x) : x ≠ y
```

lemma card_of_mem_minimalSuperSet [\[source\]](#)

```
lemma card_of_mem_minimalSuperSet {S5 S10 : Finset ℤ} {x : Charges}
  (y : Charges) (hy : y ∈ minimalSuperSet S5 S10 x) :
  card y = card x + 1
```

lemma insert_Q5_mem_minimalSuperSet [\[source\]](#)

```
lemma insert_Q5_mem_minimalSuperSet {S5 S10 : Finset ℤ} {x : Charges}
  (z : ℤ) (hz : z ∈ S5) (hznot : z ∉ x.2.2.1) :
  (x.1, x.2.1, insert z x.2.2.1, x.2.2.2) ∈ minimalSuperSet S5 S10 x
```

lemma insert_Q10_mem_minimalSuperSet [\[source\]](#)

```
lemma insert_Q10_mem_minimalSuperSet {S5 S10 : Finset ℤ} {x : Charges}
  (z : ℤ) (hz : z ∈ S10) (hznot : z ∉ x.2.2.2) :
  (x.1, x.2.1, x.2.2.1, insert z x.2.2.2) ∈ minimalSuperSet S5 S10 x
```

lemma some_qHd_mem_minimalSuperSet_of_none [\[source\]](#)

```
lemma some_qHd_mem_minimalSuperSet_of_none {S5 S10 : Finset ℤ} {x2 : Option ℤ × Finset ℤ ×
  Finset ℤ}
  (z : ℤ) (hz : z ∈ S5) :
  (some z, x2) ∈ minimalSuperSet S5 S10 (none, x2)
```

lemma some_qHu_mem_minimalSuperSet_of_none[\[source\]](#)

```

lemma some_qHu_mem_minimalSuperSet_of_none {S5 S10 : Finset ℤ}
  {x1 : Option ℤ} {x2 : Finset ℤ × Finset ℤ} (z : ℤ) (hz : z ∈ S5) :
  (x1, some z, x2) ∈ minimalSuperSet S5 S10 (x1, none, x2)

```

lemma exists_minimalSuperSet[\[source\]](#)

```

lemma exists_minimalSuperSet (S5 S10 : Finset ℤ) {x y : Charges}
  (hy : y ∈ offFinset S5 S10) (hsubset : x ⊆ y)
  (hxneqy : x ≠ y) : ∃ z ∈ minimalSuperSet S5 S10 x, z ⊆ y

```

lemma minimalSuperSet_induction_on_inductive[\[source\]](#)

```

lemma minimalSuperSet_induction_on_inductive {S5 S10 : Finset ℤ}
  (p : Charges → Prop) (hp : (x : Charges) → p x → ∀ y ∈ minimalSuperSet S5 S10 x, p y)
  (x : Charges) (hbase : p x)
  (y : Charges) (hy : y ∈ offFinset S5 S10) (hsubset : x ⊆ y) :
  (n : ℕ) → (hn : n = y.card - x.card) → p y
| 0, hn => by
have hxy : x = y

```

2.21 PhysLean.Particles.SuperSymmetry.SU5.Charges.OfFieldLabel

[PhysLean/Particles/SuperSymmetry/SU5/Charges/OfFieldLabel.lean](#) on GitHub.

def offFieldLabel[\[source\]](#)

Given an $x : \text{Charges}$, the charges associated with a given FieldLabel .

```

def offFieldLabel (x : Charges) : FieldLabel → Finset ℤ
| .fiveBarHd => x.1.toFinset
| .fiveBarHu => x.2.1.toFinset
| .fiveBarMatter => x.2.2.1
| .tenMatter => x.2.2.2
| .fiveHd => x.1.toFinset.map ⟨Neg.neg, neg_injective⟩
| .fiveHu => x.2.1.toFinset.map ⟨Neg.neg, neg_injective⟩
| .fiveMatter => x.2.2.1.map ⟨Neg.neg, neg_injective⟩

```

lemma offFieldLabel_empty[\[source\]](#)

```

@[simp]
lemma offFieldLabel_empty (F : FieldLabel) :
  offFieldLabel ∅ F = ∅

```

lemma offFieldLabel_mono[\[source\]](#)

```

lemma offFieldLabel_mono {x y : Charges} (h : x ⊆ y) (F : FieldLabel) :
  x.offFieldLabel F ⊆ y.offFieldLabel F

```

lemma mem_offFieldLabel_fiveHd[\[source\]](#)

```

@[simp]
lemma mem_offFieldLabel_fiveHd (x : ℤ) (y : Charges) :

```

```
x ∈ y.ofFieldLabel FieldLabel.fiveHd ↔ -x ∈ y.ofFieldLabel .fiveBarHd
```

lemma mem_ofFieldLabel_fiveHu

[\[source\]](#)

@[simp]

```
lemma mem_ofFieldLabel_fiveHu (x : ℤ) (y : Charges) :
  x ∈ y.ofFieldLabel FieldLabel.fiveHu ↔ -x ∈ y.ofFieldLabel .fiveBarHu
```

lemma mem_ofFieldLabel_fiveMatter

[\[source\]](#)

@[simp]

```
lemma mem_ofFieldLabel_fiveMatter (x : ℤ) (y : Charges) :
  x ∈ y.ofFieldLabel FieldLabel.fiveMatter ↔ -x ∈ y.ofFieldLabel .fiveBarMatter
```

lemma ext_ofFieldLabel

[\[source\]](#)

Two charges are equal if they are equal on all field labels.

```
lemma ext_ofFieldLabel {x y : Charges} (h : ∀ F, x.ofFieldLabel F = y.ofFieldLabel F) :
  x = y
```

2.22 PhysLean.Particles.SuperSymmetry.SU5.Charges.OfPotentialTerm

[PhysLean/Particles/SuperSymmetry/SU5/Charges/OfPotentialTerm.lean](#) on GitHub.

def ofPotentialTerm

[\[source\]](#)

Given a charges $x : \text{Charges}$ associated to the representations, and a potential term T , the charges associated with instances of that potential term.

```
def ofPotentialTerm (x : Charges) (T : PotentialTerm) : Multiset ℤ
```

lemma ofPotentialTerm_mono

[\[source\]](#)

```
lemma ofPotentialTerm_mono {x y : Charges} (h : x ⊆ y) (T : PotentialTerm) :
  x.ofPotentialTerm T ⊆ y.ofPotentialTerm T
```

lemma ofPotentialTerm_empty

[\[source\]](#)

@[simp]

```
lemma ofPotentialTerm_empty (T : PotentialTerm) :
  ofPotentialTerm ∅ T = ∅
```

def ofPotentialTerm'

[\[source\]](#)

Given a charges $x : \text{Charges}$ associated to the representations, and a potential term T , the charges associated with instances of that potential term.

This is a more explicit form of `PotentialTerm`, which has the benefit that it is quick with `decide`, but it is not defined based on more fundamental concepts, like `ofPotentialTerm` is.

```
def ofPotentialTerm' (y : Charges) (T : PotentialTerm) : Multiset ℤ
```

lemma ofPotentialTerm_subset_ofPotentialTerm'[\[source\]](#)

```
lemma ofPotentialTerm_subset_ofPotentialTerm' {x : Charges} (T : PotentialTerm) :
  x.ofPotentialTerm T  $\subseteq$  x.ofPotentialTerm' T
```

lemma ofPotentialTerm'_subset_ofPotentialTerm[\[source\]](#)

```
lemma ofPotentialTerm'_subset_ofPotentialTerm {x : Charges} (T : PotentialTerm) :
  x.ofPotentialTerm' T  $\subseteq$  x.ofPotentialTerm T
```

lemma mem_ofPotentialTerm_iff_mem_ofPotentialTerm[\[source\]](#)

```
lemma mem_ofPotentialTerm_iff_mem_ofPotentialTerm {T : PotentialTerm} {n :  $\mathbb{Z}$ } {y : Charges} :
  n  $\in$  y.ofPotentialTerm T  $\leftrightarrow$  n  $\in$  y.ofPotentialTerm' T
```

2.23 PhysLean.Particles.SuperSymmetry.SU5.Charges.PhenoConstrained

[PhysLean/Particles/SuperSymmetry/SU5/Charges/PhenoConstrained.lean](#) on GitHub.

def IsPhenoConstrained[\[source\]](#)

A charge is pheno-constrained if it leads to the presence of any term causing proton decay $\{W1, \Lambda, W2, K1\}$ or R-parity violation $\{\beta, \Lambda, W2, W4, K1, K2\}$.

```
def IsPhenoConstrained (x : Charges) : Prop
```

instance decidableIsPhenoConstrained[\[source\]](#)

```
instance decidableIsPhenoConstrained (x : Charges) : Decidable x.IsPhenoConstrained
```

lemma not_isPhenoConstrained_empty[\[source\]](#)

```
@[simp]
lemma not_isPhenoConstrained_empty :
   $\neg$  IsPhenoConstrained  $\emptyset$ 
```

lemma isPhenoConstrained_mono[\[source\]](#)

```
lemma isPhenoConstrained_mono {x y : Charges} (h : x  $\subseteq$  y)
  (hx : x.IsPhenoConstrained) : y.IsPhenoConstrained
```

def phenoConstrainingChargesSP[\[source\]](#)

The collection of charges of super-potential terms leading to a pheno-constrained model.

```
def phenoConstrainingChargesSP (x : Charges) : Multiset  $\mathbb{Z}$ 
```

lemma phenoConstrainingChargesSP_empty[\[source\]](#)

```
@[simp]
lemma phenoConstrainingChargesSP_empty :
  phenoConstrainingChargesSP  $\emptyset$  =  $\emptyset$ 
```

lemma phenoConstrainingChargesSP_mono[\[source\]](#)

```
lemma phenoConstrainingChargesSP_mono {x y : Charges} (h : x ≤ y) :
  x.phenoConstrainingChargesSP ≤ y.phenoConstrainingChargesSP
```

def Twig.phenoInsertQ10[\[source\]](#)

The twig obtained by taking the new, not pheno-constrained, charges obtained by inserting q10 into all leafs of a twig. This assumes that all existing charges in the twig are already not pheno constrained.

```
def Twig.phenoInsertQ10 (t : Twig (Finset ℤ) (Finset ℤ)) (qHd : Option ℤ) (x : ℤ) :
  Twig (Finset ℤ) (Finset ℤ)
```

def Branch.phenoInsertQ10[\[source\]](#)

The branch obtained by taking the new, not pheno-constrained, charges obtained by inserting q10 into all leafs of a branch. This assumes that all existing charges in the branch are already not pheno constrained.

```
def Branch.phenoInsertQ10 (b : Branch (Option ℤ) (Finset ℤ) (Finset ℤ)) (qHd : Option ℤ) (x :
  ℤ) :
  Branch (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Trunk.phenoInsertQ10[\[source\]](#)

The trunk obtained by taking the new, not pheno-constrained, charges obtained by inserting q10 into all leafs of a trunk. This assumes that all existing charges in the trunk are already not pheno constrained.

```
def Trunk.phenoInsertQ10 (T : Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (x : ℤ) :
  Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Tree.phenoInsertQ10[\[source\]](#)

The tree obtained by taking the new, not pheno-constrained, charges obtained by inserting q10 into all leafs of a tree. This assumes that all existing charges in the tree are already not pheno constrained.

```
def phenoInsertQ10 (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (x : ℤ) :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

lemma mem_phenoInsertQ10_of_mem_allowsTerm[\[source\]](#)

```
lemma mem_phenoInsertQ10_of_mem_allowsTerm
  (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (q10 : ℤ) (C : Charges)
  (h : C ∈ (T.uniqueMap4 (insert q10))) (hC : ¬ AllowsTerm (C.1, C.2.1, ∅, {q10}) K2
    ∧ ¬ AllowsTerm (none, none, C.2.2.1, {q10}) Λ
    ∧ ¬ AllowsTerm (none, none, C.2.2.1, C.2.2.2) W1 ∧
    ¬ AllowsTerm (none, none, C.2.2.1, C.2.2.2) K1
    ∧ ¬ AllowsTerm (C.1, none, ∅, C.2.2.2) W2) :
  C ∈ phenoInsertQ10 T q10
```

def Branch.phenoInsertQ5[\[source\]](#)

The branch obtained by taking the new, not pheno-constrained, charges obtained by inserting q5 into all leafs of a branch. This assumes that all existing charges in the branch are already not pheno constrained.

```
def Branch.phenoInsertQ5 (b : Branch (Option ℤ) (Finset ℤ) (Finset ℤ)) (qHd : Option ℤ) (x : ℤ) :
  :
  Branch (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Trunk.phenoInsertQ5[\[source\]](#)

The trunk obtained by taking the new, not pheno-constrained, charges obtained by inserting q5 into all leafs of a trunk. This assumes that all existing charges in the trunk are already not pheno constrained.

```
def Trunk.phenoInsertQ5 (T : Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (x : ℤ) :
  Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Tree.phenoInsertQ5[\[source\]](#)

The tree obtained by taking the new, not pheno-constrained, charges obtained by inserting q5 into all leafs of a tree. This assumes that all existing charges in the tree are already not pheno constrained.

```
def phenoInsertQ5 (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (x : ℤ) :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

lemma mem_phenoInsertQ5_of_mem_allowsTerm[\[source\]](#)

```
lemma mem_phenoInsertQ5_of_mem_allowsTerm (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (
  Finset ℤ))
  (q5 : ℤ) (C : Charges)
  (h : C ∈ (T.uniqueMap3 (insert q5))) (hC : ¬ AllowsTerm (none, C.2.1, {q5}, ∅) β
    ∧ ¬ AllowsTerm (C.1, C.2.1, {q5}, ∅) W4 ∧
    ¬ AllowsTerm (none, none, {q5}, C.2.2.2) W1 ∧ ¬ AllowsTerm (none, none, {q5}, C.2.2.2) K1
    ∧ ¬ AllowsTerm (none, none, C.2.2.1, C.2.2.2) ∧) :
  C ∈ phenoInsertQ5 T q5
```

2.24 PhysLean.Particles.SuperSymmetry.SU5.Charges.Tree

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Tree.lean](#) on GitHub.

instance Decidable (x ∈ T)[\[source\]](#)

```
instance (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (x : Charges) :
  Decidable (x ∈ T)
```

def root[\[source\]](#)

An explicit form of FourTree.root for charges.

```
def root : Multiset (Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) →
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def trunk[\[source\]](#)*An explicit form of FourTree.Trunk.trunk for charges.*

```
def trunk : (Option ℤ) → Multiset (Branch (Option ℤ) (Finset ℤ) (Finset ℤ)) →
  Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def branch[\[source\]](#)*An explicit form of FourTree.Branch.branch for charges.*

```
def branch : (Option ℤ) → Multiset (Twig (Finset ℤ) (Finset ℤ)) →
  Branch (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def twig[\[source\]](#)*An explicit form of FourTree.Twig.twig for charges.*

```
def twig : Finset ℤ → Multiset (Leaf (Finset ℤ)) → Twig (Finset ℤ) (Finset ℤ)
```

def leaf[\[source\]](#)*An explicit form of FourTree.Leaf.leaf for charges.*

```
def leaf : Finset ℤ → Leaf (Finset ℤ)
```

lemma insert_filter_card_zero[\[source\]](#)

```
lemma insert_filter_card_zero
  (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)) (S5 S10 : Finset ℤ)
  (p : Charges → Prop) [DecidablePred p]
  (hCompleat : ∀ x ∈ T, IsComplete x)
  (h10 : ∀ q10 : S10, (T.uniqueMap4 (insert q10.1)).toMultiset.filter p = ∅)
  (h5 : ∀ q5 : S5, (T.uniqueMap3 (insert q5.1)).toMultiset.filter p = ∅) :
  ∀ x ∈ T, ∀ y ∈ minimalSuperSet S5 S10 x, y ∉ T → ¬ p y
```

lemma subset_insert_filter_card_zero_inductive[\[source\]](#)

```
lemma subset_insert_filter_card_zero_inductive
  (T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ))
  (S5 S10 : Finset ℤ)
  (p : Charges → Prop) [DecidablePred p]
  (hnotSubset : ∀ (x y : Charges), x ≤ y → ¬ p x → ¬ p y)
  (hCompleat : ∀ x ∈ T, IsComplete x)
  (x : Charges)
  (hx : x ∈ T) (y : Charges) (hsubset : x ≤ y)
  (hy : y ∈ ofFinset S5 S10)
  (h10 : ∀ q10 : S10, (T.uniqueMap4 (insert q10.1)).toMultiset.filter p = ∅)
  (h5 : ∀ q5 : S5, (T.uniqueMap3 (insert q5.1)).toMultiset.filter p = ∅) :
  (n : ℕ) → (hn : n = y.card - x.card) → y ∉ T → ¬ p y
| 0, hn, hnot_in_T => by
  have hxy : x = y
```

lemma subset_insert_filter_card_zero[\[source\]](#)

For a proposition p if $(T.\text{uniqueMap4 } (\text{insert } q10.1)).\text{toMultiset}.\text{filter } p$ and $(T.\text{uniqueMap3 } (\text{insert } q5.1)).\text{toMultiset}.\text{filter } p$ for all $q5 \in S5$ and $q10 \in S10$ then if $x \in T$ and $x \subseteq y$ if $y \notin T$ then $\neg p \ y$. This assumes that all charges in T are complete, and that p satisfies $x \subseteq y \rightarrow \neg p \ x \rightarrow \neg p \ y$.

lemma subset_insert_filter_card_zero

```
(T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ))
(S5 S10 : Finset ℤ)
(p : Charges → Prop) [DecidablePred p]
(hnotSubset : ∀ (x y : Charges), x ⊆ y → ¬ p x → ¬ p y)
(hCompleat : ∀ x ∈ T, IsComplete x)
(x : Charges)
(hx : x ∈ T) (y : Charges) (hsubset : x ⊆ y)
(hy : y ∈ offinset S5 S10)
(h10 : ∀ q10 : S10, (T.uniqueMap4 (insert q10.1)).toMultiset.filter p = ∅)
(h5 : ∀ q5 : S5, (T.uniqueMap3 (insert q5.1)).toMultiset.filter p = ∅) :
  y ∉ T → ¬ p y
```

2.25 PhysLean.Particles.SuperSymmetry.SU5.Charges.Yukawa

[PhysLean/Particles/SuperSymmetry/SU5/Charges/Yukawa.lean](#) on GitHub.

def ofYukawaTerms[\[source\]](#)

The collection of charges associated with Yukawa terms. Correspondingly, the (negative) of the charges of the singlets needed to regenerate all Yukawa terms in the potential.

```
def ofYukawaTerms (x : Charges) : Multiset ℤ
```

lemma ofYukawaTerms_subset_of_subset[\[source\]](#)

```
lemma ofYukawaTerms_subset_of_subset {x y : Charges} (h : x ⊆ y) :
  x.ofYukawaTerms ⊆ y.ofYukawaTerms
```

def ofYukawaTermsNSum[\[source\]](#)

The charges of those terms which can be regenerated with up-to n insertions of singlets needed to regenerate the Yukawa terms. Equivalently, the sum of up-to n integers each corresponding to a charge of the Yukawa terms.

```
def ofYukawaTermsNSum (x : Charges) : ℕ → Multiset ℤ
| 0 => {0}
| n + 1 => x.ofYukawaTermsNSum n + (x.ofYukawaTermsNSum n).bind fun sSum =>
  (x.ofYukawaTerms.map fun s => sSum + s)
```

lemma ofYukawaTermsNSum_subset_of_subset[\[source\]](#)

```
lemma ofYukawaTermsNSum_subset_of_subset {x y : Charges} (h : x ⊆ y) (n : ℕ) :
  x.ofYukawaTermsNSum n ⊆ y.ofYukawaTermsNSum n
```

def YukawaGeneratesDangerousAtLevel[\[source\]](#)

For charges $x : \text{Charges}$, the proposition which states that the singlets needed to regenerate the Yukawa couplings regenerate a dangerous coupling (in the superpotential) with up-to n insertions of the scalars.

```
def YukawaGeneratesDangerousAtLevel (x : Charges) (n : ℕ) : Prop
```

lemma not_yukawaGeneratesDangerousAtLevel_of_empty[\[source\]](#)

```
@[simp]
```

```
lemma not_yukawaGeneratesDangerousAtLevel_of_empty (n : ℕ) :  
  ¬ YukawaGeneratesDangerousAtLevel ∅ n
```

instance Decidable (YukawaGeneratesDangerousAtLevel x n)[\[source\]](#)

```
instance (x : Charges) (n : ℕ) : Decidable (YukawaGeneratesDangerousAtLevel x n)
```

lemma yukawaGeneratesDangerousAtLevel_of_subset[\[source\]](#)

```
lemma yukawaGeneratesDangerousAtLevel_of_subset {x y : Charges} {n : ℕ} (h : x ⊆ y)  
  (hx : x.YukawaGeneratesDangerousAtLevel n) :  
  y.YukawaGeneratesDangerousAtLevel n
```

lemma yukawaGeneratesDangerousAtLevel_succ[\[source\]](#)

```
lemma yukawaGeneratesDangerousAtLevel_succ {x : Charges} {n : ℕ}  
  (hx : x.YukawaGeneratesDangerousAtLevel n) :  
  x.YukawaGeneratesDangerousAtLevel (n + 1)
```

lemma yukawaGeneratesDangerousAtLevel_add_of_left[\[source\]](#)

```
lemma yukawaGeneratesDangerousAtLevel_add_of_left {x : Charges} {n k : ℕ}  
  (hx : x.YukawaGeneratesDangerousAtLevel n) :  
  x.YukawaGeneratesDangerousAtLevel (n + k)
```

lemma yukawaGeneratesDangerousAtLevel_of_le[\[source\]](#)

```
lemma yukawaGeneratesDangerousAtLevel_of_le {x : Charges} {n m : ℕ}  
  (h : n ≤ m) (hx : x.YukawaGeneratesDangerousAtLevel n) :  
  x.YukawaGeneratesDangerousAtLevel m
```

2.26 PhysLean.Particles.SuperSymmetry.SU5.FieldLabels

[PhysLean/Particles/SuperSymmetry/SU5/FieldLabels.lean](#) on GitHub.

inductive FieldLabel[\[source\]](#)

The types of field present in an $SU(5)$ GUT.

```
inductive FieldLabel  
  | fiveBarHu  
  | fiveHu
```

```
| fiveBarHd
| fiveHd
| fiveBarMatter
| fiveMatter
| tenMatter
```

def RParity[\[source\]](#)

The R-Parity of a field, landing on 1 if it is in the non-trivial representation and 0 otherwise.

```
def FieldLabel.RParity : FieldLabel → Fin 2
| fiveBarHu => 0
| fiveHu => 0
| fiveBarHd => 0
| fiveHd => 0
| fiveBarMatter => 1
| fiveMatter => 1
| tenMatter => 1
```

2.27 PhysLean.Particles.SuperSymmetry.SU5.Potential

[PhysLean/Particles/SuperSymmetry/SU5/Potential.lean](#) on GitHub.

inductive PotentialTerm[\[source\]](#)

Relevant terms part of the superpotential and Kahler potential of the SU(5) SUSY GUT.

```
inductive PotentialTerm
| μ : PotentialTerm
| β : PotentialTerm
| Λ : PotentialTerm
| W1 : PotentialTerm
| W2 : PotentialTerm
| W3 : PotentialTerm
| W4 : PotentialTerm
| K1 : PotentialTerm
| K2 : PotentialTerm
| topYukawa : PotentialTerm
| bottomYukawa : PotentialTerm
```

def toFieldLabel[\[source\]](#)

The fields contained within a given term of the potential.

```
def toFieldLabel : PotentialTerm → List FieldLabel
| μ => [.fiveBarHd, .fiveHu]
| β => [.fiveHu, .fiveBarMatter]
| Λ => [.fiveBarMatter, .fiveBarMatter, .tenMatter]
| W1 => [.tenMatter, .tenMatter, .tenMatter, .fiveBarMatter]
| W2 => [.tenMatter, .tenMatter, .tenMatter, .fiveBarHd]
| W3 => [.fiveBarMatter, .fiveBarMatter, .fiveHu, .fiveHu]
| W4 => [.fiveBarMatter, .fiveBarHd, .fiveHu, .fiveHu]
| K1 => [.tenMatter, .tenMatter, .fiveMatter]
| K2 => [.fiveBarHu, .fiveBarHd, .tenMatter]
```

```
| topYukawa => [.tenMatter, .tenMatter, .fiveHu]
| bottomYukawa => [.tenMatter, .fiveBarMatter, .fiveBarHd]
```

def InSuperPotential[\[source\]](#)

The proposition which is true on those terms which are members of the super potential.

```
def InSuperPotential : PotentialTerm → Prop
| μ => True
| β => True
| Λ => True
| W1 => True
| W2 => True
| W3 => True
| W4 => True
| K1 => False
| K2 => False
| topYukawa => True
| bottomYukawa => True
```

instance (T : PotentialTerm) → Decidable (InSuperPotential T) | μ => inferInstan... [\[source\]](#)

```
instance : (T : PotentialTerm) → Decidable (InSuperPotential T)
| μ => inferInstanceAs (Decidable True)
| β => inferInstanceAs (Decidable True)
| Λ => inferInstanceAs (Decidable True)
| W1 => inferInstanceAs (Decidable True)
| W2 => inferInstanceAs (Decidable True)
| W3 => inferInstanceAs (Decidable True)
| W4 => inferInstanceAs (Decidable True)
| K1 => inferInstanceAs (Decidable False)
| K2 => inferInstanceAs (Decidable False)
| topYukawa => inferInstanceAs (Decidable True)
| bottomYukawa => inferInstanceAs (Decidable True)
```

lemma no_conjugate_in_toFieldLabel_of_inSuperPotential[\[source\]](#)

The terms within the super-potential contain no conjugate fields.

```
lemma no_conjugate_in_toFieldLabel_of_inSuperPotential {T : PotentialTerm}
(h : T.InSuperPotential) : FieldLabel.fiveMatter ∉ T.toFieldLabel ∧
FieldLabel.fiveHd ∉ T.toFieldLabel ∧ FieldLabel.fiveBarHu ∉ T.toFieldLabel
```

def degree[\[source\]](#)

The degree of a term in the potential.

```
def degree (T : PotentialTerm) : ℕ
```

lemma degree_le_four[\[source\]](#)

```
lemma degree_le_four (T : PotentialTerm) : T.degree ≤ 4
```

def RParity[\[source\]](#)

The R-parity of a term in the potential.

```
def RParity (T : PotentialTerm) : Fin 2
```

lemma violates_RParity_iff_mem[\[source\]](#)

```
lemma violates_RParity_iff_mem {T : PotentialTerm} :  
  T.RParity = 1 ↔ T ∈ ({β, Λ, W2, W4, K1, K2} : Finset PotentialTerm)
```

def causeProtonDecay[\[source\]](#)

The finite set of terms in the superpotential and Kahler potential which are involved in proton decay. $- W^1_{ijk} 10^i 10^j 10^k 5^M{}^i - \lambda_{ijk} 5^M{}^i 5^M{}^j 10^k - W^2_{ijk} 10^i 10^j 10^k 5^Hd - K^1_{ijk} 10^i 10^j 5M^k$

```
def causeProtonDecay : Finset PotentialTerm
```

2.28 PhysLean.QuantumMechanics.PlanckConstant

[PhysLean/QuantumMechanics/PlanckConstant.lean](#) on GitHub.

axiom ħ[\[source\]](#)

Planck's constant.

```
axiom ħ : Subtype fun x : ℝ => 0 < x
```

lemma ħ_pos[\[source\]](#)

Planck's constant is positive.

```
lemma ħ_pos : 0 < (ħ : ℝ)
```

lemma ħ_nonneg[\[source\]](#)

Planck's constant is non-negative.

```
lemma ħ_nonneg : 0 ≤ (ħ : ℝ)
```

lemma ħ_ne_zero[\[source\]](#)

Planck's constant is not equal to zero.

```
lemma ħ_ne_zero : (ħ : ℝ) ≠ 0
```

2.29 PhysLean.Relativity.LorentzGroup.Boosts.Generalized

[PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean](#) on GitHub.

lemma genBoost_mul_one_plus_contr[\[source\]](#)

```

lemma genBoost_mul_one_plus_contr (u v : Velocity d) (x : Vector d) :
  (1 + ⟨⟨u, v.1⟩⟩m) • genBoost u v x =
  (1 + ⟨⟨u, v.1⟩⟩m) • x + (2 * ⟨⟨x, u⟩⟩m * (1 + ⟨⟨u, v.1⟩⟩m)) • v - ⟨⟨x, u + v⟩⟩m • (u + v)

```

lemma genBoostAux₁_apply_basis[\[source\]](#)

```

lemma genBoostAux1_apply_basis (u v : Velocity d) (μ : Fin 1 ⊗ Fin d) :
  (genBoostAux1 u v) (Vector.basis μ) = (2 * η μ μ * toCoord u μ) • v

```

lemma genBoostAux₂_apply_basis[\[source\]](#)

```

lemma genBoostAux2_apply_basis (u v : Velocity d) (μ : Fin 1 ⊗ Fin d) :
  (genBoostAux2 u v) (Vector.basis μ) =
  - (η μ μ * (toCoord u μ + toCoord v μ) / (1 + ⟨⟨u.1, v.1⟩⟩m)) • (u.1 + v.1)

```

lemma genBoost_apply_basis[\[source\]](#)

```

lemma genBoost_apply_basis (u v : Velocity d) (μ : Fin 1 ⊗ Fin d) :
  (genBoost u v) (Vector.basis μ) =
  Vector.basis μ + (2 * η μ μ * toCoord u μ) • v - (η μ μ * (toCoord u μ + toCoord v μ)
  / (1 + ⟨⟨u.1, v.1⟩⟩m)) • (u.1 + v.1)

```

lemma genBoostAux₁_toMatrix_apply[\[source\]](#)

```

lemma genBoostAux1_toMatrix_apply (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  (LinearMap.toMatrix Vector.basis Vector.basis (genBoostAux1 u v)) μ v =
  η v v * (2 * toCoord u v * toCoord v μ)

```

lemma genBoostAux₂_toMatrix_apply[\[source\]](#)

```

lemma genBoostAux2_toMatrix_apply (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  (LinearMap.toMatrix Vector.basis Vector.basis (genBoostAux2 u v)) μ v =
  η v v * (- ((toCoord u.1 μ) + (toCoord v.1 μ)) * ((toCoord u.1 v) + (toCoord v.1 v))
  / (1 + ⟨⟨u.1, v.1⟩⟩m))

```

lemma toMatrix_apply_eq_minkowskiProduct[\[source\]](#)

```

lemma toMatrix_apply_eq_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  (toMatrix u v) μ v = η μ μ * (⟨⟨Vector.basis μ, Vector.basis v⟩⟩m + 2 *
  ⟨⟨Vector.basis v, u⟩⟩m * ⟨⟨Vector.basis μ, v.1⟩⟩m
  - ⟨⟨Vector.basis μ, u + v⟩⟩m * ⟨⟨Vector.basis v, u + v⟩⟩m / (1 + ⟨⟨u.1, v.1⟩⟩m))

```

lemma toMatrix_apply_eq_toCoord[\[source\]](#)

```

lemma toMatrix_apply_eq_toCoord (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
  (toMatrix u v) μ v = ((1 : Matrix (Fin 1 ⊗ Fin d) (Fin 1 ⊗ Fin d) ℝ) μ v +
  2 * η v v * toCoord u v * toCoord v μ
  - η v v * (toCoord u μ + toCoord v μ) * (toCoord u v + toCoord v v) / (1 + ⟨⟨u.1, v.1⟩⟩m))

```

lemma toMatrix_continuous_snd[\[source\]](#)

@[fun_prop]

lemma toMatrix_continuous_snd (u : Velocity d) : Continuous (toMatrix u)**lemma toMatrix_continuous_fst**[\[source\]](#)

@[fun_prop]

lemma toMatrix_continuous_fst (u : Velocity d) : Continuous (toMatrix · u)**lemma genBoostAux₁_basis_minkowskiProduct**[\[source\]](#)

lemma genBoostAux₁_basis_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
 $\langle\langle \text{genBoostAux}_1 u v (\text{Vector.basis } \mu), \text{genBoostAux}_1 u v (\text{Vector.basis } v) \rangle\rangle_m =$
 $4 * \eta \mu \mu * \eta v v * \text{toCoord } u \mu * \text{toCoord } u v$

lemma genBoostAux₂_basis_minkowskiProduct[\[source\]](#)

lemma genBoostAux₂_basis_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
 $\langle\langle \text{genBoostAux}_2 u v (\text{Vector.basis } \mu), \text{genBoostAux}_2 u v (\text{Vector.basis } v) \rangle\rangle_m =$
 $2 * \eta \mu \mu * \eta v v * (\text{toCoord } u \mu + \text{toCoord } v \mu) * (\text{toCoord } u v + \text{toCoord } v v)$
 $* (1 + \langle\langle u, v.1 \rangle\rangle_m)^{-1}$

lemma genBoostAux₁_basis_genBoostAux₂_minkowskiProduct[\[source\]](#)

lemma genBoostAux₁_basis_genBoostAux₂_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
 $\langle\langle \text{genBoostAux}_1 u v (\text{Vector.basis } \mu), \text{genBoostAux}_2 u v (\text{Vector.basis } v) \rangle\rangle_m =$
 $- 2 * \eta \mu \mu * \eta v v * \text{toCoord } u \mu * (\text{toCoord } u v + \text{toCoord } v v)$

lemma genBoostAux₁_add_genBoostAux₂_minkowskiProduct[\[source\]](#)

lemma genBoostAux₁_add_genBoostAux₂_minkowskiProduct (u v : Velocity d) (μ v : Fin 1 ⊗ Fin d) :
 $\langle\langle \text{genBoostAux}_1 u v (\text{Vector.basis } \mu) + \text{genBoostAux}_2 u v (\text{Vector.basis } \mu),$
 $\text{genBoostAux}_1 u v (\text{Vector.basis } v) + \text{genBoostAux}_2 u v (\text{Vector.basis } v) \rangle\rangle_m =$
 $2 * \eta \mu \mu * \eta v v * (- \text{toCoord } u \mu * (\text{toCoord } u v + \text{toCoord } v v)$
 $- \text{toCoord } u v * (\text{toCoord } u \mu + \text{toCoord } v \mu)$
 $+ (\text{toCoord } u \mu + \text{toCoord } v \mu) * (\text{toCoord } u v + \text{toCoord } v v) * (1 + \langle\langle u, v.1 \rangle\rangle_m)^{-1} +$
 $2 * \text{toCoord } u \mu * \text{toCoord } u v)$

lemma basis_minkowskiProduct_genBoostAux₁_add_genBoostAux₂[\[source\]](#)

lemma basis_minkowskiProduct_genBoostAux₁_add_genBoostAux₂ (u v : Velocity d)
(μ v : Fin 1 ⊗ Fin d) :
 $\langle\langle \text{Vector.basis } \mu, \text{genBoostAux}_1 u v (\text{Vector.basis } v) + \text{genBoostAux}_2 u v (\text{Vector.basis } v) \rangle\rangle_m =$
 $\eta \mu \mu * \eta v v * (2 * \text{toCoord } u v * \text{toCoord } v \mu$
 $- (\text{toCoord } u \mu + \text{toCoord } v \mu) * (\text{toCoord } u v + \text{toCoord } v v) * (1 + \langle\langle u.1, v.1 \rangle\rangle_m)^{-1})$

lemma isOrthochronous[\[source\]](#)**lemma** isOrthochronous (u v : Velocity d) : IsOrthochronous (toLorentz u v)

lemma toLorentz_mem_restricted[\[source\]](#)

```
lemma toLorentz_mem_restricted (u v : Velocity d) :
  toLorentz u v ∈ LorentzGroup.restricted d
```

semiformal_result toMatrix_timeComponent_eq[\[source\]](#)

The time component of a generalised boost is equal to $\frac{1 + \|u.1.timeComponent \cdot v.1.spatialPart - v.1.timeComponent \cdot u.1.spatialPart\|}{(1 + \langle\langle u.1, v.1 \rangle\rangle_m)}$

A proof of this result can be found at the below link: <https://leanprover.zulipchat.com/#narrow/channel/47PhysLean/topic/Lorentz.20group/near/523249684>

Note that the declaration of this semiformal result will be similar once the TODO item FXQ45 is completed.

```
semiformal_result "FXNQY" toMatrix_timeComponent_eq (u v : Velocity d) :
  (toMatrix u v) (Sum.inl 0) (Sum.inl 0) = 1 +
    \|u.1.timeComponent \cdot v.1.spatialPart - v.1.timeComponent \cdot u.1.spatialPart\| / (1 + \langle\langle u.1, v.1 \rangle\rangle_m)
```

2.30 PhysLean.Relativity.LorentzGroup.Orthochronous

[PhysLean/Relativity/LorentzGroup/Orthochronous.lean](#) on GitHub.

lemma isOrthochronous_iff_futurePointing[\[source\]](#)

A Lorentz transformation is orthochronous if and only if its first column is future pointing.

```
lemma isOrthochronous_iff_futurePointing :
  IsOrthochronous Λ ↔ toNormOne Λ ∈ NormOne.FuturePointing d
```

lemma isOrthochronous_iff_transpose[\[source\]](#)

A Lorentz transformation is orthochronous if and only if its transpose is orthochronous.

```
lemma isOrthochronous_iff_transpose :
  IsOrthochronous Λ ↔ IsOrthochronous (transpose Λ)
```

lemma isOrthochronous_iff_ge_one[\[source\]](#)

A Lorentz transformation is orthochronous if and only if its $0 \ 0$ element is greater or equal to one.

```
lemma isOrthochronous_iff_ge_one :
  IsOrthochronous Λ ↔ 1 ≤ Λ.1 (Sum.inl 0) (Sum.inl 0)
```

lemma not_isOrthochronous_iff_le_neg_one[\[source\]](#)

A Lorentz transformation is not orthochronous if and only if its $0 \ 0$ element is less than or equal to minus one.

```
lemma not_isOrthochronous_iff_le_neg_one :
  ¬ IsOrthochronous Λ ↔ Λ.1 (Sum.inl 0) (Sum.inl 0) ≤ -1
```

lemma not_isOrthochronous_iff_le_zero[\[source\]](#)

A Lorentz transformation is not orthochronous if and only if its θ element is non-positive.

```
lemma not_isOrthochronous_iff_le_zero : ¬ IsOrthochronous  $\Lambda$   $\leftrightarrow$   $\Lambda.1$  (Sum.inl 0) (Sum.inl 0)  $\leq$  0
```

lemma id_isOrthochronous[\[source\]](#)

The identity Lorentz transformation is orthochronous.

```
lemma id_isOrthochronous : @IsOrthochronous d 1
```

lemma isOrthochronous_iff_of_orthchroMap_eq[\[source\]](#)

Two Lorentz transformations are both orthochronous or both not orthochronous if they are mapped to the same element via the homomorphism from LorentzGroup to $\mathbb{Z}_2$.

```
lemma isOrthochronous_iff_of_orthchroMap_eq { $\Lambda$   $\Lambda'$  : LorentzGroup d}
(h : orthchroMap  $\Lambda$  = orthchroMap  $\Lambda'$ ) : IsOrthochronous  $\Lambda$   $\leftrightarrow$  IsOrthochronous  $\Lambda'$ 
```

lemma isOrthochronous_on_connected_component[\[source\]](#)

Two Lorentz transformations which are in the same connected component are either both orthochronous or both not orthochronous.

```
lemma isOrthochronous_on_connected_component { $\Lambda$   $\Lambda'$  : LorentzGroup d}
(h :  $\Lambda' \in$  connectedComponent  $\Lambda$ ) : IsOrthochronous  $\Lambda$   $\leftrightarrow$  IsOrthochronous  $\Lambda'$ 
```

2.31 PhysLean.Relativity.LorentzGroup.Restricted

[PhysLean/Relativity/LorentzGroup/Restricted.lean](#) on GitHub.

semiformal_result restricted_isConnected[\[source\]](#)

The restricted Lorentz group is connected.

```
semiformal_result "FXNL5" restricted_isConnected {d :  $\mathbb{N}$ } :
IsConnected (restricted d : Set (LorentzGroup d))
```

lemma restricted_eq_identity_component_of_isConnected[\[source\]](#)

Given the hypothesis that the restricted Lorentz group is connected, the proof that the restricted lorentz group is equal to the connected component of the identity.

```
lemma restricted_eq_identity_component_of_isConnected {d :  $\mathbb{N}$ }
(h1 : IsConnected (restricted d : Set (LorentzGroup d))) :
(restricted d) = connectedComponent (1 : LorentzGroup d)
```

2.32 PhysLean.Relativity.MinkowskiMatrix

[PhysLean/Relativity/MinkowskiMatrix.lean](#) on GitHub.

lemma $\eta_apply_sq_eq_one$ [\[source\]](#)

```
@[simp]
lemma  $\eta\_apply\_sq\_eq\_one$  ( $\mu : \text{Fin } 1 \oplus \text{Fin } d$ ) :
   $\eta \mu \mu^2 = 1$ 
```

lemma $\eta_diag_ne_zero$ [\[source\]](#)

```
lemma  $\eta\_diag\_ne\_zero$  ( $\mu : \text{Fin } 1 \oplus \text{Fin } d$ ) :
   $\eta \mu \mu \neq 0$ 
```

lemma $mul_ \eta_diag_eq_iff$ [\[source\]](#)

```
lemma  $mul\_ \eta\_diag\_eq\_iff$  ( $\mu : \text{Fin } 1 \oplus \text{Fin } d$ ) ( $\{x\ y : \mathbb{R}\}$ ) :
   $\eta \mu \mu * x = \eta \mu \mu * y \leftrightarrow x = y$ 
```

2.33 PhysLean.Relativity.PauliMatrices.Matrix

[PhysLean/Relativity/PauliMatrices/Matrix.lean](#) on GitHub.

lemma $\sigma 0_eq$ [\[source\]](#)

```
lemma  $\sigma 0\_eq$  :  $\sigma 0 = !![1, 0; 0, 1]$ 
```

2.34 PhysLean.Relativity.Tensors.RealTensor.Vector.Basic

[PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean](#) on GitHub.

def basis[\[source\]](#)

The basis on Vector d indexed by $\text{Fin } 1 \oplus \text{Fin } d$.

```
def basis {d :  $\mathbb{N}$ } : Basis ( $\text{Fin } 1 \oplus \text{Fin } d$ )  $\mathbb{R}$  (Vector d)
```

lemma toCoord_tensor_basis_apply[\[source\]](#)

```
lemma toCoord_tensor_basis_apply {d :  $\mathbb{N}$ } ( $\mu : \text{Fin } (1 + d)$ ) ( $v : \text{Fin } 1 \oplus \text{Fin } d$ ) :
  toCoord (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ => Fin.cast (by simp)
 $\mu$ )) v
= (Finsupp.single (finSumFinEquiv.symm  $\mu$ ) 1) v
```

lemma tensor_basis_repr_apply[\[source\]](#)

```
lemma tensor_basis_repr_apply {d :  $\mathbb{N}$ } (p : Vector d)
  (b : ComponentIdx (S := realLorentzTensor d) ![Color.up]) :
  (Tensor.basis (S := realLorentzTensor d) ![Color.up]).repr p b =
  p (finSumFinEquiv.symm (b 0))
```

lemma basis_repr_eq_toCoord[\[source\]](#)

```
lemma basis_repr_eq_toCoord {d :  $\mathbb{N}$ } :
  basis_repr = toCoord.trans (Finsupp.linearEquivFunOnFinite  $\mathbb{R} \mathbb{R}$  ( $\text{Fin } 1 \oplus \text{Fin } d$ )).symm
```

lemma basis_repr_apply_eq_toCoord[\[source\]](#)

```
lemma basis_repr_apply_eq_toCoord {d : ℕ} (p : Vector d) :
  basis_repr p = toCoord p
```

lemma toMatrix_basis_mulVec_toCoord[\[source\]](#)

```
lemma toMatrix_basis_mulVec_toCoord {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) (p : Vector d) :
  (LinearMap.toMatrix basis basis f) *v p.toCoord = (f p).toCoord
```

lemma toCoord_basis[\[source\]](#)

```
@[simp]
lemma toCoord_basis {d : ℕ} (μ v : Fin 1 ⊗ Fin d) :
  toCoord (basis μ) v = if μ = v then 1 else 0
```

lemma toCoord_map_apply[\[source\]](#)

```
lemma toCoord_map_apply {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) (p : Vector d) :
  toCoord (f p) = (LinearMap.toMatrix basis basis f) *v (toCoord p)
```

lemma spatialPart_apply_eq_toCoord[\[source\]](#)

```
lemma spatialPart_apply_eq_toCoord {d : ℕ} (v : Vector d) (i : Fin d) :
  spatialPart v i = v (Sum.inr i)
```

def minkowskiProductMap[\[source\]](#)

The Minkowski product of Lorentz vectors in the +— convention..

```
def minkowskiProductMap {d : ℕ} (p q : Vector d) : ℝ
```

lemma minkowskiProductMap_toCoord[\[source\]](#)

```
lemma minkowskiProductMap_toCoord {d : ℕ} (p q : Vector d) :
  minkowskiProductMap p q = p (Sum.inl 0) * q (Sum.inl 0) -
  ∑ i, p (Sum.inr i) * q (Sum.inr i)
```

lemma minkowskiProductMap_symm[\[source\]](#)

```
lemma minkowskiProductMap_symm {d : ℕ} (p q : Vector d) :
  minkowskiProductMap p q = minkowskiProductMap q p
```

lemma minkowskiProductMap_add_fst[\[source\]](#)

```
@[simp]
lemma minkowskiProductMap_add_fst {d : ℕ} (p q r : Vector d) :
  minkowskiProductMap (p + q) r = minkowskiProductMap p r + minkowskiProductMap q r
```

lemma minkowskiProductMap_add_snd[\[source\]](#)

@[simp]

```
lemma minkowskiProductMap_add_snd {d : ℕ} (p q r : Vector d) :
  minkowskiProductMap p (q + r) = minkowskiProductMap p q + minkowskiProductMap p r
```

lemma minkowskiProductMap_smul_fst[\[source\]](#)

@[simp]

```
lemma minkowskiProductMap_smul_fst {d : ℕ} (c : ℝ) (p q : Vector d) :
  minkowskiProductMap (c • p) q = c * minkowskiProductMap p q
```

lemma minkowskiProductMap_smul_snd[\[source\]](#)

@[simp]

```
lemma minkowskiProductMap_smul_snd {d : ℕ} (c : ℝ) (p q : Vector d) :
  minkowskiProductMap p (c • q) = c * minkowskiProductMap p q
```

def minkowskiProduct[\[source\]](#)

The Minkowski product of two Lorentz vectors as a linear map.

```
def minkowskiProduct {d : ℕ} : Vector d →l[ℝ] Vector d →l[ℝ] ℝ where
```

lemma minkowskiProduct_apply[\[source\]](#)

```
lemma minkowskiProduct_apply {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = minkowskiProductMap p q
```

lemma minkowskiProduct_symm[\[source\]](#)

```
lemma minkowskiProduct_symm {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = ⟨⟨q, p⟩⟩m
```

lemma minkowskiProduct_toCoord[\[source\]](#)

```
lemma minkowskiProduct_toCoord {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = p (Sum.inl 0) * q (Sum.inl 0) - ∑ i, p (Sum.inr i) * q (Sum.inr i)
```

lemma minkowskiProduct_toCoord_minkowskiMatrix[\[source\]](#)

```
lemma minkowskiProduct_toCoord_minkowskiMatrix {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = ∑ μ, minkowskiMatrix μ μ * (toCoord p μ) * (toCoord q μ)
```

lemma minkowskiProduct_invariant[\[source\]](#)

@[simp]

```
lemma minkowskiProduct_invariant {d : ℕ} (p q : Vector d) (Λ : LorentzGroup d) :
  ⟨⟨Λ • p, Λ • q⟩⟩m = ⟨⟨p, q⟩⟩m
```

lemma minkowskiProduct_eq_timeComponent_spatialPart[\[source\]](#)

```

lemma minkowskiProduct_eq_timeComponent_spatialPart {d : ℕ} (p q : Vector d) :
  ⟨⟨p, q⟩⟩m = p.timeComponent * q.timeComponent -
  ⟨⟨p.spatialPart, q.spatialPart⟩⟩ℝ

```

lemma minkowskiProduct_self_eq_timeComponent_spatialPart[\[source\]](#)

```

lemma minkowskiProduct_self_eq_timeComponent_spatialPart {d : ℕ} (p : Vector d) :
  ⟨⟨p, p⟩⟩m = ||p.timeComponent|| ^ 2 - ||p.spatialPart|| ^ 2

```

lemma minkowskiProduct_basis_left[\[source\]](#)

```

@[simp]
lemma minkowskiProduct_basis_left {d : ℕ} (μ : Fin 1 ⊕ Fin d) (p : Vector d) :
  ⟨⟨basis μ, p⟩⟩m = minkowskiMatrix μ μ * toCoord p μ

```

lemma minkowskiProduct_basis_right[\[source\]](#)

```

@[simp]
lemma minkowskiProduct_basis_right {d : ℕ} (μ : Fin 1 ⊕ Fin d) (p : Vector d) :
  ⟨⟨p, basis μ⟩⟩m = minkowskiMatrix μ μ * toCoord p μ

```

lemma minkowskiProduct_eq_zero_forall_iff[\[source\]](#)

```

@[simp]
lemma minkowskiProduct_eq_zero_forall_iff {d : ℕ} (p : Vector d) :
  (∀ q : Vector d, ⟨⟨p, q⟩⟩m = 0) ↔ p = 0

```

lemma map_minkowskiProduct_eq_self_forall_iff[\[source\]](#)

```

lemma map_minkowskiProduct_eq_self_forall_iff {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) :
  (∀ p q : Vector d, ⟨⟨f p, q⟩⟩m = ⟨⟨p, q⟩⟩m) ↔ f = LinearMap.id

```

def adjoint[\[source\]](#)

The adjoint of a linear map from Vector d to itself with respect to the minkowskiProduct.

```

def adjoint {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) : Vector d →ℓ[ℝ] Vector d

```

lemma map_minkowskiProduct_eq_adjoint[\[source\]](#)

```

lemma map_minkowskiProduct_eq_adjoint {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) (p q : Vector d) :
  ⟨⟨f p, q⟩⟩m = ⟨⟨p, adjoint f q⟩⟩m

```

lemma minkowskiProduct_map_eq_adjoint[\[source\]](#)

```

lemma minkowskiProduct_map_eq_adjoint {d : ℕ} (f : Vector d →ℓ[ℝ] Vector d) (p q : Vector d) :
  ⟨⟨p, f q⟩⟩m = ⟨⟨adjoint f p, q⟩⟩m

```

def IsLorentz[\[source\]](#)

A linear map $\text{Vector } d \rightarrow_l[\mathbb{R}] \text{ Vector } d$ satisfies `IsLorentz` if it preserves the minkowski product.

```
def IsLorentz {d : ℕ} (f : Vector d →l[ℝ] Vector d) :
  Prop
```

lemma isLorentz_iff[\[source\]](#)

```
lemma isLorentz_iff {d : ℕ} (f : Vector d →l[ℝ] Vector d) :
  IsLorentz f ↔ ∀ p q : Vector d, ⟨⟨f p, f q⟩⟩m = ⟨⟨p, q⟩⟩m
```

lemma isLorentz_iff_basis[\[source\]](#)

```
lemma isLorentz_iff_basis {d : ℕ} (f : Vector d →l[ℝ] Vector d) :
  IsLorentz f ↔ ∀ μ ν : Fin 1 ⊕ Fin d, ⟨⟨f (basis μ), f (basis ν)⟩⟩m = ⟨⟨basis μ, basis ν⟩⟩m
```

lemma isLorentz_iff_comp_adjoint_eq_id[\[source\]](#)

```
lemma isLorentz_iff_comp_adjoint_eq_id {d : ℕ} (f : Vector d →l[ℝ] Vector d) :
  IsLorentz f ↔ adjoint f ∘l f = LinearMap.id
```

lemma isLorentz_iff_toMatrix_mem_lorentzGroup[\[source\]](#)

```
lemma isLorentz_iff_toMatrix_mem_lorentzGroup {d : ℕ} (f : Vector d →l[ℝ] Vector d) :
  IsLorentz f ↔ LinearMap.toMatrix Vector.basis Vector.basis f ∈ LorentzGroup d
```

2.35 PhysLean.Relativity.Tensors.RealTensor.Velocity.Basic

[PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean](#) on GitHub.

def Velocity[\[source\]](#)

A Lorentz Velocity is a Lorentz vector which has norm equal to one and which is future-directed.

```
def Velocity (d : ℕ := 3) : Set (Vector d)
```

instance TopologicalSpace (Velocity d)[\[source\]](#)

The instance of a topological space on `Velocity d` defined as the subspace topology.

```
instance : TopologicalSpace (Velocity d)
```

lemma ext[\[source\]](#)

```
@[ext]
lemma ext {v w : Velocity d} (h : v.1 = w.1) : v = w
```

lemma mem_iff[\[source\]](#)

```
lemma mem_iff {v : Vector d} : v ∈ Velocity d ↔  $\langle\langle v, v \rangle\rangle_m = (1 : \mathbb{R}) \wedge 0 < v.timeComponent$ 
```

lemma minkowskiProduct_self_eq_one[\[source\]](#)

```
@[simp]
```

```
lemma minkowskiProduct_self_eq_one (v : Velocity d) :  $\langle\langle v.1, v.1 \rangle\rangle_m = (1 : \mathbb{R})$ 
```

lemma timeComponent_pos[\[source\]](#)

```
@[simp]
```

```
lemma timeComponent_pos (v : Velocity d) :  $0 < v.1.timeComponent$ 
```

lemma timeComponent_nonneg[\[source\]](#)

```
@[simp]
```

```
lemma timeComponent_nonneg (v : Velocity d) :  $0 \leq v.1.timeComponent$ 
```

lemma timeComponent_abs[\[source\]](#)

```
@[simp]
```

```
lemma timeComponent_abs (v : Velocity d) :  
   $|v.1.timeComponent| = v.1.timeComponent$ 
```

lemma norm_spatialPart_le_timeComponent[\[source\]](#)

```
lemma norm_spatialPart_le_timeComponent (v : Velocity d) :  
   $\|v.1.spatialPart\| \leq \|v.1.timeComponent\|$ 
```

lemma norm_spatialPart_sq_eq[\[source\]](#)

```
lemma norm_spatialPart_sq_eq (v : Velocity d) :  
   $\|v.1.spatialPart\|^2 = (toCoord v.1 (Sum.inl 0))^2 - 1$ 
```

lemma zero_le_minkowskiProduct[\[source\]](#)

```
lemma zero_le_minkowskiProduct (u v : Velocity d) :  
   $0 \leq \langle\langle u.1, v.1 \rangle\rangle_m$ 
```

lemma one_add_minkowskiProduct_neq_zero[\[source\]](#)

```
lemma one_add_minkowskiProduct_neq_zero (u v : Velocity d) :  
   $1 + \langle\langle u.1, v.1 \rangle\rangle_m \neq 0$ 
```

lemma minkowskiProduct_continuous_snd[\[source\]](#)

```
lemma minkowskiProduct_continuous_snd (u : Vector d) :  
  Continuous fun (x : Velocity d) =>  $\langle\langle u, x.1 \rangle\rangle_m$ 
```

lemma minkowskiProduct_continuous_fst[\[source\]](#)

```
@[fun_prop]
lemma minkowskiProduct_continuous_fst (u : Vector d) :
  Continuous fun (x : Velocity d) => ⟨⟨x.1, u⟩⟩m
```

def zero[\[source\]](#)

The Velocity d which has all space components zero.

```
noncomputable def zero : Velocity d
```

instance Zero (Velocity d)[\[source\]](#)

```
noncomputable instance : Zero (Velocity d)
```

def pathFromZero[\[source\]](#)

A continuous path from a velocity u to the zero velocity.

```
noncomputable def pathFromZero (u : Velocity d) : Path zero u where
```

lemma isPathConnected[\[source\]](#)

```
lemma isPathConnected : IsPathConnected (@Set.univ (Velocity d))
```

2.36 PhysLean.SpaceAndTime.Space.Basic

[PhysLean/SpaceAndTime/Space/Basic.lean](#) on GitHub.

structure Direction[\[source\]](#)

Notion of direction where unit returns a unit vector in the direction specified.

```
structure Direction (d : ℕ := 3) where
  unit : EuclideanSpace ℝ (Fin d)
  norm : ||unit|| = 1
```

def toDirection[\[source\]](#)

Direction of a Space value with respect to the origin.

```
noncomputable def toDirection {d : ℕ} (x : Space d) (h : x ≠ 0) : Direction d where
```

2.37 PhysLean.SpaceAndTime.Space.VectorIdentities

[PhysLean/SpaceAndTime/Space/VectorIdentities.lean](#) on GitHub.

lemma grad_eq_sum[\[source\]](#)

```
lemma grad_eq_sum {d} (f : Space d → ℝ) (x : Space d) :
  ∇ f x = ∑ i, deriv i f x • basis i
```

lemma grad_eq_gradiant[\[source\]](#)

```
lemma grad_eq_gradiant {d} (f : Space d → ℝ) :
  ∀ f = gradient f
```

2.38 PhysLean.StatisticalMechanics.BoltzmannConstant

[PhysLean/StatisticalMechanics/BoltzmannConstant.lean](#) on GitHub.

axiom kB[\[source\]](#)

The Boltzmann constant.

```
axiom kB : ℝ ≥ 0
```

lemma kB_nonneg[\[source\]](#)

The Boltzmann constant is non-negative.

```
lemma kB_nonneg : 0 ≤ kB
```

2.39 PhysLean.StatisticalMechanics.CanonicalEnsemble.Basic

[PhysLean/StatisticalMechanics/CanonicalEnsemble/Basic.lean](#) on GitHub.

def CanonicalEnsemble[\[source\]](#)

A Canonical ensemble is described by a type ι , corresponding to the type of microstates, and a map $\iota \rightarrow \mathbb{R}$ which associates which each microstate an energy.

```
def CanonicalEnsemble (ι : Type) : Type
```

instance HAdd (CanonicalEnsemble ι1) (CanonicalEnsemble ι2) (CanonicalEnsemble (... [\[source\]](#)

The addition of two CanonicalEnsemble.

```
instance {ι1 ι2 : Type} : HAdd (CanonicalEnsemble ι1) (CanonicalEnsemble ι2)
  (CanonicalEnsemble (ι1 × ι2)) where
```

def nsmul[\[source\]](#)

Scalar multiplication of CanonicalEnsemble, defined such that nsmul n C is n coppies of the canonical ensemble C.

```
def nsmul (n : ℕ) (C1 : CanonicalEnsemble ι) : CanonicalEnsemble (Fin n → ι)
```

abbrev microstates[\[source\]](#)

The microstates of a the canonical ensemble

```
@[nolint unusedArguments]
abbrev microstates {ι : Type} (C : CanonicalEnsemble ι) : Type
```

abbrev energy[\[source\]](#)

The energy of associated with a microstate of the canonical ensemble.

```
abbrev energy (C : CanonicalEnsemble ι) : microstates C → ℝ
```

lemma energy_add_apply[\[source\]](#)

```
@[simp]
lemma energy_add_apply (i : microstates (C + C1)) :
  (C + C1).energy i = C.energy i.1 + C1.energy i.2
```

lemma energy_nsmul_apply[\[source\]](#)

```
@[simp]
lemma energy_nsmul_apply (n : ℕ) (f : Fin n → microstates C) :
  (nsmul n C).energy f = ∑ i, C.energy (f i)
```

def partitionFunction[\[source\]](#)

The partition function of the canonical ensemble.

```
noncomputable def partitionFunction [Fintype ι] (T : Temperature) : ℝ
```

lemma partitionFunction_add[\[source\]](#)

```
lemma partitionFunction_add [Fintype ι] [Fintype ι1] :
  (C + C1).partitionFunction T = C.partitionFunction T * C1.partitionFunction T
```

lemma partitionFunction_nsmul[\[source\]](#)

The partition function of n copies of a canonical ensemble.

```
@[sorryful]
lemma partitionFunction_nsmul [Fintype ι] (n : ℕ) (T : Temperature) :
  (nsmul n C).partitionFunction T = (C.partitionFunction T) ^ n
```

lemma partitionFunction_pos[\[source\]](#)

```
lemma partitionFunction_pos [Fintype ι] [Nonempty ι] (T : Temperature) :
  0 < partitionFunction C T
```

lemma partitionFunction_neq_zero[\[source\]](#)

```
@[simp]
lemma partitionFunction_neq_zero [Fintype ι] [Nonempty ι] (T : Temperature) :
  partitionFunction C T ≠ 0
```

def partitionFunctionβ[\[source\]](#)

The partition function of the canonical ensemble as a function of β

```
noncomputable def partitionFunctionβ [Fintype ι] (β : ℝ) : ℝ
```

lemma partitionFunction β _def[\[source\]](#)

```
lemma partitionFunction $\beta$ _def [Fintype  $\mathfrak{I}$ ]:
  partitionFunction $\beta$   $\mathcal{C}$  = fun  $\beta \Rightarrow \sum i, \exp (- \beta * \mathcal{C}.\text{energy } i)$ 
```

lemma partitionFunction β _differentiable[\[source\]](#)

```
@[fun_prop]
lemma partitionFunction $\beta$ _differentiable [Fintype  $\mathfrak{I}$ ] :
  Differentiable  $\mathbb{R}$   $\mathcal{C}.\text{partitionFunction}\beta$ 
```

lemma partitionFunction_eq_partitionFunction β [\[source\]](#)

```
lemma partitionFunction_eq_partitionFunction $\beta$  [Fintype  $\mathfrak{I}$ ] (T : Temperature) :
  partitionFunction  $\mathcal{C}$  T = partitionFunction $\beta$   $\mathcal{C}$  ( $\beta$  T)
```

def probability[\[source\]](#)

The probability of been in a given microstate.

```
noncomputable def probability [Fintype  $\mathfrak{I}$ ] (i : microstates  $\mathcal{C}$ ) (T : Temperature) :  $\mathbb{R}$ 
```

lemma probability_le_one[\[source\]](#)

Probability of a microstate in a canonical ensemble is less then or equal to 1.

```
@[sorryful]
lemma probability_le_one [Fintype  $\mathfrak{I}$ ] (i : microstates  $\mathcal{C}$ ) (T : Temperature) :
   $\mathcal{C}.\text{probability } i$  T  $\leq 1$ 
```

lemma probability_nonneg[\[source\]](#)

Probability of a microstate in a canonical ensemble is non-negative.

```
@[sorryful]
lemma probability_nonneg [Fintype  $\mathfrak{I}$ ] (i : microstates  $\mathcal{C}$ ) (T : Temperature) :
   $0 \leq \mathcal{C}.\text{probability } i$  T
```

lemma probability_neq_zero[\[source\]](#)

```
lemma probability_neq_zero [Fintype  $\mathfrak{I}$ ] [Nonempty  $\mathfrak{I}$ ] (i : microstates  $\mathcal{C}$ ) (T : Temperature) :
  probability  $\mathcal{C}$  i T  $\neq 0$ 
```

lemma probability_add[\[source\]](#)

```
@[simp]
lemma probability_add [Fintype  $\mathfrak{I}$ ] [Fintype  $\mathfrak{I}1$ ]
  (i : microstates ( $\mathcal{C} + \mathcal{C}1$ )) (T : Temperature) :
  ( $\mathcal{C} + \mathcal{C}1$ ).probability i T =  $\mathcal{C}.\text{probability } i.1$  T *  $\mathcal{C}1.\text{probability } i.2$  T
```

lemma probability_nsmul[\[source\]](#)

The probability of a microstate in n copies of a canonical ensemble is equal to the product of the probability of the corresponding individual microstates.

```
@[sorryful]
lemma probability_nsmul [Fintype ι] (n : ℕ)
  (f : microstates (nsmul n C)) (T : Temperature) :
  (nsmul n C).probability f T = ∏ i, C.probability (f i) T
```

lemma sum_probability_eq_one[\[source\]](#)

```
@[simp]
lemma sum_probability_eq_one [Fintype ι] [Nonempty ι] (T : Temperature) :
  ∑ i, probability C i T = 1
```

def meanEnergy[\[source\]](#)

The mean energy of the canonical ensemble.

```
noncomputable def meanEnergy [Fintype ι] (T : Temperature) : ℝ
```

lemma meanEnergy_add[\[source\]](#)

```
@[simp]
lemma meanEnergy_add [Fintype ι] [Nonempty ι] (C1 : CanonicalEnsemble ι) [Fintype ι1]
  [Nonempty ι1]
  (T : Temperature) :
  (C + C1).meanEnergy T = C.meanEnergy T + C1.meanEnergy T
```

lemma meanEnergy_nsmul[\[source\]](#)

The mean energy of n copies of a canonical ensemble is equal to n times the mean energy of the canonical ensemble.

Note, can't make this $SMul$ since the target type depends on the value of n .

```
@[sorryful]
lemma meanEnergy_nsmul [Fintype ι] (n : ℕ) (T : Temperature) :
  (nsmul n C).meanEnergy T = n * C.meanEnergy T
```

lemma meanEnergy_eq_logDeriv_partitionFunctionβ[\[source\]](#)

```
lemma meanEnergy_eq_logDeriv_partitionFunctionβ [Fintype ι] (T : Temperature) :
  meanEnergy C T = - logDeriv (partitionFunctionβ C) (β T)
```

def entropy[\[source\]](#)

The entropy of the canonical ensemble.

```
noncomputable def entropy [Fintype ι] (T : Temperature) : ℝ
```

lemma entropy_add[\[source\]](#)

Entropy is additive on adding canonical ensembles.

```
@[simp]
lemma entropy_add [Fintype ι] [Nonempty ι] (C1 : CanonicalEnsemble ι) [Fintype ι1]
  [Nonempty ι1] (T : Temperature) :
  (C + C1).entropy T = C.entropy T + C1.entropy T
```

lemma entropy_nsmul[\[source\]](#)

The entropy of n copies of a canonical ensemble is equal to n times the entropy of the canonical ensemble.

```
@[sorryful]
lemma entropy_nsmul [Fintype ι] (n : ℕ) (T : Temperature) :
  (nsmul n C).entropy T = n * C.entropy T
```

def helmholtzFreeEnergy[\[source\]](#)

The (Helmholtz) free energy of the canonical ensemble.

```
noncomputable def helmholtzFreeEnergy [Fintype ι] (T : Temperature) : ℝ
```

lemma helmholtzFreeEnergy_add[\[source\]](#)

The Helmholtz free energy is additive.

```
@[simp]
lemma helmholtzFreeEnergy_add [Fintype ι] [Nonempty ι]
  [Fintype ι1] [Nonempty ι1] (T : Temperature) :
  (C + C1).helmholtzFreeEnergy T = C.helmholtzFreeEnergy T + C1.helmholtzFreeEnergy T
```

lemma helmholtzFreeEnergy_nsmul[\[source\]](#)

The free energy of n copies of a canonical ensemble is equal to n times the entropy of the canonical ensemble.

```
@[sorryful]
lemma helmholtzFreeEnergy_nsmul [Fintype ι] (n : ℕ) (T : Temperature) :
  (nsmul n C).helmholtzFreeEnergy T = n * C.helmholtzFreeEnergy T
```

2.40 PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState

[PhysLean/StatisticalMechanics/CanonicalEnsemble/TwoState.lean](#) on GitHub.

def twoState[\[source\]](#)

The canonical ensemble corresponding to state system, with one state of energy E and the other state of energy 0 .

```
def twoState (E : ℝ) : CanonicalEnsemble (Fin 2)
```

lemma twoState_partitionFunction_apply_eq_one_add_exp[\[source\]](#)

```
lemma twoState_partitionFunction_apply_eq_one_add_exp (E : ℝ) (T : Temperature) :
  (twoState E).partitionFunction T = 1 + exp (- β T * E)
```

lemma twoState_partitionFunction_apply_eq_cosh[\[source\]](#)

```
lemma twoState_partitionFunction_apply_eq_cosh (E : ℝ) (T : Temperature) :
  (twoState E).partitionFunction T = 2 * exp (- β T * E / 2) * cosh (β T * E / 2)
```

lemma twoState_partitionFunctionβ_eq[\[source\]](#)

```
lemma twoState_partitionFunctionβ_eq (E : ℝ) :
  (twoState E).partitionFunctionβ = fun β => 2 * exp (- β * E / 2) * cosh (β * E / 2)
```

lemma twoState_meanEnergy_eq[\[source\]](#)

```
lemma twoState_meanEnergy_eq (E : ℝ) (T : Temperature) :
  (twoState E).meanEnergy T = E / 2 * (1 - tanh (β T * E / 2))
```

informal_lemma twoState_entropy_eq[\[source\]](#)

A simplification of the entropy of the two-state canonical ensemble.

```
informal_lemma twoState_entropy_eq where
  tag := "EVJJI"
  deps := [``twoState, ``entropy]
```

informal_lemma twoState_helmholtzFreeEnergy_eq[\[source\]](#)

A simplification of the helmholtzFreeEnergy of the two-state canonical ensemble.

```
informal_lemma twoState_helmholtzFreeEnergy_eq where
  tag := "EVMPR"
  deps := [``twoState, ``helmholtzFreeEnergy]
```

2.41 PhysLean.StatisticalMechanics.Temperature

[PhysLean/StatisticalMechanics/Temperature.lean](#) on GitHub.

def Temperature[\[source\]](#)

The type of temperatures.

```
def Temperature : Type
```

instance LinearOrder Temperature[\[source\]](#)

```
noncomputable instance : LinearOrder Temperature
```

def toReal [source]

The underlying real-number associated with the tempature.

```
noncomputable def toReal (T : Temperature) : ℝ
```

instance Coe Temperature ℝ [source]

```
noncomputable instance : Coe Temperature ℝ
```

def β [source]

The inverse temperature.

```
noncomputable def  $\beta$  (T : Temperature) : ℝ
```

2.42 PhysLean.StringTheory.FTheory.SU5.Fluxes.Basic

[PhysLean/StringTheory/FTheory/SU5/Fluxes/Basic.lean](#) on GitHub.

abbrev FluxesFive [source]

The fluxes (M, N) of the 5-bar matter curves of a theory.

```
abbrev FluxesFive : Type
```

instance DecidableEq FluxesFive [source]

```
instance : DecidableEq FluxesFive
```

abbrev FluxesFive.HasNoZero [source]

The proposition on FluxesFive such that $(0, 0)$ is not in F and as such each component in F leads to chiral matter.

```
abbrev HasNoZero (F : FluxesFive) : Prop
```

def chiralIndicesOfD [source]

The multiset of chiral indices of the representation $D = (\bar{3}, 1)_{\{1/3\}}$ arising from the matter 5d representations.

```
def chiralIndicesOfD (F : FluxesFive) : Multiset ℤ
```

def numChiralD [source]

The total number of chiral D representations arising from the matter 5d representations.

```
def numChiralD (F : FluxesFive) : ℤ
```

def numAntiChiralD[\[source\]](#)

The total number of anti-chiral D representations arising from the matter 5d representations.

```
def numAntiChiralD (F : FluxesFive) : ℤ
```

def chiralIndicesOfL[\[source\]](#)

The multiset of chiral indices of the representation $L = (1,2)_{\{-1/2\}}$ arising from the matter 5d representations.

```
def chiralIndicesOfL (F : FluxesFive) : Multiset ℤ
```

def numChiralL[\[source\]](#)

The total number of chiral L representations arising from the matter 5d representations.

```
def numChiralL (F : FluxesFive) : ℤ
```

def numAntiChiralL[\[source\]](#)

The total number of anti-chiral L representations arising from the matter 5d representations.

```
def numAntiChiralL (F : FluxesFive) : ℤ
```

def FluxesFive.NoExotics[\[source\]](#)

The condition that the 5d-matter representations do not lead to exotic chiral matter in the MSSM spectrum. This corresponds to the conditions that: - There are 3 chiral L representations and no anti-chiral L representations. - There are 3 chiral D representations and no anti-chiral D representations.

```
def NoExotics (F : FluxesFive) : Prop
```

instance Decidable (F.NoExotics)[\[source\]](#)

```
instance (F : FluxesFive) : Decidable (F.NoExotics)
```

abbrev FluxesTen[\[source\]](#)

The fluxes (M, N) of the 10d matter curves of a theory.

```
abbrev FluxesTen : Type
```

abbrev FluxesTen.HasNoZero[\[source\]](#)

The proposition on FluxesTen such that $(0, 0)$ is not in F and as such each component in F leads to chiral matter.

```
abbrev HasNoZero (F : FluxesTen) : Prop
```

def chiralIndicesOfQ[\[source\]](#)

The multiset of chiral indices of the representation $Q = (3,2)_{\{1/6\}}$ arising from the matter 10d representations, corresponding to M .

```
def chiralIndicesOfQ (F : FluxesTen) : Multiset ℤ
```

def numChiralQ[\[source\]](#)

The total number of chiral Q representations arising from the matter 10d representations.

```
def numChiralQ (F : FluxesTen) : ℤ
```

def numAntiChiralQ[\[source\]](#)

The total number of anti-chiral Q representations arising from the matter 10d representations.

```
def numAntiChiralQ (F : FluxesTen) : ℤ
```

def chiralIndicesOfU[\[source\]](#)

The multiset of chiral indices of the representation $U = (\bar{3},1)_{\{-2/3\}}$ arising from the matter 10d representations, corresponding to $M - N$

```
def chiralIndicesOfU (F : FluxesTen) : Multiset ℤ
```

def numChiralU[\[source\]](#)

The total number of chiral U representations arising from the matter 10d representations.

```
def numChiralU (F : FluxesTen) : ℤ
```

def numAntiChiralU[\[source\]](#)

The total number of anti-chiral U representations arising from the matter 10d representations.

```
def numAntiChiralU (F : FluxesTen) : ℤ
```

def chiralIndicesOfE[\[source\]](#)

The multiset of chiral indices of the representation $E = (1,1)_{\{1\}}$ arising from the matter 10d representations, corresponding to $M + N$

```
def chiralIndicesOfE (F : FluxesTen) : Multiset ℤ
```

def numChiralE[\[source\]](#)

The total number of chiral E representations arising from the matter 10d representations.

```
def numChiralE (F : FluxesTen) : ℤ
```

```
def numAntiChiralE
```

[\[source\]](#)

The total number of anti-chiral E representations arising from the matter 10d representations.

```
def numAntiChiralE (F : FluxesTen) : ℤ
```

```
def FluxesTen.NoExotics
```

[\[source\]](#)

The condition that the 10d-matter representations do not lead to exotic chiral matter in the MSSM spectrum. This corresponds to the conditions that: - There are 3 chiral Q representations and no anti-chiral Q representations. - There are 3 chiral U representations and no anti-chiral U representations. - There are 3 chiral E representations and no anti-chiral E representations.

```
def NoExotics (F : FluxesTen) : Prop
```

```
instance Decidable (F.NoExotics)
```

[\[source\]](#)

```
instance (F : FluxesTen) : Decidable (F.NoExotics)
```

2.43 PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ChiralIndices

[PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/ChiralIndices.lean](#) on GitHub.

```
lemma chiralIndicesOfD_noneg_of_noExotics
```

[\[source\]](#)

The chiral indices of the representations $D = (\bar{3}, 1)_{1/3}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfD_noneg_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) : 0 ≤ ci
```

```
lemma chiralIndicesOfD_sum_eq_three_of_noExotics
```

[\[source\]](#)

The sum of the chiral indices of the representations $D = (\bar{3}, 1)_{1/3}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfD_sum_eq_three_of_noExotics (F : FluxesFive) (hF : NoExotics F) :
  F.chiralIndicesOfD.sum = 3
```

```
lemma chiralIndicesOfD_le_three_of_noExotics
```

[\[source\]](#)

The chiral indices of the representation $D = (\bar{3}, 1)_{1/3}$ are less then or equal to 3.

```
lemma chiralIndicesOfD_le_three_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) : ci ≤ 3
```

lemma mem_chiralIndicesOfD_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfD_mem_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfD) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfD_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndicesOfD_subset_sum_le_three_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F) : (S.map (fun x => x.1)).sum ≤ 3
```

lemma chiralIndicesOfL_noneg_of_noExotics[\[source\]](#)

The chiral indices of the representations $L = (1,2)_{-1/2}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfL_noneg_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) : 0 ≤ ci
```

lemma chiralIndicesOfL_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $L = (1,2)_{-1/2}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfL_sum_eq_three_of_noExotics (F : FluxesFive) (hF : NoExotics F) :
  F.chiralIndicesOfL.sum = 3
```

lemma chiralIndicesOfL_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $L = (1,2)_{-1/2}$ are less then or equal to 3.

```
lemma chiralIndicesOfL_le_three_of_noExotics (F : FluxesFive) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) : ci ≤ 3
```

lemma mem_chiralIndicesOfL_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfL_mem_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfL) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfL_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndicesOfL_subset_sum_le_three_of_noExotics (F : FluxesFive)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F) : (S.map (fun x => (x.1 + x.2))).sum ≤ 3
```

lemma chiralIndicesOfQ_noneg_of_noExotics[\[source\]](#)

The chiral indices of the representations $Q = (3,2)_{1/6}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfQ_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) : 0 ≤ ci
```

lemma chiralIndicesOfQ_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $Q = (3,2)_{\{1/6\}}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfQ_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndicesOfQ.sum = 3
```

lemma chiralIndicesOfQ_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $Q = (3,2)_{\{1/6\}}$ are less then or equal to 3.

```
lemma chiralIndicesOfQ_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) : ci ≤ 3
```

lemma mem_chiralIndicesOfQ_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndicesOfQ_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfQ) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndicesOfQ_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndicesOfQ_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F) : (S.map (fun x => x.1)).sum ≤ 3
```

lemma chiralIndicesOfU_noneg_of_noExotics[\[source\]](#)

The chiral indices of the representations $U = (\bar{3},1)_{\{-2/3\}}$ are all non-negative if there are no chiral exotics in the spectrum.

```
lemma chiralIndicesOfU_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfU) : 0 ≤ ci
```

lemma chiralIndicesOfU_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $U = (\bar{3},1)_{\{-2/3\}}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndicesOfU_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndicesOfU.sum = 3
```

lemma chiralIndicesOfU_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $U = (\bar{3},1)_{\{-2/3\}}$ are less then or equal to 3.

```
lemma chiralIndicesOfU_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndicesOfU) : ci ≤ 3
```

lemma mem_chiralIndices0fU_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndices0fU_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndices0fU) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndices0fU_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndices0fU_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F) : (S.map (fun x => (x.1 - x.2))).sum ≤ 3
```

lemma chiralIndices0fE_noneg_of_noExotics[\[source\]](#)

```
lemma chiralIndices0fE_noneg_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndices0fE) : 0 ≤ ci
```

lemma chiralIndices0fE_sum_eq_three_of_noExotics[\[source\]](#)

The sum of the chiral indices of the representations $E = (1,1)_{\{1\}}$ is equal to 3 in the presences of no exotics.

```
lemma chiralIndices0fE_sum_eq_three_of_noExotics (F : FluxesTen) (hF : NoExotics F) :
  F.chiralIndices0fE.sum = 3
```

lemma chiralIndices0fE_le_three_of_noExotics[\[source\]](#)

The chiral indices of the representation $E = (1,1)_{\{1\}}$ are less then or equal to 3.

```
lemma chiralIndices0fE_le_three_of_noExotics (F : FluxesTen) (hF : NoExotics F)
  (ci : ℤ) (hci : ci ∈ F.chiralIndices0fE) : ci ≤ 3
```

lemma mem_chiralIndices0fE_mem_of_noExotics[\[source\]](#)

```
lemma mem_chiralIndices0fE_mem_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (ci : ℤ) (hci : ci ∈ F.chiralIndices0fE) :
  ci ∈ ({0, 1, 2, 3} : Finset ℤ)
```

lemma chiralIndices0fE_subset_sum_le_three_of_noExotics[\[source\]](#)

```
lemma chiralIndices0fE_subset_sum_le_three_of_noExotics (F : FluxesTen)
  (hF : NoExotics F) (S : Multiset (ℤ × ℤ))
  (hSle : S ≤ F) : (S.map (fun x => (x.1 + x.2))).sum ≤ 3
```

2.44 PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Completeness

[PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Completeness.lean](#) on GitHub.

lemma FluxesFive.mem_mem_finset_of_noExotics[\[source\]](#)

```
lemma mem_mem_finset_of_noExotics (F : FluxesFive) (hF : F.NoExotics)
  (hnZ : F.HasNoZero)
  (x : ℤ × ℤ) (hx : x ∈ F) :
```

```
x ∈ ({(0, 1), (0, 2), (0, 3), (1, -1), (1, 0), (1, 1), (1, 2),
      (2, -2), (2, -1), (2, 0), (2, 1), (3, -3), (3, -2), (3, -1), (3, 0)} : Finset (ℤ × ℤ))
```

lemma FluxesFive.subset_le_mem_of_card_eq_one[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_one {F : FluxesFive} (hF : F.NoExotics)
  (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 1) (hS : S ≤ F) :
  S ∈ ({{(0, 1)}, {(0, 2)}, {(0, 3)}, {(1, -1)}, {(1, 0)}, {(1, 1)}, {(1, 2)},
        {(2, -2)}, {(2, -1)}, {(2, 0)}, {(2, 1)}, {(3, -3)}, {(3, -2)}, {(3, -1)}, {(3, 0)}} :
        Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesFive.subset_le_mem_of_card_eq_two[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_two {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 2) (hS : S ≤ F) :
  S ∈ ({{(0, 1), (0, 1)}, {(0, 1), (1, -1)}, {(0, 1), (1, 0)}, {(0, 1), (0, 2)},
        {(0, 1), (1, 1)}, {(0, 1), (2, -2)}, {(0, 1), (2, -1)}, {(0, 1), (2, 0)},
        {(0, 1), (3, -3)}, {(0, 1), (3, -2)}, {(0, 1), (3, -1)},
        {(0, 2), (1, -1)}, {(0, 2), (1, 0)}, {(0, 2), (2, -2)}, {(0, 2), (2, -1)},
        {(0, 2), (3, -3)}, {(0, 2), (3, -2)}, {(0, 3), (1, -1)}, {(0, 3), (2, -2)},
        {(0, 3), (3, -3)}, {(1, -1), (1, -1)}, {(1, -1), (1, 0)}, {(1, -1), (1, 1)},
        {(1, -1), (1, 2)}, {(1, -1), (2, -2)}, {(1, -1), (2, -1)}, {(1, -1), (2, 0)},
        {(1, 0), (1, 0)}, {(1, -1), (2, 1)}, {(1, 0), (1, 1)}, {(1, 0), (2, -2)},
        {(1, 0), (2, -1)}, {(1, 0), (2, 0)}, {(1, 1), (2, -2)}, {(1, 1), (2, -1)},
        {(1, 2), (2, -2)}} : Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesFive.subset_le_mem_of_card_eq_three[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_three {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 3) (hS : S ≤ F) :
  S ∈ ({{(0, 1), (0, 1), (0, 1)}, {(0, 1), (0, 1), (1, -1)},
        {(0, 1), (0, 1), (1, 0)}, {(0, 1), (0, 1), (2, -2)},
        {(0, 1), (0, 1), (2, -1)}, {(0, 1), (0, 1), (3, -3)}, {(0, 1), (0, 1), (3, -2)},
        {(0, 1), (0, 2), (1, -1)}, {(0, 1), (0, 2), (2, -2)}, {(0, 1), (0, 2), (3, -3)},
        {(0, 1), (1, -1), (1, -1)}, {(0, 1), (1, -1), (1, 0)}, {(0, 1), (1, -1), (1, 1)},
        {(0, 1), (1, -1), (2, -2)}, {(0, 1), (1, -1), (2, -1)}, {(0, 1), (1, -1), (2, 0)},
        {(0, 1), (1, 0), (1, 0)}, {(0, 1), (1, 0), (2, -2)}, {(0, 1), (1, 0), (2, -1)},
        {(0, 1), (1, 1), (2, -2)}, {(0, 2), (1, -1), (1, -1)}, {(0, 2), (1, -1), (1, 0)},
        {(0, 2), (1, -1), (2, -2)}, {(0, 2), (1, -1), (2, -1)}, {(0, 2), (1, 0), (2, -2)},
        {(0, 3), (1, -1), (1, -1)}, {(0, 3), (1, -1), (2, -2)}, {(1, -1), (1, -1), (1, -1)},
        {(1, -1), (1, -1), (1, 0)}, {(1, -1), (1, -1), (1, 1)}, {(1, -1), (1, -1), (1, 2)},
        {(1, -1), (1, 0), (1, 0)}, {(1, -1), (1, 0), (1, 1)}, {(1, 0), (1, 0), (1, 0)}} :
        Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesFive.subset_le_mem_of_card_eq_four[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_four {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 4) (hS : S ≤ F) :
  S ∈ ({{(0, 1), (0, 1), (0, 1), (1, -1)},
        {(0, 1), (0, 1), (0, 1), (2, -2)}, {(0, 1), (0, 1), (0, 1), (3, -3)},
        {(0, 1), (0, 1), (1, -1), (1, -1)}, {(0, 1), (0, 1), (1, -1), (1, 0)},
        {(0, 1), (0, 1), (1, -1), (2, -2)}, {(0, 1), (0, 1), (1, -1), (2, -1)},
        {(0, 1), (0, 1), (1, 0), (2, -2)}, {(0, 1), (0, 2), (1, -1), (1, -1)},
        {(0, 1), (0, 2), (1, -1), (2, -2)}, {(0, 1), (1, -1), (1, -1), (1, -1)},
```

```

{(0, 1), (1, -1), (1, -1), (1, 0)}, {(0, 1), (1, -1), (1, -1), (1, 1)},
{(0, 1), (1, -1), (1, 0), (1, 0)}, {(0, 2), (1, -1), (1, -1), (1, -1)},
{(0, 2), (1, -1), (1, -1), (1, 0)}, {(0, 3), (1, -1), (1, -1), (1, -1)}} :
Finset (Multiset (ℤ × ℤ))

```

lemma subset_le_mem_of_card_eq_five[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_five {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 5) (hS : S ≤ F) :
  S ∈ ({(0, 1), (0, 1), (0, 1), (1, -1), (1, -1)},
    {(0, 1), (0, 1), (0, 1), (1, -1), (2, -2)},
    {(0, 1), (0, 1), (1, -1), (1, -1), (1, -1)},
    {(0, 1), (0, 1), (1, -1), (1, -1), (1, 0)},
    {(0, 1), (0, 2), (1, -1), (1, -1), (1, -1)}} :
  Finset (Multiset (ℤ × ℤ))

```

lemma subset_le_mem_of_card_eq_six[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_six {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 6) (hS : S ≤ F) :
  S ∈ ({(0, 1), (0, 1), (0, 1), (1, -1), (1, -1), (1, -1)}} :
  Finset (Multiset (ℤ × ℤ))

```

lemma subset_le_mem_of_card_eq_seven[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_seven {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 7) (hS : S ≤ F) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma subset_le_mem_of_card_eq_seven_add[\[source\]](#)

```

lemma subset_le_mem_of_card_eq_seven_add {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) {n : ℕ} (hcard : S.card = 7 + n) (hS : S ≤ F) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma subset_le_mem_of_card_ge_seven[\[source\]](#)

```

lemma subset_le_mem_of_card_ge_seven {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : 7 ≤ S.card) (hS : S ≤ F) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))

```

lemma card_le_six_of_noExotics[\[source\]](#)

```

lemma card_le_six_of_noExotics {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero) :
  F.card ≤ 6

```

lemma card_mem_range_seven_of_noExotics[\[source\]](#)

```

lemma card_mem_range_seven_of_noExotics {F : FluxesFive} (hF : F.NoExotics) (hnZ : F.HasNoZero) :
  :
  F.card ∈ Finset.range 7

```

lemma FluxesFive.mem_elemsNoExotics_of_noExotics[\[source\]](#)

```
lemma mem_elemsNoExotics_of_noExotics (F : FluxesFive) (hNE : F.NoExotics) (hnZ : F.HasNoZero) :
  F ∈ elemsNoExotics
```

lemma FluxesFive.noExotics_iff_mem_elemsNoExotics[\[source\]](#)

Completness of elemsNoExotics, that is, every element of FluxesFive which obeys NoExotics is an element of elemsNoExotics, and every element of elemsNoExotics obeys NoExotics.

```
lemma noExotics_iff_mem_elemsNoExotics (F : FluxesFive) :
  F.NoExotics ∧ F.HasNoZero ↔ F ∈ elemsNoExotics
```

lemma FluxesTen.mem_mem_finset_of_noExotics[\[source\]](#)

```
lemma mem_mem_finset_of_noExotics (F : FluxesTen) (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (x : ℤ × ℤ) (hx : x ∈ F) :
  x ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (ℤ × ℤ))
```

lemma FluxesTen.subset_le_mem_of_card_eq_one[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_one {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 1) (hS : S ≤ F) :
  S ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesTen.subset_le_mem_of_card_eq_two[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_two {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 2) (hS : S ≤ F) :
  S ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesTen.subset_le_mem_of_card_eq_three[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_three {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 3) (hS : S ≤ F) :
  S ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (Multiset (ℤ × ℤ)))
```

lemma FluxesTen.subset_le_mem_of_card_eq_four[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_four {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : S.card = 4) (hS : S ≤ F) :
  S ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (Multiset (ℤ × ℤ)))
```

lemma subset_le_mem_of_card_eq_four_add[\[source\]](#)

```
lemma subset_le_mem_of_card_eq_four_add {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) {n : ℕ} (hcard : S.card = 4 + n) (hS : S ≤ F) :
  S ∈ ({(1, -1), (1, 0), (1, 1), (2, -1), (2, 0), (2, 1), (3, 0)} : Finset (Multiset (ℤ × ℤ)))
```

lemma subset_le_mem_of_card_ge_four[\[source\]](#)

```
lemma subset_le_mem_of_card_ge_four {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero)
  (S : Multiset (ℤ × ℤ)) (hcard : 4 ≤ S.card) (hS : S ≤ F) :
  S ∈ ({ } : Finset (Multiset (ℤ × ℤ)))
```

lemma card_le_three_of_noExotics[\[source\]](#)

```
lemma card_le_three_of_noExotics {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero) :
  F.card ≤ 3
```

lemma card_mem_range_four_of_noExotics[\[source\]](#)

```
lemma card_mem_range_four_of_noExotics {F : FluxesTen} (hF : F.NoExotics) (hnZ : F.HasNoZero) :
  F.card ∈ Finset.range 4
```

lemma FluxesTen.mem_elemsNoExotics_of_noExotics[\[source\]](#)

```
lemma mem_elemsNoExotics_of_noExotics (F : FluxesTen) (hNE : F.NoExotics) (hnZ : F.HasNoZero) :
  F ∈ elemsNoExotics
```

lemma FluxesTen.noExotics_iff_mem_elemsNoExotics[\[source\]](#)

Completeness of elemsNoExotics, that is, every element of FluxesTen which obeys NoExotics is an element of elemsNoExotics, and every element of elemsNoExotics obeys NoExotics.

```
lemma noExotics_iff_mem_elemsNoExotics (F : FluxesTen) :
  F.NoExotics ∧ F.HasNoZero ↔ F ∈ elemsNoExotics
```

2.45 PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.Elems

[PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/Elems.lean](#) on GitHub.

def FluxesFive.elemsNoExotics[\[source\]](#)

The elements of FluxesFive for which the NoExotics condition holds.

```
def elemsNoExotics : Multiset FluxesFive
```

lemma FluxesFive.elemsNoExotics_card[\[source\]](#)

```
lemma elemsNoExotics_card : elemsNoExotics.card = 31
```

lemma FluxesFive.elemsNoExotics_nodup[\[source\]](#)

```
lemma elemsNoExotics_nodup : elemsNoExotics.Nodup
```

lemma FluxesFive.noExotics_of_mem_elemsNoExotics[\[source\]](#)

```
lemma noExotics_of_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  NoExotics F
```

lemma toFinset_card_le_four_mem_elemsNoExotics[\[source\]](#)

```
lemma toFinset_card_le_four_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  F.toFinset.card ≤ 4
```

lemma FluxesFive.sum_of_mem_elemsNoExotics[\[source\]](#)

```
lemma sum_of_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  F.sum = (3, 0)
```

lemma FluxesFive.hasNoZero_of_mem_elemsNoExotics[\[source\]](#)

```
lemma hasNoZero_of_mem_elemsNoExotics (F : FluxesFive) (h : F ∈ elemsNoExotics) :
  F.HasNoZero
```

def FluxesTen elemsNoExotics[\[source\]](#)

The elements of FluxesTen for which the NoExotics condition holds.

```
def elemsNoExotics : Multiset FluxesTen
```

lemma FluxesTen elemsNoExotics_card[\[source\]](#)

```
lemma elemsNoExotics_card : elemsNoExotics.card = 6
```

lemma FluxesTen elemsNoExotics_nodup[\[source\]](#)

```
lemma elemsNoExotics_nodup : elemsNoExotics.Nodup
```

lemma FluxesTen.noExotics_of_mem_elemsNoExotics[\[source\]](#)

```
lemma noExotics_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :
  NoExotics F
```

lemma toFinset_card_le_three_mem_elemsNoExotics[\[source\]](#)

```
lemma toFinset_card_le_three_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :
  F.toFinset.card ≤ 3
```

lemma FluxesTen.sum_of_mem_elemsNoExotics[\[source\]](#)

```
lemma sum_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :
  F.sum = (3, 0)
```

lemma FluxesTen.hasNoZero_of_mem_elemsNoExotics[\[source\]](#)

```
lemma hasNoZero_of_mem_elemsNoExotics (F : FluxesTen) (h : F ∈ elemsNoExotics) :
  F.HasNoZero
```

2.46 PhysLean.StringTheory.FTheory.SU5.Fluxes.NoExotics.ToList

[PhysLean/StringTheory/FTheory/SU5/Fluxes/NoExotics/ToList.lean](#) on GitHub.

def FluxesFive.allowedPairs [\[source\]](#)

The allowed pairs of fluxes in FluxesFive which have no exotics.

This can be found using: (elemsNoExotics.bind (fun x => x.1)).toFinset

```
def allowedPairs : List (ℤ × ℤ)
```

lemma FluxesFive.mem_allowedPairs_count_one [\[source\]](#)

```
lemma mem_allowedPairs_count_one (x : ℤ × ℤ) (hx : x ∈ allowedPairs) :
  @List.count (ℤ × ℤ) instBEqOfDecidableEq x allowedPairs = 1
```

lemma FluxesFive.mem_allowedPairs_of_mem_noExotics [\[source\]](#)

```
lemma mem_allowedPairs_of_mem_noExotics (F : FluxesFive)
  (hf : F.NoExotics ∧ F.HasNoZero) (x : ℤ × ℤ)
  (h : x ∈ F) : x ∈ allowedPairs
```

def FluxesFive.toList [\[source\]](#)

Constructs a list of $\mathbb{Z} \times \mathbb{Z}$ from a $F : \text{FluxesFive}$, if F obeys `NoExotics` and `HasNoZero`, then this returns a list equivalent to the underlying multiset of F , else it returns junk.

```
def toList (F : FluxesFive) : List (ℤ × ℤ)
```

lemma FluxesFive.mem_allowedPairs_of_mem_toList [\[source\]](#)

```
lemma mem_allowedPairs_of_mem_toList (F : FluxesFive) (x : ℤ × ℤ)
  (h : x ∈ F.toList) : x ∈ allowedPairs
```

lemma FluxesFive.zero_not_mem_toList [\[source\]](#)

```
lemma zero_not_mem_toList (F : FluxesFive) : (0, 0) ∉ F.toList
```

lemma FluxesFive.mem_toList_iff [\[source\]](#)

```
lemma mem_toList_iff {F : FluxesFive}
  (hf : F.NoExotics ∧ F.HasNoZero) {a : ℤ × ℤ} : a ∈ F.toList ↔ a ∈ F
```

lemma FluxesFive.toList_count [\[source\]](#)

```
lemma toList_count {F : FluxesFive} {a : ℤ × ℤ} (hf : F.NoExotics ∧ F.HasNoZero)
  (hmema : a ∈ allowedPairs) : F.toList.count a = F.count a
```

lemma FluxesFive.coe_toList[\[source\]](#)

```
lemma coe_toList(F : FluxesFive) (h : F.NoExotics ∧ F.HasNoZero) : ↑(F.toList) = F
```

lemma FluxesFive.toList_length[\[source\]](#)

```
lemma toList_length (F : FluxesFive) (h : F.NoExotics ∧ F.HasNoZero) :  
  F.toList.length = F.card
```

def FluxesTen.allowedPairs[\[source\]](#)

The allowed pairs of fluxes in FluxesTen which have no exotics.

This can be found using: (elemsNoExotics.bind (fun x => x.1)).toFinset

```
def allowedPairs : List (ℤ × ℤ)
```

lemma FluxesTen.mem_allowedPairs_count_one[\[source\]](#)

```
lemma mem_allowedPairs_count_one (x : ℤ × ℤ) (hx : x ∈ allowedPairs) :  
  @List.count (ℤ × ℤ) instBEqOfDecidableEq x allowedPairs = 1
```

lemma FluxesTen.mem_allowedPairs_of_mem_noExotics[\[source\]](#)

```
lemma mem_allowedPairs_of_mem_noExotics (F : FluxesTen)  
  (hf : F.NoExotics ∧ F.HasNoZero) (x : ℤ × ℤ)  
  (h : x ∈ F) : x ∈ allowedPairs
```

def FluxesTen.toList[\[source\]](#)

*Constructs a list of $\mathbb{Z} \times \mathbb{Z}$ from a $F : \text{FluxesTen}$, if F is obeys *NoExotics* and *HasNoZero*, then this returns a list equivalent to the underlying multiset of F , else it returns junk.*

```
def toList (F : FluxesTen) : List (ℤ × ℤ)
```

lemma FluxesTen.mem_allowedPairs_of_mem_toList[\[source\]](#)

```
lemma mem_allowedPairs_of_mem_toList (F : FluxesTen) (x : ℤ × ℤ)  
  (h : x ∈ F.toList) : x ∈ allowedPairs
```

lemma FluxesTen.zero_not_mem_toList[\[source\]](#)

```
lemma zero_not_mem_toList (F : FluxesTen) : (0, 0) ∉ F.toList
```

lemma FluxesTen.mem_toList_iff[\[source\]](#)

```
lemma mem_toList_iff {F : FluxesTen}  
  (hf : F.NoExotics ∧ F.HasNoZero) {a : ℤ × ℤ} : a ∈ F.toList ↔ a ∈ F
```

lemma FluxesTen.toList_count[\[source\]](#)

```
lemma toList_count {F : FluxesTen} {a :  $\mathbb{Z} \times \mathbb{Z}$ } (hf : F.NoExotics  $\wedge$  F.HasNoZero)
  (hmema : a  $\in$  allowedPairs) : F.toList.count a = F.count a
```

lemma FluxesTen.coe_toList[\[source\]](#)

```
lemma coe_toList (F : FluxesTen) (h : F.NoExotics  $\wedge$  F.HasNoZero) :  $\uparrow$ (F.toList) = F
```

lemma FluxesTen.toList_length[\[source\]](#)

```
lemma toList_length (F : FluxesTen) (h : F.NoExotics  $\wedge$  F.HasNoZero) :
  F.toList.length = F.card
```

2.47 PhysLean.StringTheory.FTheory.SU5U1.Charges.OfRationalSection

[PhysLean/StringTheory/FTheory/SU5U1/Charges/OfRationalSection.lean](#) on GitHub.

lemma fiveChargeMultisetToList_length[\[source\]](#)

```
lemma fiveChargeMultisetToList_length (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ )
  (hs :  $\forall s \in S, s \in I.allowedBarFiveCharges$ ) :
  (fiveChargeMultisetToList I S).length = S.card
```

lemma tenChargeMultisetToList_length[\[source\]](#)

```
lemma tenChargeMultisetToList_length (I : CodimensionOneConfig) (S : Multiset  $\mathbb{Z}$ )
  (hs :  $\forall s \in S, s \in I.allowedTenCharges$ ) :
  (I.tenChargeMultisetToList S).length = S.card
```

2.48 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Completeness

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Completeness.lean](#) on GitHub.

lemma nonPhenoConstrainedCharges_insertQ10_filter[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_insertQ10_filter (I : CodimensionOneConfig) :
   $\forall (q10 : \{x // x \in I.allowedTenCharges\}),$ 
  Multiset.filter (fun x =>  $\neg$  IsPhenoConstrained x)
    ((nonPhenoConstrainedCharges I).uniqueMap4 (insert q10.1)).toMultiset =  $\emptyset$ 
```

lemma nonPhenoConstrainedCharges_insertQ5_filter[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_insertQ5_filter (I : CodimensionOneConfig) :
   $\forall (q5 : \{x // x \in I.allowedBarFiveCharges\}),$ 
  Multiset.filter (fun x =>  $\neg$  IsPhenoConstrained x)
    ((nonPhenoConstrainedCharges I).uniqueMap3 (insert q5.1)).toMultiset =  $\emptyset$ 
```

def completionTopYukawa[\[source\]](#)

The tree of charges which contains all completions of charges which minimally allow the top Yukawa, which are not pheno-constrained.

This can be constructed via

```
#eval FourTree.fromMultiset </| ((minimallyAllowsTermsOfFinset same.allowedBarFiveCharges
same.allowedTenCharges topYukawa).bind </| completions same.allowedBarFiveCharges
same.allowedTenCharges).dedup.filter fun x => ¬ IsPhenoConstrained x
```

```
private def completionTopYukawa (I : CodimensionOneConfig) :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

lemma completionTopYukawa_complete_of_same

[\[source\]](#)

```
lemma completionTopYukawa_complete_of_same :
  ∀ x ∈ (minimallyAllowsTermsOfFinset CodimensionOneConfig.same.allowedBarFiveCharges
  CodimensionOneConfig.same.allowedTenCharges topYukawa),
  ¬ x.IsPhenoConstrained →
  ∀ y ∈ completions same.allowedBarFiveCharges same.allowedTenCharges x, ¬ y.
  IsPhenoConstrained
  → y ∈ completionTopYukawa .same
```

lemma completionTopYukawa_complete_of_nearestNeighbor

[\[source\]](#)

```
lemma completionTopYukawa_complete_of_nearestNeighbor :
  ∀ x ∈ (minimallyAllowsTermsOfFinset
  CodimensionOneConfig.nearestNeighbor.allowedBarFiveCharges
  CodimensionOneConfig.nearestNeighbor.allowedTenCharges topYukawa),
  ¬ x.IsPhenoConstrained →
  ∀ y ∈ completions nearestNeighbor.allowedBarFiveCharges
  nearestNeighbor.allowedTenCharges x, ¬ y.IsPhenoConstrained
  → y ∈ completionTopYukawa .nearestNeighbor
```

lemma completionTopYukawa_complete_of_nextToNearestNeighbor

[\[source\]](#)

```
lemma completionTopYukawa_complete_of_nextToNearestNeighbor :
  ∀ x ∈ (minimallyAllowsTermsOfFinset
  CodimensionOneConfig.nextToNearestNeighbor.allowedBarFiveCharges
  CodimensionOneConfig.nextToNearestNeighbor.allowedTenCharges topYukawa),
  ¬ x.IsPhenoConstrained →
  ∀ y ∈ completions nextToNearestNeighbor.allowedBarFiveCharges
  nextToNearestNeighbor.allowedTenCharges x, ¬ y.IsPhenoConstrained
  → y ∈ completionTopYukawa .nextToNearestNeighbor
```

lemma completionTopYukawa_complete

[\[source\]](#)

```
lemma completionTopYukawa_complete {I : CodimensionOneConfig} (x : Charges)
  (hx : x ∈ (minimallyAllowsTermsOfFinset I.allowedBarFiveCharges
  I.allowedTenCharges topYukawa))
  (hPheno : ¬ x.IsPhenoConstrained) :
  ∀ y ∈ completions I.allowedBarFiveCharges I.allowedTenCharges x, ¬ y.IsPhenoConstrained
  → y ∈ completionTopYukawa I
```

lemma exists_subset_completionTopYukawa_of_not_isPhenoConstrained

[\[source\]](#)

```
lemma exists_subset_completionTopYukawa_of_not_isPhenoConstrained {x : Charges}
  (hx : ¬ x.IsPhenoConstrained)
  (htopYukawa : AllowsTerm x topYukawa)
```

```
(hsub : x ∈ ofFinset I.allowedBarFiveCharges I.allowedTenCharges)
(hcomplete : IsComplete x) : ∃ (y : Charges), y ∈ completionTopYukawa I ∧ y ⊆ x
```

lemma completionTopYukawa_subset_nonPhenoConstrainedCharges

[\[source\]](#)

```
lemma completionTopYukawa_subset_nonPhenoConstrainedCharges :
  ∀ x ∈ (completionTopYukawa I).toMultiset, x ∈ nonPhenoConstrainedCharges I
```

lemma not_isPhenoConstrained_mem_nonPhenoConstrainedCharges

[\[source\]](#)

```
lemma not_isPhenoConstrained_mem_nonPhenoConstrainedCharges {x : Charges}
  (hx : ¬ x.IsPhenoConstrained)
  (hsub : x ∈ ofFinset I.allowedBarFiveCharges I.allowedTenCharges)
  (hcomplete : IsComplete x) :
  x ∉ nonPhenoConstrainedCharges I → ¬ (¬ IsPhenoConstrained x ∧
    AllowsTerm x topYukawa)
```

lemma not_isPhenoConstrained_iff_mem_nonPhenoConstrainedCharge

[\[source\]](#)

```
lemma not_isPhenoConstrained_iff_mem_nonPhenoConstrainedCharge {x : Charges}
  (hsub : x ∈ ofFinset I.allowedBarFiveCharges I.allowedTenCharges) :
  AllowsTerm x topYukawa ∧
  ¬ IsPhenoConstrained x ∧ IsComplete x ↔
  x ∈ nonPhenoConstrainedCharges I
```

2.49 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenConstrained.Elems.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/Basic.lean](#) on GitHub.

def nonPhenoConstrainedChargesExt

[\[source\]](#)

For a given $I : \text{CodimensionOneConfig}$ the tree of charges containing all charges which are not phenomenologically constrained, and which permit a top Yukawa coupling. Unlike `nonPhenoConstrainedCharges`, these trees are calculated and therefore not good when using `decide` etc.

```
def nonPhenoConstrainedChargesExt (I : CodimensionOneConfig) :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def nonPhenoConstrainedCharges

[\[source\]](#)

For a given $I : \text{CodimensionOneConfig}$ the tree of charges containing all charges which are not phenomenologically constrained, and which permit a top Yukawa coupling.

These trees can be found with e.g. `#eval nonPhenoConstrainedChargesExt nextToNearestNeighbor`.

```
def nonPhenoConstrainedCharges : (I : CodimensionOneConfig) →
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
| same => nonPhenoConstrainedChargesSame
| nearestNeighbor => nonPhenoConstrainedChargesNearestNeighbor
| nextToNearestNeighbor => nonPhenoConstrainedChargesNextToNearestNeighbor
```

2.50 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Card

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/Card.lean](#) on GitHub.

lemma nonPhenoConstrainedCharges_same_card [\[source\]](#)

lemma nonPhenoConstrainedCharges_same_card : (nonPhenoConstrainedCharges same).card = 1148

lemma nonPhenoConstrainedCharges_nearestNeighbor_card [\[source\]](#)

lemma nonPhenoConstrainedCharges_nearestNeighbor_card :
(nonPhenoConstrainedCharges nearestNeighbor).card = 407

lemma nonPhenoConstrainedCharges_nextToNearestNeighbor_card [\[source\]](#)

lemma nonPhenoConstrainedCharges_nextToNearestNeighbor_card :
(nonPhenoConstrainedCharges nextToNearestNeighbor).card = 234

2.51 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.IsComplete

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/IsComplete.lean](#) on GitHub.

lemma isComplete_of_mem_nonPhenoConstrainedCharge_same [\[source\]](#)

lemma isComplete_of_mem_nonPhenoConstrainedCharge_same :
 $\forall x \in (\text{nonPhenoConstrainedCharges same}).\text{toMultiset}, \text{IsComplete } x$

lemma isComplete_of_mem_nonPhenoConstrainedCharge_nearestNeighbor [\[source\]](#)

lemma isComplete_of_mem_nonPhenoConstrainedCharge_nearestNeighbor :
 $\forall x \in (\text{nonPhenoConstrainedCharges nearestNeighbor}).\text{toMultiset}, \text{IsComplete } x$

lemma isComplete_of_mem_nonPhenoConstrainedCharge_nextToNearestNeighbor [\[source\]](#)

lemma isComplete_of_mem_nonPhenoConstrainedCharge_nextToNearestNeighbor :
 $\forall x \in (\text{nonPhenoConstrainedCharges nextToNearestNeighbor}).\text{toMultiset}, \text{IsComplete } x$

lemma isComplete_of_mem_nonPhenoConstrainedCharge [\[source\]](#)

lemma isComplete_of_mem_nonPhenoConstrainedCharge (I : CodimensionOneConfig) :
 $\forall x \in (\text{nonPhenoConstrainedCharges I}).\text{toMultiset}, \text{IsComplete } x$

2.52 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Lemmas

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/Lemmas.lean](#) on GitHub.

lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_same [\[source\]](#)

lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_same :
 $\forall x \in (\text{nonPhenoConstrainedCharges same}).\text{toMultiset}, x.2.2.1.\text{card} \leq 4$

lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_nearestNeighbor [\[source\]](#)

```
lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_nearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nearestNeighbor).toMultiset, x.2.2.1.card ≤ 4
```

lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_nextToNearestNeighbor [\[source\]](#)

```
lemma card_Q5_le_three_of_nonPhenoConstrainedCharges_of_nextToNearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nextToNearestNeighbor).toMultiset, x.2.2.1.card ≤ 4
```

lemma card_Q5_le_three_of_nonPhenoConstrainedCharges [\[source\]](#)

```
lemma card_Q5_le_three_of_nonPhenoConstrainedCharges (I : CodimensionOneConfig) :
  ∀ x ∈ (nonPhenoConstrainedCharges I).toMultiset, x.2.2.1.card ≤ 4
```

lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_same [\[source\]](#)

```
lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_same :
  ∀ x ∈ (nonPhenoConstrainedCharges same).toMultiset, x.2.2.2.card ≤ 3
```

lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_nearestNeighbor [\[source\]](#)

```
lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_nearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nearestNeighbor).toMultiset, x.2.2.2.card ≤ 3
```

lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_nextToNearestNeighbor [\[source\]](#)

```
lemma card_Q10_le_three_of_nonPhenoConstrainedCharges_of_nextToNearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nextToNearestNeighbor).toMultiset, x.2.2.2.card ≤ 3
```

lemma card_Q10_le_three_of_nonPhenoConstrainedCharges [\[source\]](#)

```
lemma card_Q10_le_three_of_nonPhenoConstrainedCharges (I : CodimensionOneConfig) :
  ∀ x ∈ (nonPhenoConstrainedCharges I).toMultiset, x.2.2.2.card ≤ 3
```

2.53 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenConstrained.Elems.NearestNeighbor

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/NearestNeighbor.lean](#) on GitHub.

def nonPhenoConstrainedChargesNearestNeighbor [\[source\]](#)

For $I = \text{nearestNeighbor}$ the tree of charges containing all charges which are not phenomenologically constrained, and which permit a top Yukawa coupling.

These trees can be found with e.g. `#eval nonPhenoConstrainedChargesExt nextToNearestNeighbor`.

```
def nonPhenoConstrainedChargesNearestNeighbor :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

2.54 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenConstrained.Elems.NextToNearestNeighbor

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/NextToNearestNeighbor.lean](#) on GitHub.

def nonPhenoConstrainedChargesNextToNearestNeighbor[\[source\]](#)

For $I = \text{nextToNearestNeighbor}$ the tree of charges containing all charges which are not phenomenologically constrained, and which permit a top Yukawa coupling.

These trees can be found with e.g. `#eval nonPhenoConstrainedChargesExt nextToNearestNeighbor`.

```
def nonPhenoConstrainedChargesNextToNearestNeighbor :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

2.55 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoConstrained

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/PhenoConstrained.lean](#) on GitHub.

lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_same[\[source\]](#)

```
lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_same :
  ∀ x ∈ (nonPhenoConstrainedCharges same).toMultiset, ¬ IsPhenoConstrained x
```

lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_nearestNeighbor[\[source\]](#)

```
lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_nearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nearestNeighbor).toMultiset, ¬ IsPhenoConstrained x
```

lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_nextToNearestNeighbor[\[source\]](#)

```
lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges_nextToNearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nextToNearestNeighbor).toMultiset,
  ¬ IsPhenoConstrained x
```

lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges[\[source\]](#)

```
lemma not_isPhenoConstrained_of_mem_nonPhenoConstrainedCharges (I : CodimensionOneConfig) :
  ∀ x ∈ (nonPhenoConstrainedCharges I).toMultiset, ¬ IsPhenoConstrained x
```

lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_same[\[source\]](#)

```
lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_same :
  ∀ x ∈ (nonPhenoConstrainedCharges same).toMultiset,
  AllowsTerm x topYukawa
```

lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_nearestNeighbor[\[source\]](#)

```
lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_nearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nearestNeighbor).toMultiset,
  AllowsTerm x topYukawa
```

lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_nextToNearestNeighbor[\[source\]](#)

```
lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges_nextToNearestNeighbor :
  ∀ x ∈ (nonPhenoConstrainedCharges nextToNearestNeighbor).toMultiset,
  AllowsTerm x topYukawa
```

lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges[\[source\]](#)

```

lemma allowsTerm_topYukawa_of_mem_nonPhenoConstrainedCharges (I : CodimensionOneConfig) :
  ∀ x ∈ (nonPhenoConstrainedCharges I).toMultiset,
    AllowsTerm x topYukawa

```

2.56 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoInsertQ10

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/PhenoInsertQ10.lean](#) on GitHub.

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_same[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_same : ∀ q10 ∈ same.allowedTenCharges
  ,
  (phenoInsertQ10 (nonPhenoConstrainedCharges same) q10).card = 0

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_nearestNeighbor[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_nearestNeighbor :
  ∀ q10 ∈ nearestNeighbor.allowedTenCharges,
  (phenoInsertQ10 (nonPhenoConstrainedCharges nearestNeighbor) q10).card = 0

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_nextToNearestNeighbor[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero_nextToNearestNeighbor :
  ∀ q10 ∈ nextToNearestNeighbor.allowedTenCharges,
  (phenoInsertQ10 (nonPhenoConstrainedCharges nextToNearestNeighbor) q10).card = 0

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ10_card_zero (I : CodimensionOneConfig) :
  ∀ q10 ∈ I.allowedTenCharges,
  (phenoInsertQ10 (nonPhenoConstrainedCharges I) q10).card = 0

```

2.57 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.PhenoInsertQ5

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/PhenoInsertQ5.lean](#) on GitHub.

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same_of_neg[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same_of_neg :
  ∀ q5 ∈ same.allowedBarFiveCharges,
  q5 < 0 → (phenoInsertQ5 (nonPhenoConstrainedCharges same) q5).card = 0

```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same_of_pos[\[source\]](#)

```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same_of_pos :
  ∀ q5 ∈ same.allowedBarFiveCharges,
  0 ≤ q5 → (phenoInsertQ5 (nonPhenoConstrainedCharges same) q5).card = 0

```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_same :
  ∀ q5 ∈ same.allowedBarFiveCharges,
    (phenoInsertQ5 (nonPhenoConstrainedCharges same) q5).card = 0
```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_nearestNeighbor[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_nearestNeighbor :
  ∀ q5 ∈ nearestNeighbor.allowedBarFiveCharges,
    (phenoInsertQ5 (nonPhenoConstrainedCharges nearestNeighbor) q5).card = 0
```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_nextToNearestNeighbor[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero_nextToNearestNeighbor :
  ∀ q5 ∈ nextToNearestNeighbor.allowedBarFiveCharges,
    (phenoInsertQ5 (nonPhenoConstrainedCharges nextToNearestNeighbor) (q5)).card = 0
```

lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero[\[source\]](#)

```
lemma nonPhenoConstrainedCharges_phenoInsertQ5_card_zero (I : CodimensionOneConfig) :
  ∀ q5 ∈ I.allowedBarFiveCharges,
    (phenoInsertQ5 (nonPhenoConstrainedCharges I) q5).card = 0
```

2.58 PhysLean.StringTheory.FTheory.SU5U1.Charges.PhenoConstrained.Elems.Same

[PhysLean/StringTheory/FTheory/SU5U1/Charges/PhenoConstrained/Elems/Same.lean](#) on GitHub.

def nonPhenoConstrainedChargesSame[\[source\]](#)

For $I = \text{same}$ the tree of charges containing all charges which are not phenomenologically constrained, and which permit a top Yukawa coupling.

These trees can be found with e.g. `#eval nonPhenoConstrainedChargesExt nextToNearestNeighbor`.

```
def nonPhenoConstrainedChargesSame :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

2.59 PhysLean.StringTheory.FTheory.SU5U1.Quanta.AnomalyCancellation

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/AnomalyCancellation.lean](#) on GitHub.

def FiveQuanta.anomalyCoefficient[\[source\]](#)

The anomaly coefficient of a `FiveQuanta` is given by the pair of integers: $(\sum_i q_i N_i, \sum_i q_i^2 N_i)$.

The first components is for the mixed $U(1)$ -MSSM, see equation (22) of [arXiv:1401.5084](#). The second component is for the mixed $U(1)$ - Y - $U(1)$ - $U(1)$ gauge anomaly, see equation (23) of [arXiv:1401.5084](#).

```
def FiveQuanta.anomalyCoefficient (F : FiveQuanta) : ℤ × ℤ
```

def TenQuanta.anomalyCoefficient[\[source\]](#)

The anomaly coefficient of a *TenQuanta* is given by the pair of integers: $(\sum_i q_i N_i, 3 * \sum_i q_i^2 N_i)$.

The first components is for the mixed $U(1)$ -MSSM, see equation (22) of [arXiv:1401.5084](#).
The second component is for the mixed $U(1)$ - Y - $U(1)$ - $U(1)$ gauge anomaly, see equation (23) of [arXiv:1401.5084](#).

```
def TenQuanta.anomalyCoefficient (F : TenQuanta) : ℤ × ℤ
```

def HdAnomalyCoefficient[\[source\]](#)

The pair of anomaly cancellation coefficients associated with the *Hd* particle.

```
def HdAnomalyCoefficient (qHd : Option ℤ) : ℤ × ℤ
```

def HuAnomalyCoefficient[\[source\]](#)

The pair of anomaly cancellation coefficients associated with the *Hu* particle.

```
def HuAnomalyCoefficient (qHu : Option ℤ) : ℤ × ℤ
```

def AnomalyCancellation[\[source\]](#)

The anomaly cancellation conditions on quanta making up the fields present in the theory. This corresponds to the conditions that:

- $\sum_i q_i N_i + \sum_a q_a N_a = 0$ where the first sum is over all 5-bar representations and the second is over all 10d representations.
- $\sum_i q_i^2 N_i + 3 * \sum_a q_a^2 N_a = 0$ where the first sum is over all 5-bar representations and the second is over all 10d representations.

```
def AnomalyCancellation (qHd qHu : Option ℤ) (F : FiveQuanta) (T : TenQuanta) : Prop
```

instance Decidable (AnomalyCancellation qHd qHu F T)[\[source\]](#)

```
instance : Decidable (AnomalyCancellation qHd qHu F T)
```

lemma anomalyCoefficient_snd_eq_zero_of_anomalyCancellation[\[source\]](#)

```
lemma anomalyCoefficient_snd_eq_zero_of_anomalyCancellation
  {qHd qHu : Option ℤ} {F : FiveQuanta} {T : TenQuanta} (h : AnomalyCancellation qHd qHu F T) :
  ((HdAnomalyCoefficient qHd).2 + (HuAnomalyCoefficient qHu).2
  + (F.anomalyCoefficient).2 + (T.anomalyCoefficient).2) = 0
```

lemma five_anomalyCoefficient_mod_three_zero_of_anomalyCancellation[\[source\]](#)

```
lemma five_anomalyCoefficient_mod_three_zero_of_anomalyCancellation
  {qHd qHu : Option ℤ} {F : FiveQuanta} {T : TenQuanta} (h : AnomalyCancellation qHd qHu F T) :
  ((HdAnomalyCoefficient qHd).2 + (HuAnomalyCoefficient qHu).2
  + (F.anomalyCoefficient).2) % 3 = 0
```

def Twig.filterAnomalyCancellation[\[source\]](#)

Given charges q_{Hd} and q_{Hu} filters a twig of $Q5$ and $Q10$ charges by the anomaly cancellation condition.

```
def Twig.filterAnomalyCancellation (I : CodimensionOneConfig) (qHd qHu : Option ℤ) :
  Twig (Finset ℤ) (Finset ℤ) → Twig (Finset ℤ) (Finset ℤ)
```

def Branch.filterAnomalyCancellation[\[source\]](#)

Given charges q_{Hd} filters a branch of q_{Hu} , $Q5$ and $Q10$ charges by the anomaly cancellation condition.

```
def Branch.filterAnomalyCancellation (I : CodimensionOneConfig) (qHd : Option ℤ) :
  Branch (Option ℤ) (Finset ℤ) (Finset ℤ) →
  Branch (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Trunk.filterAnomalyCancellation[\[source\]](#)

Filters a trunk of q_{Hd} , q_{Hu} , $Q5$ and $Q10$ charges by the anomaly cancellation condition.

```
def Trunk.filterAnomalyCancellation (I : CodimensionOneConfig) :
  Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ) →
  Trunk (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

def Tree.filterAnomalyCancellation[\[source\]](#)

Filters a tree of q_{Hd} , q_{Hu} , $Q5$ and $Q10$ charges by the anomaly cancellation condition.

```
def filterAnomalyCancellation (I : CodimensionOneConfig) :
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ) →
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
```

lemma mem_filterAnomalyCancellation_iff[\[source\]](#)

```
lemma mem_filterAnomalyCancellation_iff {I : CodimensionOneConfig}
  {T : FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)} {x : Charges} :
  x ∈ filterAnomalyCancellation I T ↔ x ∈ T ∧
  ∃ F : FiveQuanta, ∃ T : TenQuanta,
    F ∈ FiveQuanta.ofCharges I x.2.2.1.val ∧ T ∈ TenQuanta.ofCharges I x.2.2.2.val ∧
    AnomalyCancellation x.1 x.2.1 F T
```

2.60 PhysLean.StringTheory.FTheory.SU5U1.Quanta.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/Basic.lean](#) on GitHub.

abbrev FiveQuanta[\[source\]](#)

*The quanta of 5-bar representations corresponding to a multiset of (q, M, N) for each particle. (M, N) are defined in the *FluxesFive* module.*

```
abbrev FiveQuanta : Type
```

def toFluxesFive[\[source\]](#)*The underlying FluxesFive from a FiveQuanta.*

```
def toFluxesFive (x : FiveQuanta) : FluxesFive
```

def FiveQuanta.toCharges[\[source\]](#)*The underlying Multiset charges from a FiveQuanta.*

```
def toCharges (x : FiveQuanta) : Multiset  $\mathbb{Z}$ 
```

abbrev TenQuanta[\[source\]](#)*The quanta of $w0d$ representations corresponding to a multiset of (q, M, N) for each particle. (M, N) are defined in the FluxesFive module.*

```
abbrev TenQuanta : Type
```

def toFluxesTen[\[source\]](#)*The underlying FluxesTen from a TenQuanta.*

```
def toFluxesTen (x : TenQuanta) : FluxesTen
```

def TenQuanta.toCharges[\[source\]](#)*The underlying Multiset charges from a TenQuanta.*

```
def toCharges (x : TenQuanta) : Multiset  $\mathbb{Z}$ 
```

abbrev Quanta[\[source\]](#)*The quanta associated with the representations in a $SU(5) \times U(1)$ F-theory. This contains the value of the charges and the flux intergers (M, N) for the 5-bar matter content and the 10d matter content, and the charges of the H_d and H_u particles (there values of (M, N) are not included as they are forced to be $(0, 1)$ and $(0, -1)$ respectively.*

```
abbrev Quanta : Type
```

instance DecidableEq Quanta[\[source\]](#)

```
instance : DecidableEq Quanta
```

def Quanta.toCharges[\[source\]](#)*The underlying Charges of a Quanta.*

```
def toCharges (x : Quanta) : Charges
```

2.61 PhysLean.StringTheory.FTheory.SU5U1.Quanta.FromParts[PhysLean/StringTheory/FTheory/SU5U1/Quanta/FromParts.lean](#) on GitHub.

def FiveQuanta.fromParts[\[source\]](#)

The list of FiveQuanta which arise from charges and fluxes. Note: This gives junk unless the charges correspond to a CodimensionOneConfig and fluxes has no exotics and no zero entries.

```
def fromParts (I : CodimensionOneConfig) (charges : Multiset ℤ) (fluxes : FluxesFive) :
  List FiveQuanta
```

lemma self_mem_fromParts_of_toCharges_toFluxesFive[\[source\]](#)

```
lemma self_mem_fromParts_of_toCharges_toFluxesFive (I : CodimensionOneConfig) (F : FiveQuanta)
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges) :
  F ∈ fromParts I F.toCharges F.toFluxesFive
```

lemma FiveQuanta.card_eq_charges_card_of_mem_fromParts[\[source\]](#)

```
lemma card_eq_charges_card_of_mem_fromParts (I : CodimensionOneConfig) (charges : Multiset ℤ)
  (fluxes : FluxesFive) (hf : fluxes.NoExotics ∧ fluxes.HasNoZero)
  (hc : ∀ s ∈ charges, s ∈ I.allowedBarFiveCharges) :
  ∀ F ∈ fromParts I charges fluxes, F.card = charges.card
```

lemma FiveQuanta.card_eq_fluxes_card_of_mem_fromParts[\[source\]](#)

```
lemma card_eq_fluxes_card_of_mem_fromParts (I : CodimensionOneConfig) (charges : Multiset ℤ)
  (fluxes : FluxesFive) (hf : fluxes.NoExotics ∧ fluxes.HasNoZero)
  (hc : ∀ s ∈ charges, s ∈ I.allowedBarFiveCharges) :
  ∀ F ∈ fromParts I charges fluxes, F.card = fluxes.card
```

lemma FiveQuanta.fromParts_eq_preimage[\[source\]](#)

```
lemma fromParts_eq_preimage (I : CodimensionOneConfig) (charges : Multiset ℤ) (fluxes :
  FluxesFive)
  (hf : fluxes.NoExotics ∧ fluxes.HasNoZero)
  (hc : ∀ s ∈ charges, s ∈ I.allowedBarFiveCharges)
  (F : FiveQuanta) :
  F.toCharges = charges ∧ F.toFluxesFive = fluxes ↔
  F ∈ fromParts I charges fluxes
```

def FiveQuanta.ofCharges[\[source\]](#)

The multiset of FiveQuanta obtained from a multiset of charges Multiset ℤ, which have a FluxesFive in FluxesFive.elimsNoExotics.

```
def ofCharges (I : CodimensionOneConfig) (c : Multiset ℤ) : Multiset FiveQuanta
```

lemma FiveQuanta.mem_ofCharges_iff[\[source\]](#)

```
lemma mem_ofCharges_iff (I : CodimensionOneConfig) (charges : Multiset ℤ)
  (hc : ∀ s ∈ charges, s ∈ I.allowedBarFiveCharges) (F : FiveQuanta) :
  F ∈ ofCharges I charges ↔
  F.toCharges = charges ∧ F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero
```

lemma FiveQuanta.mem_ofCharges_self[\[source\]](#)

```

lemma mem_ofCharges_self (I : CodimensionOneConfig) (c : FiveQuanta)
  (h : c.toFluxesFive.NoExotics) (hnz : c.toFluxesFive.HasNoZero)
  (hc :  $\forall s \in c.toCharges, s \in I.allowedBarFiveCharges$ ) :
  c  $\in$  ofCharges I c.toCharges

```

def TenQuanta.fromParts[\[source\]](#)

The list of TenQuanta which arise from charges and fluxes. Note: This gives junk unless charges the charges correspond to a CodimensionOneConfig and fluxes has no exotics and no zero entries.

```

def fromParts (I : CodimensionOneConfig) (charges : Multiset  $\mathbb{Z}$ ) (fluxes : FluxesTen) :
  List TenQuanta

```

lemma self_mem_fromParts_of_toCharges_toFluxesTen[\[source\]](#)

```

lemma self_mem_fromParts_of_toCharges_toFluxesTen (I : CodimensionOneConfig) (F : TenQuanta)
  (hf : F.toFluxesTen.NoExotics  $\wedge$  F.toFluxesTen.HasNoZero)
  (hc :  $\forall s \in F.toCharges, s \in I.allowedTenCharges$ ) :
  F  $\in$  fromParts I F.toCharges F.toFluxesTen

```

lemma TenQuanta.card_eq_charges_card_of_mem_fromParts[\[source\]](#)

```

lemma card_eq_charges_card_of_mem_fromParts (I : CodimensionOneConfig) (charges : Multiset  $\mathbb{Z}$ )
  (fluxes : FluxesTen) (hf : fluxes.NoExotics  $\wedge$  fluxes.HasNoZero)
  (hc :  $\forall s \in charges, s \in I.allowedTenCharges$ ) :
   $\forall F \in$  fromParts I charges fluxes, F.card = charges.card

```

lemma TenQuanta.card_eq_fluxes_card_of_mem_fromParts[\[source\]](#)

```

lemma card_eq_fluxes_card_of_mem_fromParts (I : CodimensionOneConfig) (charges : Multiset  $\mathbb{Z}$ )
  (fluxes : FluxesTen) (hf : fluxes.NoExotics  $\wedge$  fluxes.HasNoZero)
  (hc :  $\forall s \in charges, s \in I.allowedTenCharges$ ) :
   $\forall F \in$  fromParts I charges fluxes, F.card = fluxes.card

```

lemma TenQuanta.fromParts_eq_preimage[\[source\]](#)

```

lemma fromParts_eq_preimage (I : CodimensionOneConfig) (charges : Multiset  $\mathbb{Z}$ ) (fluxes :
  FluxesTen)
  (hf : fluxes.NoExotics  $\wedge$  fluxes.HasNoZero)
  (hc :  $\forall s \in charges, s \in I.allowedTenCharges$ ) (F : TenQuanta) :
  F.toCharges = charges  $\wedge$  F.toFluxesTen = fluxes  $\leftrightarrow$ 
  F  $\in$  fromParts I charges fluxes

```

def TenQuanta.ofCharges[\[source\]](#)

The multiset of TenQuanta obtained from a multiset of charges Multiset $\mathbb{Z}$, which have a FluxesTen in FluxesTen.elmsNoExotics.

```

def ofCharges (I : CodimensionOneConfig) (c : Multiset  $\mathbb{Z}$ ) : Multiset TenQuanta

```

lemma TenQuanta.mem_ofCharges_iff[\[source\]](#)

```

lemma mem_ofCharges_iff (I : CodimensionOneConfig) (charges : Multiset ℤ)
  (hc : ∀ s ∈ charges, s ∈ I.allowedTenCharges) (F : TenQuanta) :
  F ∈ ofCharges I charges ↔
  F.toCharges = charges ∧ F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero

```

lemma TenQuanta.mem_ofCharges_self[\[source\]](#)

```

lemma mem_ofCharges_self (I : CodimensionOneConfig) (c : TenQuanta)
  (h : c.toFluxesTen.NoExotics) (hnz : c.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ c.toCharges, s ∈ I.allowedTenCharges) :
  c ∈ ofCharges I c.toCharges

```

def Quanta.ofCharges[\[source\]](#)

The multiset of Quanta obtained from a Charges, which have a Fluxes which do not permit exotics.

```

def ofCharges (I : CodimensionOneConfig) (c : Charges) : Multiset Quanta

```

2.62 PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Basic

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/IsViable/Basic.lean](#) on GitHub.

def IsViable[\[source\]](#)

For a given I : CodimensionOneConfig the condition on a Quanta for it to be phenomenologically viable.

```

def IsViable (I : CodimensionOneConfig) (x : Quanta) : Prop

```

lemma isViable_iff_expand_ofFinset[\[source\]](#)

```

lemma isViable_iff_expand_ofFinset (I : CodimensionOneConfig) (x : Quanta) :
  IsViable I x ↔
  x.toCharges.IsComplete ∧
  ¬ x.toCharges.IsPhenoConstrained ∧
  AllowsTerm x.toCharges topYukawa ∧
  x.2.2.1.toFluxesFive.NoExotics ∧
  x.2.2.1.toFluxesFive.HasNoZero ∧
  x.2.2.2.toFluxesTen.NoExotics ∧
  x.2.2.2.toFluxesTen.HasNoZero ∧
  AnomalyCancellation x.1 x.2.1 x.2.2.1 x.2.2.2 ∧
  (x.1.toFinset ⊆ I.allowedBarFiveCharges ∧ x.2.1.toFinset ⊆ I.allowedBarFiveCharges ∧
   x.toCharges.2.2.1 ⊆ I.allowedBarFiveCharges ∧ x.toCharges.2.2.2 ⊆ I.allowedTenCharges)
  ∧
  x.2.2.1.toCharges.Nodup ∧
  x.2.2.2.toCharges.Nodup

```

instance Decidable (IsViable I x)[\[source\]](#)

```

instance (I : CodimensionOneConfig) (x : Quanta) : Decidable (IsViable I x)

```

lemma toCharges_five_nodup_of_isViable[\[source\]](#)

```
lemma toCharges_five_nodup_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.1.toCharges.Nodup
```

lemma toCharges_ten_nodup_of_isViable[\[source\]](#)

```
lemma toCharges_ten_nodup_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.2.toCharges.Nodup
```

lemma toCharges_mem_ofFinset_of_isViable[\[source\]](#)

```
lemma toCharges_mem_ofFinset_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.toCharges ∈ ofFinset I.allowedBarFiveCharges I.allowedTenCharges
```

lemma toFluxesFive_noExotics_of_isViable[\[source\]](#)

```
lemma toFluxesFive_noExotics_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.1.toFluxesFive.NoExotics
```

lemma toFluxesTen_noExotics_of_isViable[\[source\]](#)

```
lemma toFluxesTen_noExotics_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.2.toFluxesTen.NoExotics
```

lemma toFluxesFive_hasNoZero_of_isViable[\[source\]](#)

```
lemma toFluxesFive_hasNoZero_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.1.toFluxesFive.HasNoZero
```

lemma toFluxesTen_hasNoZero_of_isViable[\[source\]](#)

```
lemma toFluxesTen_hasNoZero_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.2.toFluxesTen.HasNoZero
```

lemma Q10_charges_mem_allowedBarTenCharges_of_isViable[\[source\]](#)

```
lemma Q10_charges_mem_allowedBarTenCharges_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  ∀ s ∈ x.2.2.2.toCharges, s ∈ I.allowedTenCharges
```

lemma Q5_charges_mem_allowedBarFiveCharges_of_isViable[\[source\]](#)

```
lemma Q5_charges_mem_allowedBarFiveCharges_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
```

$\forall s \in x.2.2.1.toCharges, s \in I.allowedBarFiveCharges$

lemma fiveQuanta_mem_ofCharges_of_isViable

[\[source\]](#)

```
lemma fiveQuanta_mem_ofCharges_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.1 ∈ FiveQuanta.ofCharges I x.2.2.1.toCharges
```

lemma tenQuanta_mem_ofCharges_of_isViable

[\[source\]](#)

```
lemma tenQuanta_mem_ofCharges_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.2.2.2 ∈ TenQuanta.ofCharges I x.2.2.2.toCharges
```

lemma topYukawa_allowsTerm_of_isViable

[\[source\]](#)

```
lemma topYukawa_allowsTerm_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  AllowsTerm x.toCharges topYukawa
```

lemma not_isPhenoConstrained_of_isViable

[\[source\]](#)

```
lemma not_isPhenoConstrained_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  ¬ x.toCharges.IsPhenoConstrained
```

lemma toCharges_isComplete_of_isViable

[\[source\]](#)

```
lemma toCharges_isComplete_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.toCharges.IsComplete
```

lemma anomalyCancellation_of_isViable

[\[source\]](#)

```
lemma anomalyCancellation_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  AnomalyCancellation x.1 x.2.1 x.2.2.1 x.2.2.2
```

lemma toCharges_mem_nonPhenoConstrainedCharges_of_isViable

[\[source\]](#)

```
lemma toCharges_mem_nonPhenoConstrainedCharges_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.toCharges ∈ nonPhenoConstrainedCharges I
```

lemma toCharges_mem_nonPhenoConstrainedCharges_filterAnomalyCancellation_of_isViable

[\[source\]](#)

```
lemma toCharges_mem_nonPhenoConstrainedCharges_filterAnomalyCancellation_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.toCharges ∈ Charges.Tree.filterAnomalyCancellation I (nonPhenoConstrainedCharges I)
```

lemma mem_ofCharges_self_of_isViable[\[source\]](#)

```
lemma mem_ofCharges_self_of_isViable (I : CodimensionOneConfig) (Q : Quanta)
  (h : IsViable I Q) :
  Q ∈ ofCharges I Q.toCharges
```

2.63 PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Cases

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/IsViable/Cases.lean](#) on GitHub.

def caseI14a[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.a in table 1 of arXiv:1507.05961.

```
def caseI14a : Quanta
```

lemma caseI14a_mem_viableElems[\[source\]](#)

```
lemma caseI14a_mem_viableElems : caseI14a ∈ viableElems .same
```

def caseI14b[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.b in table 1 of arXiv:1507.05961.

```
def caseI14b : Quanta
```

lemma caseI14b_mem_viableElems[\[source\]](#)

```
lemma caseI14b_mem_viableElems : caseI14b ∈ viableElems .same
```

def caseI14c[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to one-version of example I.1.4.c in table 1 of arXiv:1507.05961.

```
def caseI14c : Quanta
```

lemma caseI14c_mem_viableElems[\[source\]](#)

```
lemma caseI14c_mem_viableElems : caseI14c ∈ viableElems .nearestNeighbor
```

def caseI14c'[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to one-version of example I.1.4.c in table 1 of arXiv:1507.05961.

```
def caseI14c' : Quanta
```

lemma caseI14c'_mem_viableElems[\[source\]](#)

```
lemma caseI14c'_mem_viableElems : caseI14c' ∈ viableElems .nearestNeighbor
```

def caseI14d[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.d in table 1 of arXiv:1507.05961.

```
def caseI14d : Quanta
```

lemma caseI14d_mem_viableElems[\[source\]](#)

```
lemma caseI14d_mem_viableElems : caseI14d ∈ viableElems .same
```

def caseI14e[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.e in table 1 of arXiv:1507.05961.

```
def caseI14e : Quanta
```

lemma caseI14e_mem_viableElems[\[source\]](#)

```
lemma caseI14e_mem_viableElems : caseI14e ∈ viableElems .nearestNeighbor
```

def caseI14f[\[source\]](#)

A case of Quanta with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.f in table 1 of arXiv:1507.05961.

```
def caseI14f : Quanta
```

lemma caseI14f_mem_viableElems[\[source\]](#)

```
lemma caseI14f_mem_viableElems : caseI14f ∈ viableElems .nextToNearestNeighbor
```

def caseI24a[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the two versions of I.2.4.a in table 8 of arXiv:1507.05961.

```
def caseI24a : Quanta
```

lemma caseI24a_mem_viableElems[\[source\]](#)

```
lemma caseI24a_mem_viableElems : caseI24a ∈ viableElems .same
```

def caseI24a'[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the two versions of the I.2.4.a in table 8 of arXiv:1507.05961.

```
def caseI24a' : Quanta
```

```
lemma caseI24a'_mem_viableElems
```

[\[source\]](#)

```
lemma caseI24a'_mem_viableElems : caseI24a' ∈ viableElems .same
```

```
def caseI24b
```

[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b : Quanta
```

```
lemma caseI24b_mem_viableElems
```

[\[source\]](#)

```
lemma caseI24b_mem_viableElems : caseI24b ∈ viableElems .same
```

```
def caseI24b'
```

[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b' : Quanta
```

```
lemma caseI24b'_mem_viableElems
```

[\[source\]](#)

```
lemma caseI24b'_mem_viableElems : caseI24b' ∈ viableElems .same
```

```
def caseI24b''
```

[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b'' : Quanta
```

```
lemma caseI24b''_mem_viableElems
```

[\[source\]](#)

```
lemma caseI24b''_mem_viableElems : caseI24b'' ∈ viableElems .same
```

```
def caseI24b'''
```

[\[source\]](#)

A case of Quanta with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b''' : Quanta
```

```
lemma caseI24b'''_mem_viableElems
```

[\[source\]](#)

```
lemma caseI24b'''_mem_viableElems : caseI24b''' ∈ viableElems .same
```

def caseI34a[\[source\]](#)

A case of Quanta with three 10d representations and 4 5-bar representations. This corresponds to example I.3.4.a of arXiv:1507.05961 (Eq. A12).

```
def caseI34a : Quanta
```

lemma caseI34a_mem_viableElems[\[source\]](#)

```
lemma caseI34a_mem_viableElems : caseI34a ∈ viableElems .same
```

2.64 PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Completeness

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/IsViable/Completeness.lean](#) on GitHub.

def elemsAnomalyFree[\[source\]](#)

For a given $I : \text{CodimensionOneConfig}$ the Charges which are not phenomenologically constrained and can be lifted to Quanta that is anomaly free.

The Trees here can be found using e.g. `#eval FourTree.fromMultiset (filterAnomalyCancellation .same (nonPhenoConstrainedCharges .same)).toMultiset`

```
def elemsAnomalyFree : (I : CodimensionOneConfig) →
  FourTree (Option ℤ) (Option ℤ) (Finset ℤ) (Finset ℤ)
| .same => root {trunk (some (-3)) {branch (some 0) {twig {-2, -1} {leaf {0}, leaf {-3, 0}}},
  branch (some 1) {twig {-2, 0} {leaf {-2, 3}}, twig {-2, 0, 3} {leaf {-2, 3}}}},
  trunk (some 2) {branch (some (-2)) {twig {-3, -1} {leaf {-3, -1}},
    twig {-1, 1} {leaf {-1}, leaf {-3, -1}}, twig {-3, -1, 1} {leaf {-1}, leaf {-3, -1}}},
    branch (some (-1)) {twig {0, 1} {leaf {-3, 2}}},
    branch (some 0) {twig {1, 3} {leaf {2, 0}}},
    branch (some 1) {twig {3} {leaf {0, 1}}}},
  trunk (some (-2)) {branch (some (-1)) {twig {-3} {leaf {-1, 0}}},
    branch (some 0) {twig {-3, -1} {leaf {-2, 0}}},
    branch (some 1) {twig {-1, 0} {leaf {-2, 3}}},
    branch (some 2) {twig {-1, 1} {leaf {1}, leaf {3, 1}},
      twig {1, 3} {leaf {3, 1}}, twig {-1, 1, 3} {leaf {1}, leaf {3, 1}}}},
  trunk (some (-1)) {branch (some 0) {twig {-3, -2, -1} {leaf {0}}},
    branch (some 2) {twig {0, 1} {leaf {1}, leaf {2, 1}, leaf {3, 1}, leaf {2, 3, 1}}}},
  trunk (some 1) {branch (some (-2)) {twig {-1, 0} {leaf {-1}, leaf {-3, -1}, leaf {-2, -1},
    leaf {-3, -2, -1}}},
    branch (some 0) {twig {1, 2, 3} {leaf {0}}}},
  trunk (some 3) {branch (some 0) {twig {1, 2} {leaf {0}, leaf {3, 0}}},
    branch (some (-1)) {twig {0, 2} {leaf {-3, 2}}, twig {-3, 0, 2} {leaf {-3, 2}}}},
| .nearestNeighbor => root {trunk (some (-9)) {branch (some (-14))
  {twig {-9, -4, 1} {leaf {-7}, leaf {-12, -7}}}},
  trunk (some 1) {branch (some (-14)) {twig {-9, -4} {leaf {-7}, leaf {-12, -7}}}},
  trunk (some 6) {branch (some (-14)) {twig {-9, 1} {leaf {-7}, leaf {-12, -7}},
    twig {-9, 1, 11} {leaf {-7}}, twig {-9, -4, 1} {leaf {-7}, leaf {-12, -7}}}},
| .nextToNearestNeighbor => root {trunk (some (-3)) {branch (some 12)
  {twig {2, 7} {leaf {6}, leaf {11, 6}}}}}
```

lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_same[\[source\]](#)

```
lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_same :
```

```
(filterAnomalyCancellation .same (nonPhenoConstrainedCharges .same)).toMultiset ⊆
(elemsAnomalyFree .same).toMultiset
```

lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_NN [\[source\]](#)

```
lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_NN :
  (filterAnomalyCancellation .nearestNeighbor
    (nonPhenoConstrainedCharges .nearestNeighbor)).toMultiset ⊆
  (elemsAnomalyFree .nearestNeighbor).toMultiset
```

lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_NTNN [\[source\]](#)

```
lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree_of_NTNN :
  (filterAnomalyCancellation .nextToNearestNeighbor
    (nonPhenoConstrainedCharges .nextToNearestNeighbor)).toMultiset ⊆
  (elemsAnomalyFree .nextToNearestNeighbor).toMultiset
```

lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree [\[source\]](#)

```
lemma filterAnomalyCancellation_nonPhenoConstrainedCharges_subset_elemsAnomalyFree :
  (filterAnomalyCancellation I (nonPhenoConstrainedCharges I)).toMultiset ⊆
  (elemsAnomalyFree I).toMultiset
```

lemma toCharges_mem_elemsAnomalyFree_of_isViable [\[source\]](#)

```
lemma toCharges_mem_elemsAnomalyFree_of_isViable
  (I : CodimensionOneConfig) (x : Quanta) (h : IsViable I x) :
  x.toCharges ∈ elemsAnomalyFree I
```

lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_same [\[source\]](#)

```
lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_same :
  ∀ x ∈ (elemsAnomalyFree .same).toMultiset, ∀ y ∈ Quanta.ofCharges .same x,
  AnomalyCancellation y.1 y.2.1 y.2.2.1 y.2.2.2 → y ∈ viableElems .same
```

lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_NN [\[source\]](#)

```
lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_NN :
  ∀ x ∈ (elemsAnomalyFree .nearestNeighbor).toMultiset, ∀ y ∈ Quanta.ofCharges .
  nearestNeighbor x,
  AnomalyCancellation y.1 y.2.1 y.2.2.1 y.2.2.2 → y ∈ viableElems .nearestNeighbor
```

lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_NTNN [\[source\]](#)

```
lemma mem_viableElems_of_ofCharges_elemsAnomalyFree_of_NTNN :
  ∀ x ∈ (elemsAnomalyFree .nextToNearestNeighbor).toMultiset,
  ∀ y ∈ Quanta.ofCharges .nextToNearestNeighbor x,
  AnomalyCancellation y.1 y.2.1 y.2.2.1 y.2.2.2 → y ∈ viableElems .nextToNearestNeighbor
```

lemma mem_viableElems_of_ofCharges[\[source\]](#)

```

lemma mem_viableElems_of_ofCharges (I : CodimensionOneConfig) :
  ∀ x ∈ (elemsAnomalyFree I).toMultiset,
  ∀ y ∈ Quanta.ofCharges I x, AnomalyCancellation y.1 y.2.1 y.2.2.1 y.2.2.2 →
  y ∈ viableElems I

```

lemma mem_viableElems_of_isViable[\[source\]](#)

```

lemma mem_viableElems_of_isViable
  (I : CodimensionOneConfig) (Q : Quanta) (h : IsViable I Q) :
  Q ∈ viableElems I

```

lemma isViable_iff_mem_viableElems[\[source\]](#)

```

lemma isViable_iff_mem_viableElems (I : CodimensionOneConfig) (Q : Quanta) :
  IsViable I Q ↔ Q ∈ viableElems I

```

2.65 PhysLean.StringTheory.FTheory.SU5U1.Quanta.IsViable.Elems

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/IsViable/Elems.lean](#) on GitHub.

def viableElemsExe[\[source\]](#)

The FourTree of elements of Quanta for which the IsViable condition holds for a given I : CodimensionOneConfig. This is an executable version used to generate viableElems, but not useful in proofs.

```

def viableElemsExe (I : CodimensionOneConfig) :
  FourTree (Option ℤ) (Option ℤ) FiveQuanta TenQuanta

```

def viableElems[\[source\]](#)

The FourTree of elements of Quanta for which the IsViable condition holds for a given I : CodimensionOneConfig.

Note, these results where calculated using e.g. set_option pp.deepTerms true #eval (viableElemsExe .nextToNearestNeighbor)

```

def viableElems : (I : CodimensionOneConfig) →
  FourTree (Option ℤ) (Option ℤ) FiveQuanta TenQuanta
| .same => root {trunk (some (-3)) {branch (some 0) {twig {(-2, 3, -3), (-1, 0, 3)}
  {leaf {(0, 3, 0)}, leaf {(-3, 1, 0)}, (0, 2, 0)}, leaf {(-3, 2, 0), (0, 1, 0)}}},
  branch (some 1) {twig {(-2, 2, -2), (0, 1, 2)}
    {leaf {(-2, 1, 0), (3, 2, 0)}, leaf {(-2, 2, 0), (3, 1, 0)}}},
  twig {(-2, 3, -2), (0, 0, 2)} {leaf {(-2, 1, 0), (3, 2, 0)}, leaf {(-2, 2, 0), (3, 1, 0)}},
  twig {(-2, 2, -2), (0, 0, 2), (3, 1, 0)} {leaf {(-2, 1, 0), (3, 2, 0)},
    leaf {(-2, 2, 0), (3, 1, 0)}}},
  trunk (some 2) {branch (some (-2)) {twig {(-3, 0, 3), (-1, 3, -3)}
    {leaf {(-3, 1, -1), (-1, 2, 1)}, leaf {(-3, 2, -1), (-1, 1, 1)}},
  twig {(-1, 1, 2), (1, 2, -2)} {leaf {(-1, 3, 0)}, leaf {(-3, 1, 0), (-1, 2, 0)},
    leaf {(-3, 2, 0), (-1, 1, 0)}},
  twig {(-1, 0, 2), (1, 3, -2)} {leaf {(-1, 3, 0)}, leaf {(-3, 1, 0), (-1, 2, 0)},
    leaf {(-3, 2, 0), (-1, 1, 0)}},
  twig {(-3, 1, 0), (-1, 0, 2), (1, 2, -2)} {leaf {(-1, 3, 0)}, leaf {(-3, 1, 0), (-1, 2, 0)}
  },

```

```

    leaf {(-3, 2, 0), (-1, 1, 0)}}},
  branch (some (-1)) {twig {(0, 0, 3), (1, 3, -3)} {leaf {(-3, 1, 0), (2, 2, 0)},
    leaf {(-3, 2, 0), (2, 1, 0)}}},
  branch (some 0) {twig {(1, 1, 2), (3, 2, -2)} {leaf {(0, 1, -1), (2, 2, 1)},
    leaf {(0, 2, -1), (2, 1, 1)}}},
  twig {(1, 0, 2), (3, 3, -2)} {leaf {(0, 1, -1), (2, 2, 1)}, leaf {(0, 2, -1), (2, 1, 1)}}},
  branch (some 1) {twig {(3, 3, 0)} {leaf {(0, 2, 1), (1, 1, -1)},
    leaf {(0, 1, 1), (1, 2, -1)}}},
  trunk (some (-2)) {branch (some (-1)) {twig {(-3, 3, 0)} {leaf {(-1, 1, -1), (0, 2, 1)},
    leaf {(-1, 2, -1), (0, 1, 1)}}},
  branch (some 0) {twig {(-3, 2, -2), (-1, 1, 2)} {leaf {(-2, 2, 1), (0, 1, -1)},
    leaf {(-2, 1, 1), (0, 2, -1)}}},
  twig {(-3, 3, -2), (-1, 0, 2)} {leaf {(-2, 2, 1), (0, 1, -1)}, leaf {(-2, 1, 1), (0, 2, -1)}}},
  branch (some 1) {twig {(-1, 3, -3), (0, 0, 3)} {leaf {(-2, 1, 0), (3, 2, 0)},
    leaf {(-2, 2, 0), (3, 1, 0)}}},
  branch (some 2) {twig {(-1, 2, -2), (1, 1, 2)} {leaf {(1, 3, 0)}, leaf {(1, 1, 0), (3, 2, 0)},
    leaf {(1, 2, 0), (3, 1, 0)}}},
  twig {(-1, 3, -2), (1, 0, 2)} {leaf {(1, 3, 0)}, leaf {(1, 1, 0), (3, 2, 0)},
    leaf {(1, 2, 0), (3, 1, 0)}}},
  twig {(1, 3, -3), (3, 0, 3)} {leaf {(1, 2, 1), (3, 1, -1)}, leaf {(1, 1, 1), (3, 2, -1)}}},
  twig {(-1, 2, -2), (1, 0, 2), (3, 1, 0)} {leaf {(1, 3, 0)}, leaf {(1, 1, 0), (3, 2, 0)},
    leaf {(1, 2, 0), (3, 1, 0)}}},
  trunk (some (-1)) {branch (some 0)
    {twig {(-3, 0, 1), (-2, 3, -3), (-1, 0, 2)} {leaf {(0, 3, 0)}}},

```

lemma isViable_of_mem_viableElems

[\[source\]](#)

Every element in viableElems isViable.

```

lemma isViable_of_mem_viableElems (I : CodimensionOneConfig) (x : Quanta)
  (hx : x ∈ viableElems I) : IsViable I x

```

lemma viableElems_nodups

[\[source\]](#)

```

lemma viableElems_nodups (I : CodimensionOneConfig) : (viableElems I).toMultiset.Nodup

```

lemma viableElems_same_card

[\[source\]](#)

```

lemma viableElems_same_card : (viableElems .same).card = 70

```

lemma viableElems_nearestNeighbor_card

[\[source\]](#)

```

lemma viableElems_nearestNeighbor_card : (viableElems .nearestNeighbor).card = 16

```

lemma viableElems_nextToNearestNeighbor_card

[\[source\]](#)

```

lemma viableElems_nextToNearestNeighbor_card : (viableElems .nextToNearestNeighbor).card = 3

```

2.66 PhysLean.StringTheory.FTheory.SU5U1.Quanta.ToList

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/ToList.lean](#) on GitHub.

def FiveQuanta.allowedElems[\[source\]](#)

The allowed elements of a FiveQuanta for which the underlying FluxesFive obeys NoExotics and HasNoZero and for which the charges are determined by a CodimensionOneConfig.

```
def allowedElems (I : CodimensionOneConfig) : List (ℤ × ℤ × ℤ)
```

lemma FiveQuanta.mem_allowedElems_of_mem[\[source\]](#)

```
lemma mem_allowedElems_of_mem (F : FiveQuanta)
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F) : x ∈ allowedElems I
```

def FiveQuanta.toList[\[source\]](#)

A list from a FiveQuanta which when the underlying FluxesFive obeys NoExotics and HasNoZero, and the charges are determined by a CodimensionOneConfig, is a list whose underlying multiset is the FiveQuanta. Otherwise this list produces junk.

```
def toList (I : CodimensionOneConfig) (F : FiveQuanta) : List (ℤ × ℤ × ℤ)
```

lemma FiveQuanta.mem_allowedElems_of_mem_toList[\[source\]](#)

```
lemma mem_allowedElems_of_mem_toList (I : CodimensionOneConfig) (F : FiveQuanta)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F.toList I) : x ∈ allowedElems I
```

lemma FiveQuanta.snd_mem_allowedPairs_of_mem_toList[\[source\]](#)

```
lemma snd_mem_allowedPairs_of_mem_toList (I : CodimensionOneConfig) (F : FiveQuanta)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F.toList I) : x.2 ∈ FluxesFive.allowedPairs
```

lemma FiveQuanta.mem_toList_iff[\[source\]](#)

```
lemma mem_toList_iff {I : CodimensionOneConfig} (F : FiveQuanta)
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges) {a : ℤ × ℤ × ℤ} :
  a ∈ F.toList I ↔ a ∈ F
```

lemma FiveQuanta.toList_count[\[source\]](#)

```
lemma toList_count {I : CodimensionOneConfig} {F : FiveQuanta}
  {a : ℤ × ℤ × ℤ}
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges)
  (hmema : a ∈ allowedElems I) :
  (F.toList I).count a = F.count a
```

lemma FiveQuanta.coe_toList[\[source\]](#)

```
lemma coe_toList {I : CodimensionOneConfig} {F : FiveQuanta}
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges) :
```

```
↑(F.toList I) = F
```

lemma FiveQuanta.toList_prod_fst_eq

[\[source\]](#)

```
lemma toList_prod_fst_eq {I : CodimensionOneConfig} {F : FiveQuanta}
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges) :
  (F.toList I).map Prod.fst = I.fiveChargeMultisetToList F.toCharges
```

lemma FiveQuanta.toList_prod_snd_perm

[\[source\]](#)

```
lemma toList_prod_snd_perm {I : CodimensionOneConfig} {F : FiveQuanta}
  (hf : F.toFluxesFive.NoExotics ∧ F.toFluxesFive.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedBarFiveCharges) :
  ((F.toList I).map Prod.snd).Perm (FluxesFive.toList F.toFluxesFive)
```

def TenQuanta.allowedElems

[\[source\]](#)

The allowed elements of a FiveQuanta for which the underlying FluxesFive obeys NoExotics and HasNoZero and for which the charges are determined by a CodimensionOneConfig.

```
def allowedElems (I : CodimensionOneConfig) : List (ℤ × ℤ × ℤ)
```

lemma TenQuanta.mem_allowedElems_of_mem

[\[source\]](#)

```
lemma mem_allowedElems_of_mem {I : CodimensionOneConfig} (F : TenQuanta)
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F) : x ∈ allowedElems I
```

def TenQuanta.toList

[\[source\]](#)

A list from a TenQuanta which when the underlying FluxesTen obeys NoExotics and HasNoZero, and the charges are determined by a CodimensionOneConfig, is a list whose underlying multiset is the TenQuanta. Otherwise this list produces junk.

```
def toList (I : CodimensionOneConfig) (F : TenQuanta) : List (ℤ × ℤ × ℤ)
```

lemma TenQuanta.mem_allowedElems_of_mem_toList

[\[source\]](#)

```
lemma mem_allowedElems_of_mem_toList (I : CodimensionOneConfig) (F : TenQuanta)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F.toList I) : x ∈ allowedElems I
```

lemma TenQuanta.snd_mem_allowedPairs_of_mem_toList

[\[source\]](#)

```
lemma snd_mem_allowedPairs_of_mem_toList (I : CodimensionOneConfig) (F : TenQuanta)
  (x : ℤ × ℤ × ℤ) (h : x ∈ F.toList I) : x.2 ∈ FluxesTen.allowedPairs
```

lemma TenQuanta.mem_toList_iff

[\[source\]](#)

```
lemma mem_toList_iff {I : CodimensionOneConfig} (F : TenQuanta)
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
```

```
(hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges) {a : ℤ × ℤ × ℤ} :
a ∈ F.toList I ↔ a ∈ F
```

lemma TenQuanta.toList_count[\[source\]](#)

```
lemma toList_count {I : CodimensionOneConfig} {F : TenQuanta}
  {a : ℤ × ℤ × ℤ}
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges)
  (hmema : a ∈ allowedElems I) :
  (F.toList I).count a = F.count a
```

lemma TenQuanta.coe_toList[\[source\]](#)

```
lemma coe_toList {I : CodimensionOneConfig} {F : TenQuanta}
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges) :
  ↑(F.toList I) = F
```

lemma TenQuanta.toList_prod_fst_eq[\[source\]](#)

```
lemma toList_prod_fst_eq {I : CodimensionOneConfig} {F : TenQuanta}
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges) :
  (F.toList I).map Prod.fst = I.tenChargeMultisetToList F.toCharges
```

lemma TenQuanta.toList_prod_snd_perm[\[source\]](#)

```
lemma toList_prod_snd_perm {I : CodimensionOneConfig} {F : TenQuanta}
  (hf : F.toFluxesTen.NoExotics ∧ F.toFluxesTen.HasNoZero)
  (hc : ∀ s ∈ F.toCharges, s ∈ I.allowedTenCharges) :
  ((F.toList I).map Prod.snd).Perm (FluxesTen.toList F.toFluxesTen)
```

2.67 PhysLean.StringTheory.FTheory.SU5U1.Quanta.YukawaRegeneration

[PhysLean/StringTheory/FTheory/SU5U1/Quanta/YukawaRegeneration.lean](#) on GitHub.

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_same[\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_same :
  (viableElems same).toMultiset.filter (fun x =>
    ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 1)
  = {(some 2, some (-2), {(-1, 1, 2), (1, 2, -2)}, {(-1, 3, 0)}),
    (some 2, some (-2), {(-1, 0, 2), (1, 3, -2)}, {(-1, 3, 0)}),
    (some (-2), some 2, {(-1, 2, -2), (1, 1, 2)}, {(1, 3, 0)}),
    (some (-2), some 2, {(-1, 3, -2), (1, 0, 2)}, {(1, 3, 0)})}
```

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_NN[\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_NN :
  (viableElems nearestNeighbor).toMultiset.filter
    (fun x => ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 1)
  = {(some 6, some (-14), {(-9, 1, 2), (1, 2, -2)}, {(-7, 3, 0)})}
```

```
(some 6, some (-14), {(-9, 0, 2), (1, 3, -2)}, {(-7, 3, 0)}))
```

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_NtoNN [\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_one_eq_of_NtoNN :
  (viableElems nextToNearestNeighbor).toMultiset.filter
    (fun x => ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 1) = ∅
```

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_same [\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_same :
  (viableElems same).toMultiset.filter (fun x =>
    ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 2) = ∅
```

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_NN [\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_NN :
  (viableElems nearestNeighbor).toMultiset.filter (fun x =>
    ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 2) = ∅
```

lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_NtoNN [\[source\]](#)

```
lemma viableElems_filter_yukawaSingletsRegenerateDangerousInsertion_two_eq_of_NtoNN :
  (viableElems nextToNearestNeighbor).toMultiset.filter (fun x =>
    ¬ (toCharges x).YukawaGeneratesDangerousAtLevel 2) = ∅
```

2.68 scripts.MetaPrograms.regex_lint

[scripts/MetaPrograms/regex_lint.lean](#) on GitHub.

def unneededParentheses

[\[source\]](#)

Unneeded parentheses.

```
def unneededParentheses : PhysLeanTextLinter
```

2.69 scripts.MetaPrograms.TODO_to_yaml

[scripts/MetaPrograms/TOD0_to_yaml.lean](#) on GitHub.

def toFullTOD0Info

[\[source\]](#)

```
def SorryfulInfo.toFullTOD0Info (s : SorryfulInfo) : FullTOD0Info where
```

def allSorryfulResults

[\[source\]](#)

```
unsafe def allSorryfulResults : CoreM (Array FullTOD0Info)
```